

**A
PROJECT REPORT
ON**

“PLANTREE E-COMMERCE APPLICATION”

**Submitted
To**

**UDHNA CITIZEN COMMERCE COLLEGE &
S.P.B. COLLEGE OF BUSINESS ADMINISTRATION &
SMT. DIWALIBEN HARJIBHAI GONDALIA COLLEGE OF BCA AND IT**

**Affiliated
To**

VEER NARMAD SOUTH GUJARAT UNIVERSITY, SURAT

**As A Partial Fulfillment For The Degree Of
BACHELOR OF COMPUTER APPLICATION
(B.C.A.)**

**T.Y.B.C.A. (SEM. - 6)
ACADEMIC YEAR: 2022-23**

DEVELOPED BY:

**Mr. DUBEY ANIKET (Seat no: - 1669)
Mr. OJHA MANISH (Seat no: - 1630)
Mr. KESHARWANI ADITYA (Seat no: - 1543)
Mr. SUTHAR BHAVIK (Seat no: - 3437)**

GUIDED BY:

Dr. Anil K. Bharodiya

:: PROJECT DEVELOPED AT::

**UCCC & SPBCBA & SDHG COLLEGE OF BCA AND IT
[SURAT]**



**Udhna Citizen Commerce College
&
S.P.B. College of Business Administration
&**

Smt. Diwaliben Harjibhai Gondalia College of BCA & I.T.

(Self Financed College Affiliated To Veer Narmad South Gujarat University, Surat)

(Managed by Udhna Academy Education Trust)

(Bachelor of Computer Application)

CERTIFICATE

This is to Certify that Mr. / Ms. **DUBEY ANIKET DEVENDRA**

of BCA-6th Semester Seat No..... **1669** *has successfully carried out a Project /*

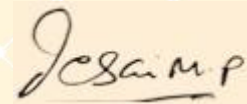
Seminar Work entitled as..... **PLANTREE E-COMMERCE APPLICATION**

*in accrodance with technical and theoretical specification for the academic
year 20.2220.23 towards a Partial fulfillment of BCA Programme.*

Guide Name : Dr. Anil K. Bharodiya **Examiner Name :**

Signature :  **Signature :**

Date : **15-04-2023**



Principal



Udhna Citizen Commerce College

&

S.P.B. College of Business Administration

&

Smt. Diwaliben Harjibhai Gondalia College of BCA & I.T.

(Self Financed College Affiliated To Veer Narmad South Gujarat University, Surat)

(Managed by Udhna Academy Education Trust)

(Bachelor of Computer Application)

CERTIFICATE

This is to Certify that Mr. / Ms. **OJHA MANISH KUMAR**

of BCA-6th Semester Seat No..... **1630** *has successfully carried out a Project /*

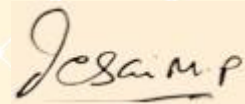
Seminar Work entitled as..... **PLANTREE E-COMMERCE APPLICATION**

*in accrodance with technical and theoretical specification for the academic
year 20.2220.23 towards a Partial fulfillment of BCA Programme.*

Guide Name : Dr. Anil K. Bharodiya **Examiner Name :**

Signature :  **Signature :**

Date : 15-04-2023



Principal



**Udhna Citizen Commerce College
&
S.P.B. College of Business Administration
&**



Smt. Diwaliben Harjibhai Gondalia College of BCA & I.T.

(Self Financed College Affiliated To Veer Narmad South Gujarat University, Surat)

(Managed by Udhna Academy Education Trust)

(Bachelor of Computer Application)

CERTIFICATE

This is to Certify that Mr. / Ms. **KESHARWANI ADITYA VIJAYKUMAR**

of BCA-6th Semester Seat No. **1543** *has successfully carried out a Project /*

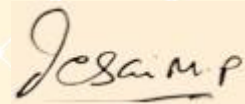
Seminar Work entitled as. **PLANTREE E-COMMERCE APPLICATION**

*in accordance with technical and theoretical specification for the academic
year 20.2220.23 towards a Partial fulfillment of BCA Programme.*

Guide Name : Dr. Anil K. Bharodiya **Examiner Name :**

Signature :  **Signature :**

Date : **15-04-2023**



Principal



**Udhna Citizen Commerce College
&
S.P.B. College of Business Administration
&**



Smt. Diwaliben Harjibhai Gondalia College of BCA & I.T.

(Self Financed College Affiliated To Veer Narmad South Gujarat University, Surat)

(Managed by Udhna Academy Education Trust)

(Bachelor of Computer Application)

CERTIFICATE

This is to Certify that Mr. / Ms. **SUTHAR BHAVIK ASHOKBHAI**

of BCA-6th Semester Seat No..... **3437** *has successfully carried out a Project /*

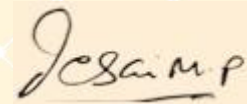
Seminar Work entitled as..... **PLANTREE E-COMMERCE APPLICATION**

*in accrodance with technical and theoretical specification for the academic
year 20.222023 towards a Partial fulfillment of BCA Programme.*

Guide Name : Dr. Anil K. Bharodiya **Examiner Name :**

Signature :  **Signature :**

Date : **15-04-2023**



Principal

ACKNOWLEDGEMENT

Success in such a comprehensive project cannot be achieved single-handedly. It is team effort that sails the ship to the coast. So we would like to express our sincere thanks to all the dignitaries who were involved in making this project a great joy and turning it into successful Piece of work.

We would like to take opportunity to thank our UCCC & SPBCBA & SDHG College Of BCA & I.T. and our principal Dr. Manish M Kayasth for giving us this tremendous opportunity to work in the real-time project.

Dr. Anil K. Bharodiya, our professor and project co-ordinator, has been very prudent to us throughout our college studies. He is the person who has been giving direction to our work and the shape to our imagination. We express our regards to her from the core of our heart.

We would also like to thank all our professors who are always ready to give best guidance. They are the individuals who give solutions whenever required.

We would also like to acknowledge all our friends and colleagues, team members for their help and encouragement from time to time. The constant support and encouragement of our friends deeply appreciates. The project indeed gave challenging and exhilarating experience in designing and developing the required system.

Finally we would like to thank our Parents for their support throughout the project. We owe a special debt to our family & friends for their supports blessing and encouragement for me.

THANKING ALL.....

DUBEY ANIKET

MANISH OJHA

KESHARWANI ADITYA

SUTHAR BHAVIK

INDEX

<u>Chapter No.</u>	<u>Chapter Name</u>	<u>Page No.</u>
1.	College Profile	01
	1.1 Brief Overview / Highlights	01
	1.2 Institute Structure / Chart	04
2.	Manual/Current / Existing System Study	05
	2.1 Major Components / Flow	05
	2.2 Minimum Hardware / Software Configuration	06
	2.3 Drawbacks / Limitations	08
3.	Propose Project Profile	09
	3.1 Introduction	09
	3.2 Objective / Goal / Aim	10
	3.3 Scope	11
	3.4 Type of Project	12
	3.5 Technology / Environments E.g. tools	13
	3.6 Applicability of the system	14
4.	Software Analysis	15
	4.1 Preliminary Investigation	15
	4.2 Problem Identification	16
	4.3 Feasibility study / Risk Analysis	17
	4.3.1 Technical Feasibility	17
	4.3.2 Economical Feasibility	17
	4.3.3 Operational Feasibility	17
	4.3.4 Management Feasibility	17
	4.3.5 Time Feasibility	17
	4.4 Requirement Analysis	18
	4.4.1 Fact Finding Techniques	19
	4.4.2 Time Line Chart	20
	4.4.3 Model with Justification/ Agile Modeling	22
	4.4.4 Flow Chart	25
	4.4.5 DFD and/or UML	35
	4.4.6 Process / Control Specification	41
	4.4.7 Data Dictionary	43

4.	4.4.8 E-R Diagram	46
	4.4.9 Data Object Description	48
5.	Software Design	49
	5.1 Project Design Process Hierarchy	49
	5.2 Database Design	50
	5.2.1 Justification of Normalization	54
	5.3 Architectural Design	56
	5.4 Algorithm Development / Pseudo-code	57
	5.5 User Interface Design	59
	5.6 Security Issues	81
	5.7 Quality / Reliability Measures	82
	5.8 System Map	83
6.	Software Coding	85
	6.1 Tools & Techniques	85
	6.2 Business Logic	86
	6.3 Result Snapshot	122
	6.4 System generated Reports	132
7.	Software Testing	133
	7.1 Test Cases & Test Data Design	133
	7.2 Output Comparison	134
	7.3 Testing Strategies	139
	7.4 Unit Testing	140
	7.5 Integration Testing	141
	7.6 System Testing	142
	7.7 Alpha Testing	143
8.	Software Implementation	144
	8.1 User Training	144
	8.2 User Manual / Help / SOP	145
9.	Limitations / Constraints	146
10.	Future Enhancement / Path-A-Head	147
11.	Bibliography / Appendix / References	148
12.	Other Software Engineering Principles / Tools / Techniques / Models / Guidelines	149



1. COLLEGE PROFILE

1.1. Brief Overview / Highlights

UDHANA ACADEMY EDUCATION TRUST



COLLEGE INFORMATION

UCCC & SPBCBA & SDHG college of bca and IT
(Managed by Udhana Academy Education Trust, Udhna)
B.C.A. Programme

214, Ranchhod Nagar, Opp. Swami Narayan Temple, Udhna - Navsari
Main Rd, Udhna, Surat, Gujarat 394210, India

www.udhnacitizen.org | uaccait@gmail.com | 0261-2277739



The Udhna Academy Education Trust Has Been Registered As An Educational Institute Of Gujarat Since June 1964. Our Motto Is All Round Quality Education. The Student Should Become A Valuable Service Provider By Sharing His/Her Knowledge In The Society And Gain A Place At The Highest Peak In His/Her Area Through Technical Professional Skill.

It Is A Premier Education Institute And A Pioneer Among Self-Financed Degree Colleges From June 2002. The Trust Was Established In 1964 With The Objective Of Catering To The Educational Needs Of The Citizens Of Udhna Area And South Gujarat. Since Its Inception Over Four Decades Ago, It Has Spread The Light Of Education In This Region Providing Education Ranging From Pre-Primary To Higher Secondary And College Level. Its Pioneers Started This Institution With A Very Noble Aim And Far Reaching Vision.



:: Goal & Objectives ::

GOAL

- Our goal is 'Excellence through performance', based on the principle, 'Intelligence and purposeful practice improve performance'.

OBJECTIVES

- To help the students develop their knowledge, skill and attitude for accepting challenges in various aspects of economy;
- To contribute to the society by providing competent human capital with relevant skill;
- To provide platform for the student to fulfil their dreams successfully; and
- To transform their individual personality from a mere student to a responsible and committed citizen.

OUR VISION

"To be an eminent and vibrant institute for education, our credo will always be excellence through innovations, sympathy, ethics and teamwork and to cater to the ever changing needs of community at large".

Mission

"To impart quality education, nurture aspirations and facilitate continuous learning and to contribute to the society by developing outstanding individuals who would take up leadership challenges in various sectors of economy.



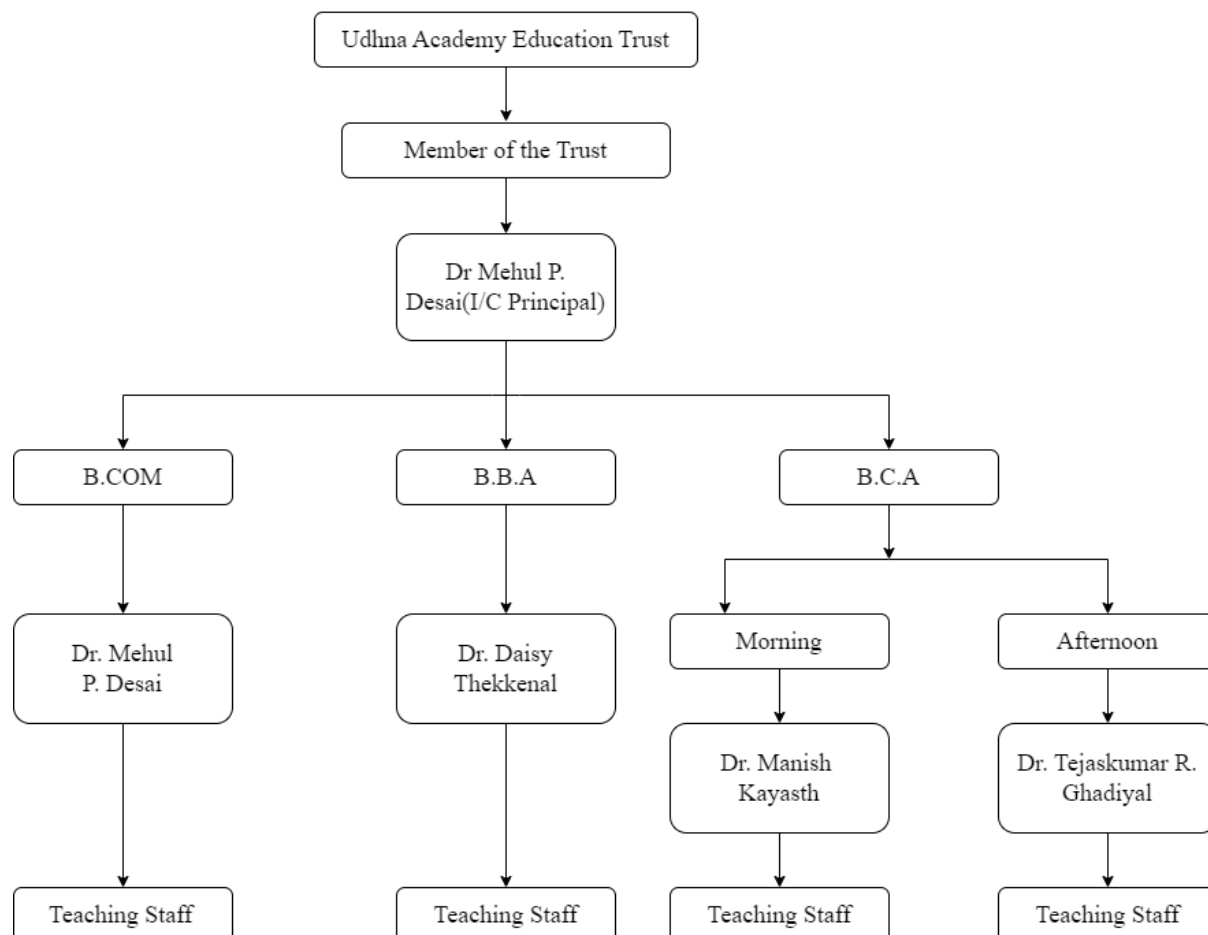
PLANTREE E-COMMERCE APPLICATION

Salient Features

- Qualified and Experienced faculty members
- Book Bank Facility
- Well-equipped Text and References Library
- Strong Industry- Institute Interaction through seminars, Guest Lectures, Projects, Visits.
- Faculty Feedback System to strengthen Teaching-Learning Process
- Indoor and outdoor Co-Curricular & Extra-curricular Activities
- Social Welfare Initiatives in Plantation of trees, Blood Donation Camp, NSS Camp and relief during Natural Calamities
- NSS Sports as Character Building Activities
- Tie-up with Health centre for free medical service to all students and staff
- Well equipped computers laboratories with Broadband Internet Connection
- Canteen Facility for Students and staff
- Scholarship to toppers in academics, extra-curricular activities and sports
- Anti-ragging Cell
- Women's Cell
- Placement Cell
- Smart Class



1.2 Institution Structure / Chart



SMT DIWALIBEN HARJIBHAI GONDALIA COLLEGE OF B.C.A & I.T. (B.C.A)

JUNE 2008 S.P.B COLLEGE OF BUSINESS ADMINISTRATION (B.B.A.) JUNE 2005

UDHNA CITIZEN COMMERCE COLLEGE (B.com.) JUNE 2002

R.N. NAIK H.S. SCHOOL (SCIENCE) - JUNE 2001

R.N. NAIK H.S. SCHOOL (COM.& ARTS) - JUNE 2001

UDHNA ACADEMY SHISHUVIHAR-2- JUNE 1998

SMT. MADHUKANTABEN J. MEHTA PRIMARY SCHOOL-JUNE 1998

UDHNA ACADEMY SHISHUVIHAR-2- JUNE 1998

SURAT NATIONAL PRIMARY SCHOOL- JUNE 1968 R.N. NAIK HIGH SCHOOL - JUNE 1964



2. CURRENT / EXISTING SYSTEM STUDY

2.1 Major Components

- Many people want to buy plants and they directly concerned to the nursery and buy the plants but sometimes people doesn't know specific information about particular plant items as well as seller which are not technically skilled.
- Customer does not compare plant price with other shopkeepers at the same time. In nursery there is no facility for online payment only cash may be consumed ,we cannot purchase plants through online mode. Limited customers reached to the nursery because sometime customer need to travel for long distance as nursery is far from their home.
- The system has the provision of orders entered by the clients along with their contact details, grading specifications, special services, job codes, and amount of request. After an order is entered, an order confirmation report will be sent to the client for review. When all orders have been entered, a surplus for sale report will be created.
- Collecting the information from various research papers and other sources. We analysis that many peoples want to buy plants and they have to directly concern with the nursery. Sometimes people do not know specific information about particular plant items as well as seller is not technically skilled.
- Customer does not compare plants prices with different shopkeeper and there is no facility for online payment only cash may consume.
- So, in this case plantree is platform where customer can compare plants pricing and make online payment easily. Customer service is extremely important. We want each customer to have a pleasant shopping experience, and it is the intention of our staff to answer questions with expertise and to offer advice when we feel it is needed.
- Retain customers to generate repeat purchases and make referrals. Continue to expand daily sales by adding to the variety of plants we sell. Communication with our customers through creative advertising. Customers can buy plants from their home. Customers can view a large number of plants available in a nursery. When an admin decides to check out the order, then information including the buyer's name, address and billing instruction is record in system for future references.



2.2 Minimum Hardware / Software Configuration

Hardware :

- Processor : Intel i3 or above
- Processor Speed : 2.3 Ghz
- Ram : 4 GB or above
- Hard-Disk : Minimum 20GB or above

Software :

- Operating System : Android OS
- Front-End : Android Studio
- Back-End : Firebase
- Documentation tools : Windows Word 2019

The minimum requirements to download and run Android Studio are:

Operating System: Microsoft Windows 7/8/10 (64-bit) or macOS 10.13 High Sierra or higher.

RAM: 8 GB RAM recommended (minimum 4 GB).

Disk Space: 4 GB of available disk space minimum (ideally SSD).

CPU: Intel Core i3 or higher, including AMD Ryzen processors.

Graphics: NVIDIA® GeForce® 710 or higher, or equivalent AMD Graphics card.

The system requirements for Android Studio for various operating systems are listed in the following table:

Windows	OS X	Linux
Microsoft Windows 8/7/Vista/2003 (32 or 64 bit)	Mac OS X 10.8.5 or higher, up to 10.9 (Mavericks)	GNOME or KDE or Unity desktop



PLANTREE E-COMMERCE APPLICATION

Windows	OS X	Linux
2 GB RAM minimum, 4 GB RAM recommended	2 GB RAM minimum, 4 GB RAM recommended	2 GB RAM minimum, 4 GB RAM recommended
400 MB hard disk space plus at least 1 GB for Android SDK, emulator system images, and caches	400 MB hard disk space plus at least 1 GB for Android SDK, emulator system images, and caches	400 MB hard disk space plus at least 1 GB for Android SDK, emulator system images, and caches
Java Development Kit (JDK) 7	Java Runtime Environment (JRE) 6 Java Development Kit (JDK) 7	Oracle Java Development Kit (JDK) 7
Optional for accelerated emulator: Intel processor with support for Intel VT-x, Intel EM64T (Intel 64), and Execute Disable (XD) Bit functionality	Optional for accelerated emulator: Intel processor with support for Intel VT-x, Intel EM64T (Intel 64), and Execute Disable (XD) Bit functionality	GNU C Library (glibc) 2.11 or later

Android Studio system requirements

The following are the system requirements for Android Studio on Windows.

- 64-bit Microsoft® Windows® 8/10/11
- x86_64 CPU architecture; 2nd generation Intel Core or newer, or AMD CPU with support for a Windows Hypervisor
- 4 GB RAM or more
- 8 GB of available disk space minimum (IDE + Android SDK + Android Emulator)
- 1280 x 800 minimum screen resolution



2.3 Drawbacks / Limitations

There are several drawbacks of offline plant shopping, which are as follows:

- **Limited options:** Offline plant shopping is often limited to the availability of plants in nearby nurseries or stores. This can lead to a limited variety of plants, which may not cater to the specific needs or preferences of the buyer.
- **Time-consuming:** Offline plant shopping requires physical travel to nurseries or stores, which can be time-consuming. Additionally, buyers may need to wait in line or search through various stores to find the desired plants.
- **Inconvenient:** Offline plant shopping may not be convenient for people who live in areas with limited access to nurseries or stores. This can be especially challenging for people who do not have their own transportation.
- **Higher costs:** The costs of plants may be higher in offline stores compared to online stores due to the overhead costs of running a physical store.
- **Lack of information:** Offline plant shopping may not provide buyers with the same level of information as online shopping. Buyers may not be able to access detailed information about the plant's care requirements, growth patterns, or other relevant information.
- **Limited availability:** Offline stores may not have a consistent supply of plants due to seasonal changes or other factors. This can make it difficult for buyers to find the plants they need at a specific time.

Overall, offline plant shopping has several drawbacks that can make it challenging for buyers to find the plants they need at a reasonable cost and in a convenient manner.



3. PROPOSE PROJECT PROFILE

3.1 Introduction

- Electronic Commerce application consist of purchasing and selling products which is intended to benefit both gardeners and users. This platform aims to help the local gardeners in selling their plants through online and to provide a more convenient and more reliable way for users to find, view and buy the desired plants that are suitable for their necessity, the customers can also get description and details from the product online.
- The app also featured an auto suggest engine which the user might use in searching the plants if ever they don't really have an idea on how and what to grow. Problems found that most gardeners had a hard time in selling their plants, in reaching their customer and the insufficient specification on plants and trees that the possible buyer might experience.
- The researchers conduct a usability test on five end-users and five IT Professionals a total of ten respondents to gather information on their experience. During the test, the researchers also provided a scenario wherein the respondents act as a buyer who needed to use the auto suggest feature and conduct the whole process of checking out the product. The result has shown that majority of the respondents have agreed that the Auto Suggest really do help them decide which plant to choose and their overall experience was pleasant and intuitive enough that this platform makes it a user friendly.
- Plantree is a e-commerce application that enables users to shop for a wide range of products online. As e-commerce has become increasingly popular, Plantree has emerged as a leading platform that offers a convenient and user-friendly shopping experience. The application is designed to cater to the needs of a diverse user base, providing access to a wide range of products, from clothing and electronics to home decor and groceries.
- Plantree offers a seamless shopping experience, allowing users to browse and search for products, compare prices, and place orders with ease. The application is designed to be user-friendly, with intuitive interfaces that make it easy to navigate and find what you're looking for. Additionally, Plantree offers a range of features and services to enhance the shopping experience, including secure payment options, fast and reliable shipping, and responsive customer support.



3.2 Objective / Goal / Aim

- Our goal is to build an online community of buyers and sellers of plants and trees, services and information and to help people provide an outlet from their busy lives & indulge in nature.
- This is a unique place for nature enthusiasts to browse & order the best quality of plants online and get them delivered. This would give convenience to a shopper, along with instant satisfaction, for the purchase of a variety of products.
- Information on how to grow the plants would also be given in a way that to consumers will be educated and encouraged in raising it well so consumers can learn how to nurture and take care of them from a professional.
- On the other hand, we could also help our local gardeners to advertise their products in a wider range of audience which had been their problem since there are numerous competitors in the market. But among all, we could be able to contribute to raise awareness regarding on the conservation of our environment and the greening of our community that would serve as small steps in saving our environment from further destruction.
- The primary objective of Plantree is to provide a user-friendly and convenient platform for buying and selling a wide range of products, including plants. The application aims to offer a seamless shopping experience for users, with a focus on high-quality products, secure transactions, and reliable customer support.
- Overall, the aim of Plantree is to create a reliable and sustainable e-commerce platform that offers a convenient and enjoyable shopping experience for users, while also promoting sustainable practices in the plant trade.



3.3 Scope

- Plantree e-commerce application has a vast scope, providing a wide range of benefits to both buyers and sellers. The application caters to a diverse range of users, offering a seamless shopping experience for a wide range of products, including clothing, electronics, home decor, groceries, and plants.
- Moreover, the Plantree e-commerce application offers a range of features and services that enhance the shopping experience for both buyers and sellers. For example, it provides secure payment options, fast and reliable shipping, and responsive customer support, ensuring a smooth and hassle-free shopping experience.
- One of the major benefits of the Plantree e-commerce application is that it provides a convenient and user-friendly platform for buying and selling plants. The application offers a wide selection of plants from different regions and climates, allowing buyers to browse and purchase plants from the comfort of their homes. Sellers, on the other hand, can reach a wider audience and sell their plants without the need for a physical store or marketplace.
- Moreover, the Plantree e-commerce application offers a range of features and services that enhance the shopping experience for both buyers and sellers. For example, it provides secure payment options, fast and reliable shipping, and responsive customer support, ensuring a smooth and hassle-free shopping experience.
- Another significant benefit of the Plantree e-commerce application is that it promotes sustainable and eco-friendly practices in the plant trade. The application encourages the purchase of locally-grown plants, reducing the need for unnecessary transportation and packaging, and supporting local and small-scale plant growers.



3.4 Type of Project

- An online application is the most basic and essential e-commerce project. It involves creating a mobile application where customers can browse and purchase products online. The project may include features such as product listings, shopping cart, checkout, payment gateway integration, and order management.
- This is an android based e-commerce application in which a buyer can sell their products and seller can buy the product.
- This main aim of the application is to buy and sell the products online by which customer can easily get the product easily on the application, the application consists of user and vendor side.
- e-commerce applications can be classified into different types based on their purpose, target audience, features, and business model. Here are some common types of e-commerce applications:

Business-to-Consumer (B2C) e-commerce: These are e-commerce applications that allow businesses to sell their products or services directly to consumers.

Consumer-to-Consumer (C2C) e-commerce: These are e-commerce applications that enable individuals to buy and sell products or services to each other.

Business-to-Business (B2B) e-commerce: These are e-commerce applications that facilitate transactions between businesses, such as suppliers and wholesalers.

Mobile commerce (m-commerce): These are e-commerce applications designed specifically for mobile devices like smartphones and tablets.

Social commerce: These are e-commerce applications that leverage social media platforms to facilitate buying and selling activities.

Subscription-based e-commerce: These are e-commerce applications that offer recurring product or service subscriptions to customers.

- The type of e-commerce application that Plantree is developing will depend on its specific business objectives and target audience.



3.5 Technology / Environment

Operating System : Android OS

Android OS is an operating system used in smartphones, tablets, and other mobile devices. It was developed by Google and is based on the Linux kernel. Android is an open-source operating system, which means that anyone can access and modify its source code. Android OS provides a platform for running mobile applications and provides many features and tools that make it easy to develop, distribute, and manage mobile applications.

IDE(Integrated development environmen) : Android Studio

Android Studio provides a complete development environment that includes tools for coding, testing, and debugging Android applications. It is based on the IntelliJ IDEA community edition, which is an open-source IDE for Java development. Android Studio is specifically designed to support Android app development and provides many features and tools that make it easy to develop high-quality Android apps.

Front- end : Java, XML(Extensible Markup Language)

Java is the primary programming language used for developing Android applications. It is a popular, object-oriented language that is used for developing both back-end and front-end components of an Android app. Java is used for writing the code that runs on the device, such as the application logic, user interface, and data storage.

XML is a markup language that is used for designing user interfaces in Android applications. It is used for defining the layout of screens and widgets in the applic

In Android Studio, Java and XML are often used together to develop Android applications. For example, Java code can be used to define the functionality of a button, while the XML file can be used to define the layout and appearance of the button on the screen.

Back-End : Firebase

Firebase is a Backend-as-a-Service (BaaS) platform developed by Google. It provides a suite of tools and services for building mobile and web applications, including hosting, database, authentication, and analytics.

Firebase offers a real-time database that allows developers to store and sync data in real-time across multiple clients. It uses the NoSQL data model, which makes it easy to store and manage structured and unstructured data.

Overall, Firebase is a powerful platform that provides a wide range of tools and services for building modern mobile and web applications. It allows developers to focus on building great user experiences while taking care of the backend infrastructure and services.



3.6 Applicability of the system

The applicability of an e-commerce application Plantree can be determined by its target audience, products/services, and business objectives. Here are some potential applicability scenarios for an e-commerce application:

Online retail: An e-commerce application can be used for online retail, allowing businesses to sell products directly to customers. This could include various products such as clothing, electronics, and groceries.

Online services: An e-commerce application can also be used to sell online services, such as subscriptions, online courses, and consulting services.

B2B e-commerce: An e-commerce application can be used to facilitate transactions between businesses, such as suppliers and wholesalers.

Social commerce: An e-commerce application can leverage social media platforms to facilitate buying and selling activities.

Marketplace: An e-commerce application can be designed as a marketplace that connects buyers and sellers, allowing them to transact with each other.

Mobile commerce: An e-commerce application can be designed specifically for mobile devices, allowing users to shop on-the-go.

The applicability of Plantree's e-commerce application will depend on its specific business model, target audience, and products/services offered.



4. SOFTWARE ANALYSIS

4.1 Preliminary Investigation

A preliminary investigation of an Plantree e-commerce application would involve an assessment of its business model, market, competition, and technology.

Business model:

The investigation would begin by examining Plantree's business model, including how it generates revenue, its pricing strategy, and its target audience. It would also look at the company's organizational structure, funding, and overall strategy.

Market analysis:

The investigation would analyze the market in which Plantree operates, including the size, growth potential, and trends in the e-commerce industry. It would also look at the demographics and behavior of Plantree's target audience, as well as the overall competitive landscape.

Competitor analysis:

The investigation would examine Plantree's competitors, including their strengths, weaknesses, and market positioning. It would also identify any potential threats or opportunities in the market.

Technology assessment:

The investigation would assess Plantree's technology stack, including its website, mobile app, backend infrastructure, and security measures. It would also examine any third-party tools or services that Plantree uses, such as payment gateways and hosting providers.

Legal and regulatory compliance:

The investigation would examine whether Plantree complies with relevant laws and regulations, such as consumer protection laws, data privacy laws, and tax laws.

The results of the investigation would inform the overall feasibility of Plantree's e-commerce application, including its potential for success, risks and challenges, and areas for improvement. It would also provide insights into the company's strengths and weaknesses, as well as potential opportunities for growth and expansion.



4.2 Problem Identification

Limited market reach:

A traditional brick-and-mortar store is limited to its local market. An e-commerce application can provide access to a global market, potentially increasing sales and revenue.

High operational costs:

A traditional store requires rent, utilities, and staff, among other costs. An e-commerce application can reduce operational costs and increase profitability.

Limited hours of operation:

A traditional store may only be open during certain hours. An e-commerce application can be open 24/7, providing customers with access to products at any time.

Inconvenient shopping experience:

A traditional store requires customers to physically travel to the location to shop. An e-commerce application can provide a convenient shopping experience from anywhere with an internet connection.



4.3 Feasibility study / Risk Analysis

A feasibility study for an e-commerce application Plantree typically includes an analysis of the technical, economic, operational, management and time feasibility of the project. Here are some key factors that might be considered in a feasibility study for Plantree e-commerce application:

4.3.1 Technical Feasibility

This involves assessing the technical requirements and capabilities of the e-commerce application. It may involve evaluating the compatibility of the application with different devices, analyzing the requirements for the database and the server, and assessing the availability of technical resources such as developers and infrastructure.

4.3.2 Economical Feasibility

This involves analyzing the financial viability of the e-commerce application. It may involve estimating the cost of development and implementation, as well as the ongoing costs of maintenance and upgrades. The study might also consider the revenue potential of the application and the expected return on investment.

4.3.3 Operational Feasibility

This involves assessing the operational requirements of the e-commerce application. It may involve evaluating the staffing needs for the project, the training and support requirements for users, and the availability of necessary resources such as raw materials or suppliers.

4.3.4 Management Feasibility

Effective project management is essential for the successful development and implementation of an e-commerce application. This includes defining the project scope, creating a detailed project plan, allocating resources appropriately, and managing risks and issues that may arise during development.

4.3.5 Time Feasibility

Time feasibility describes the time cost for converting the present system to new system. The aspect of time feasibility for our system takes approximate. This system is developing under this time period. So we can say that this system is time feasibility.

Overall, these are crucial factors in determining the feasibility of an e-commerce application like Plantree. By managing these factors effectively, the development team can ensure that the application is developed on time and within budget, and that it meets customer requirements and delivers a positive user experience.



4.4 Requirement Analysis

Requirement analysis is a process that is undertaken in software development to understand and document the needs and expectations of stakeholders for a particular software system. The purpose of requirement analysis is to gather information about what the software should do, how it should perform, and what features and functions it should have.

Functional Requirements:

The Plantree e-commerce application will need to have features such as product catalogue management, search and filtering, shopping cart and checkout process, order management, customer account management, shipping and delivery management, and reporting.

Non-functional Requirements:

The Plantree e-commerce application should have high performance, scalability, reliability, security, usability, and accessibility. The application should be able to handle a large number of users and transactions without slowing down or crashing. It should also be secure from cyber threats and ensure the privacy of customer data.

User Requirements:

The Plantree e-commerce application should be user-friendly, easy to navigate, and have a responsive design that works well on different devices. It should also offer personalized recommendations, search capabilities, and easy checkout options.

Business Requirements:

The Plantree e-commerce application should have goals such as increasing revenue, reducing costs, improving customer satisfaction, and expanding the customer base. It should also be able to integrate with other systems such as ERP and CRM to streamline business operations.

Integration Requirements:

The Plantree e-commerce application should be able to integrate with other systems such as payment gateways, shipping carriers, and inventory management systems.

Regulatory and Compliance Requirements:

The Plantree e-commerce application should comply with data privacy laws, payment regulations, and accessibility standards.

Overall, the requirement analysis of the Plantree e-commerce application will involve identifying and documenting the specific features and functions required, as well as the quality attributes, user needs, business goals, integration requirements, and regulatory compliance.



4.4.1 Fact Finding Techniques

There are several fact-finding techniques that can be used to gather information about the Plantree e-commerce application, including:

Surveys:

Surveys can be used to gather quantitative data on user preferences, habits, and feedback related to the e-commerce application.

Observation:

Observing users as they interact with the e-commerce application can provide insights into how they use it, what features they prefer, and any usability issues.

Document Analysis:

Analysing documents such as business plans, requirements documents, and user manuals can provide information about the overall goals of the e-commerce application, its scope, and specific features and functions.

Prototyping:

Creating a prototype of the e-commerce application can provide valuable feedback from users, and help to refine the requirements and design of the application.

Focus Groups:

Focus groups can be used to gather feedback and opinions from a group of users about the e-commerce application, and to identify any common issues or concerns.

Benchmarking:

Benchmarking involves comparing the e-commerce application with similar applications in the market to identify best practices and areas for improvement.

By using these fact-finding techniques, the development team can gather a comprehensive understanding of the requirements and needs of the stakeholders for the Plantree e-commerce application. This information can then be used to develop a robust and user-friendly application that meets the needs of all stakeholders.



4.4.2 Time Line Chart

A time chart line, also known as a timeline, is a graphical representation of events or activities over a specific period of time. It is a way to visually represent a series of events or milestones in chronological order.

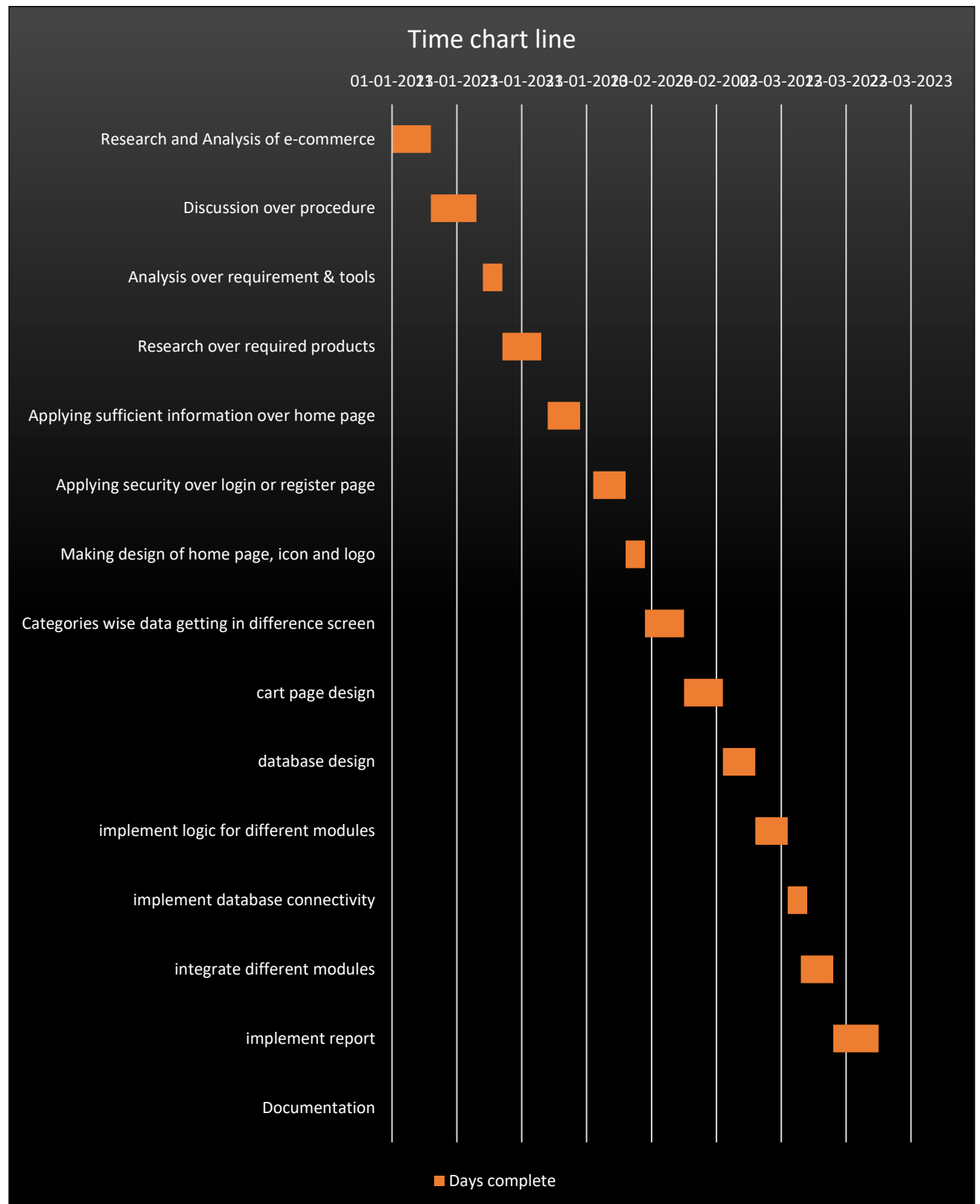
A time chart line typically consists of a horizontal line with dates or time periods marked along it, and points or markers representing events or activities at specific points in time. The length or position of the markers on the timeline can be used to indicate the duration of an event or activity.

A timeline chart for the Plantree e-commerce application could be used to illustrate the various stages and activities involved in its development and implementation. Here is an example of a possible timeline chart:

Task	Start Date	Days complete
Research and Analysis of e-commerce	01-01-2023	6
Discussion over procedure	07-01-2023	7
Analysis over requirement & tools	15-01-2023	3
Research over required products	18-01-2023	6
Applying sufficient information over home page	25-01-2023	5
Applying security over login or register page	01-02-2023	5
Making design of home page, icon and logo	06-02-2023	3
Categories wise data getting in difference screen	09-02-2023	6
Cart page design	15-02-2023	6
Database design	21-02-2023	5
Implement logic for different modules	26-02-2023	5
Implement database connectivity	03-03-2023	3
Integrate different modules	05-03-2023	5
Implement report	10-03-2023	7
Documentation		



PLANTREE E-COMMERCE APPLICATION





4.4.3 Model with Justification/ Agile Modeling

Agile Model

The meaning of Agile is swift or versatile. “**Agile process model**” refers to a software development approach based on iterative development. Agile methods break tasks into smaller iterations, or parts do not directly involve long term planning. The project scope and requirements are laid down at the beginning of the development process. Plans regarding the number of iterations, the duration and the scope of each iteration are clearly defined in advance.

Each iteration is considered as a short time "frame" in the Agile process model, which typically lasts from one to four weeks. The division of the entire project into smaller parts helps to minimize the project risk and to reduce the overall project delivery time requirements. Each iteration involves a team working through a full software development life cycle including planning, requirements analysis, design, coding, and testing before a working product is demonstrated to the client.



Fig. Agile Model



PLANTREE E-COMMERCE APPLICATION

Phases of Agile Model:

Following are the phases in the Agile model are as follows:

1. Requirements gathering
2. Design the requirements
3. Construction/ iteration
4. Testing/ Quality assurance
5. Deployment
6. Feedback

1. Requirements gathering: In this phase, you must define the requirements. You should explain business opportunities and plan the time and effort needed to build the project. Based on this information, you can evaluate technical and economic feasibility.

2. Design the requirements: When you have identified the project, work with stakeholders to define requirements. You can use the user flow diagram or the high-level UML diagram to show the work of new features and show how it will apply to your existing system.

3. Construction/ iteration: When the team defines the requirements, the work begins. Designers and developers start working on their project, which aims to deploy a working product. The product will undergo various stages of improvement, so it includes simple, minimal functionality.

4. Testing: In this phase, the Quality Assurance team examines the product's performance and looks for the bug.

5. Deployment: In this phase, the team issues a product for the user's work environment.

6. Feedback: After releasing the product, the last step is feedback. In this, the team receives feedback about the product and works through the feedback.

When to use the Agile Model?

- When frequent changes are required.
- When a highly qualified and experienced team is available.
- When a customer is ready to have a meeting with a software team all the time.
- When project size is small.



PLANTREE E-COMMERCE APPLICATION

Advantage(Pros) of Agile Method:

1. Frequent Delivery
2. Face-to-Face Communication with clients.
3. Efficient design and fulfils the business requirement.
4. Anytime changes are acceptable.
5. It reduces total development time.

Disadvantages(Cons) of Agile Model:

1. Due to the shortage of formal documents, it creates confusion and crucial decisions taken throughout various phases can be misinterpreted at any time by different team members.
2. Due to the lack of proper documentation, once the project completes and the developers allotted to another project, maintenance of the finished project can become a difficulty.



4.4.4 Flow Chart

A flowchart is a graphical representation of a process or workflow that shows the sequence of steps or actions needed to complete a task or solve a problem. Flowcharts use standardized symbols to depict each step in a process, as well as arrows to indicate the direction of flow between steps. They are often used in programming, engineering, business, and other fields as a tool for process analysis, documentation, and communication. Flowcharts can help to identify bottlenecks or areas where a process can be improved, as well as to communicate complex procedures in a clear and easily understandable way.

A flowchart is a visual representation of a process or workflow. It is a diagrammatic representation of a sequence of steps or actions required to complete a task or solve a problem. Flowcharts use standardized symbols and shapes to represent each step in the process, and arrows to indicate the direction of flow between steps.

The basic components of a flowchart include:

- Start and end points: These are represented by ovals or circles and show where the process begins and ends.
- Process steps: These are represented by rectangles or boxes and represent the actions that are taken in the process.
- Decision points: These are represented by diamonds or rhombuses and are used to show decision points in the process where a choice must be made.
- Arrows: These show the direction of flow between the different steps or components of the process.

There are several types of flowcharts that are used in different contexts, including:

- Basic flowcharts: These are the simplest type of flowchart and are used to illustrate a basic process or workflow.
- Swimlane flowcharts: These are used to show the roles and responsibilities of different individuals or departments in a process.
- Data flow diagrams: These show how data flows through a system or process.
- Workflow diagrams: These are used to illustrate a specific workflow or process within a larger system.



Flowline

1. **Flowline Symbol:** Shows the process' direction. Each flowline connects two blocks.



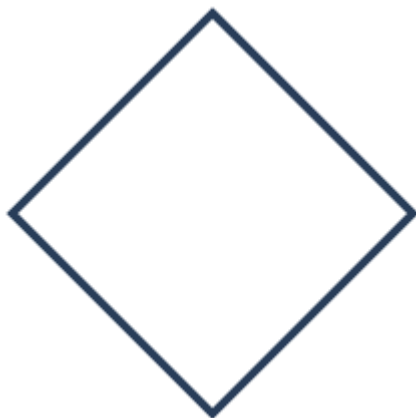
Terminal Symbol

2. **Terminal Symbol:** Indicates the beginning or end of a flowchart.



Process

3. **Process:** Represent a step in a process. This is the most common component of a flowchart.



Decision Symbol



PLANTREE E-COMMERCE APPLICATION

4. Decision Symbol: Shows a step that decides the next step in a process. This is commonly a yes/no or true/false question.



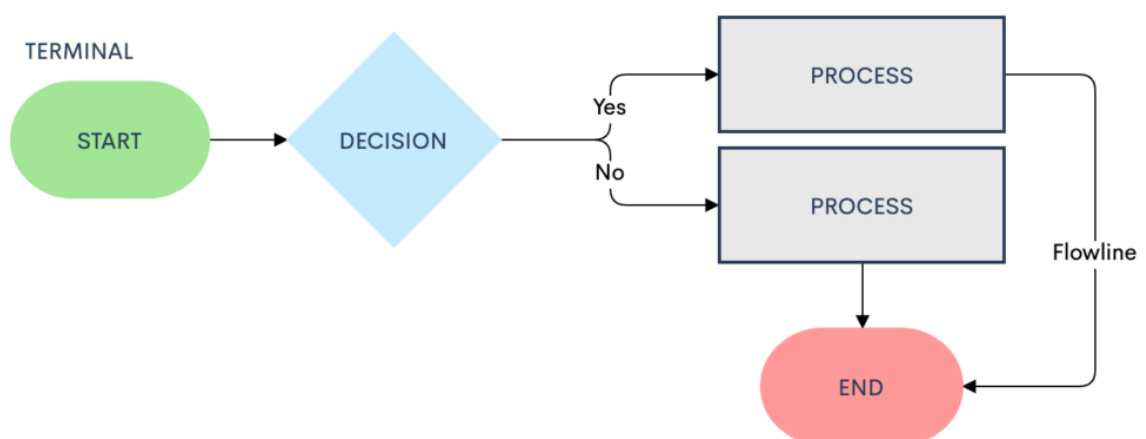
Input / Output

5. Input / Output Symbol: Indicates the process of inputting or outputting external data. This is indicated by the shape of a parallelogram.



Annotation / Comment

4 Most Common Flowchart Symbols





PLANTREE E-COMMERCE APPLICATION

Top 4 Flowchart Symbols in Use

Although there are many flowchart symbols that can be used, there are 4 main symbols that are most important. In fact, you really need only these 4 in most cases:

1. Process Symbol:



Process

Process Symbol represents a step in a process. This is the most common component of a flowchart.

2. Terminal Symbol:



Terminal

Terminal Symbol indicates the beginning or end of a flowchart. This symbol usually has the text “Start” or “End”.

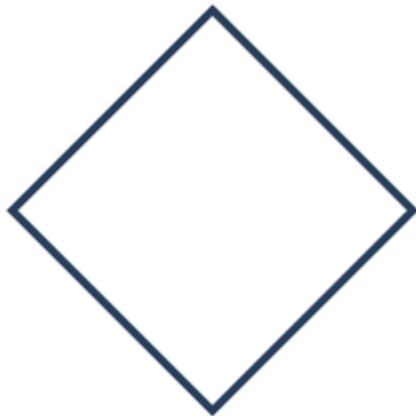
3. Flowline Symbol:



Flowline

Flow lines indicate the process' direction. Each flowline usually connects two blocks. Flowlines can be straight lines, uni-directional arrow, or both-sided arrows, representing different kinds of connections between the steps.

4. Decision Symbol:



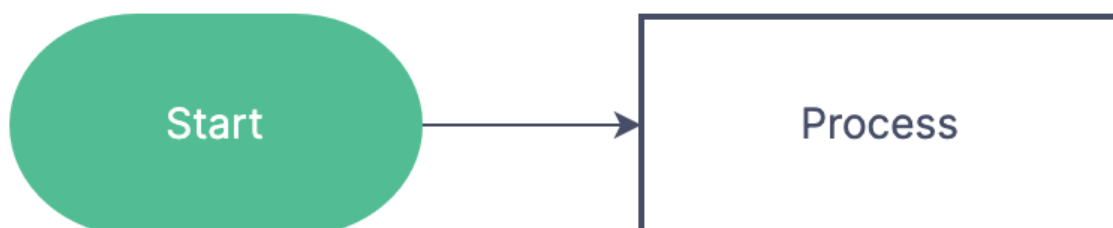
Decision

Decision Symbol indicates a step that decides the next step in a process. This is commonly a Yes/No or True/False question.

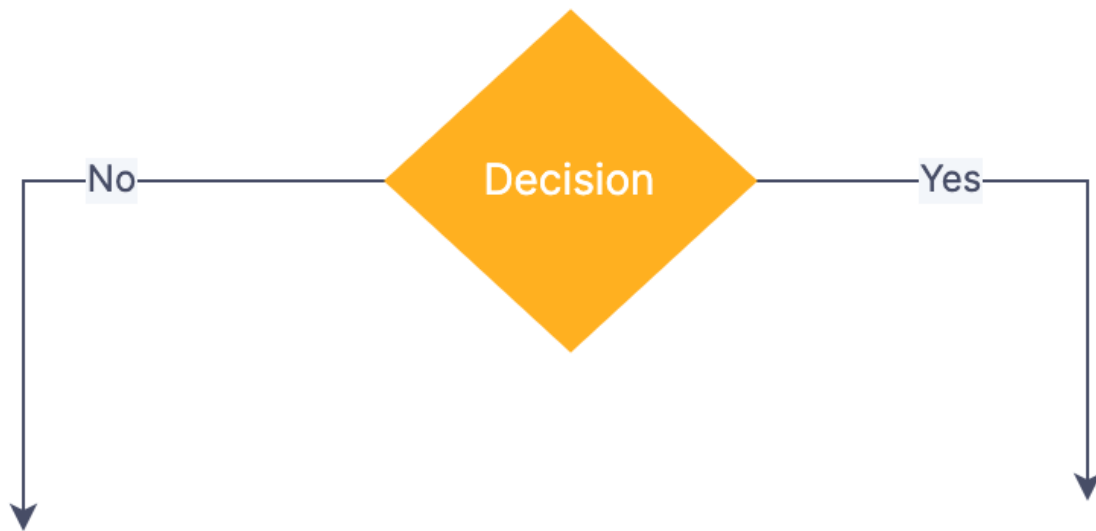
Flowchart Symbols in Practice

Flowcharts are an easy way to visualize a process or procedure. They can be used in many different areas such as teaching, business, or even in science experiments. To put **flowchart symbols** to use, follow these steps and guidelines to get started.

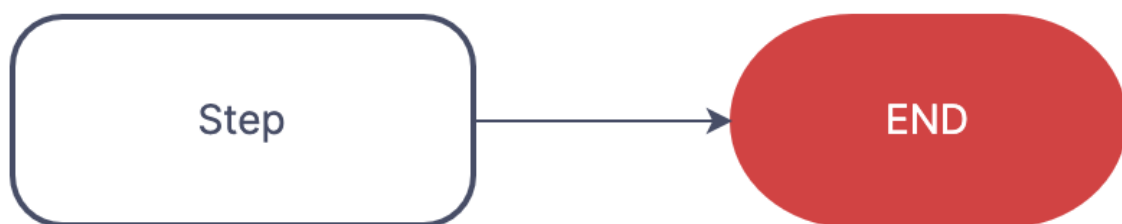
1. Start with a **Terminal** symbol. Often this Terminal symbol has the word "START" in this. This way, viewers of your flow chart can quickly identify where the flowchart begins.



2. Insert a **Process** symbol connected by flowlines.
3. Insert a **Decision** symbol whenever there's a choice to be made.



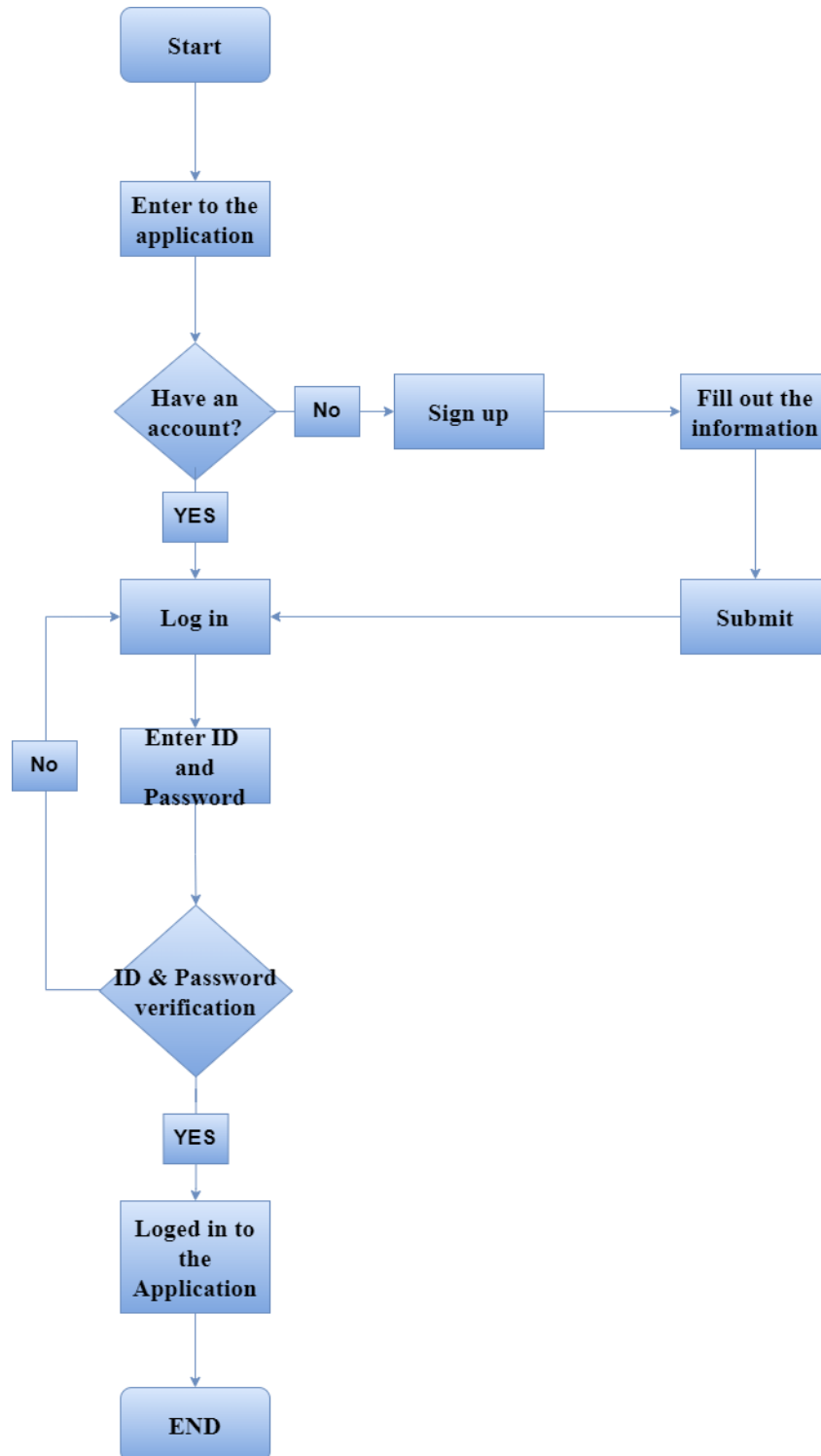
4. Build out the rest of the flowchart using the same principles.
5. End the chart with another **Terminal** symbol with the word "END" in it.





PLANTREE E-COMMERCE APPLICATION

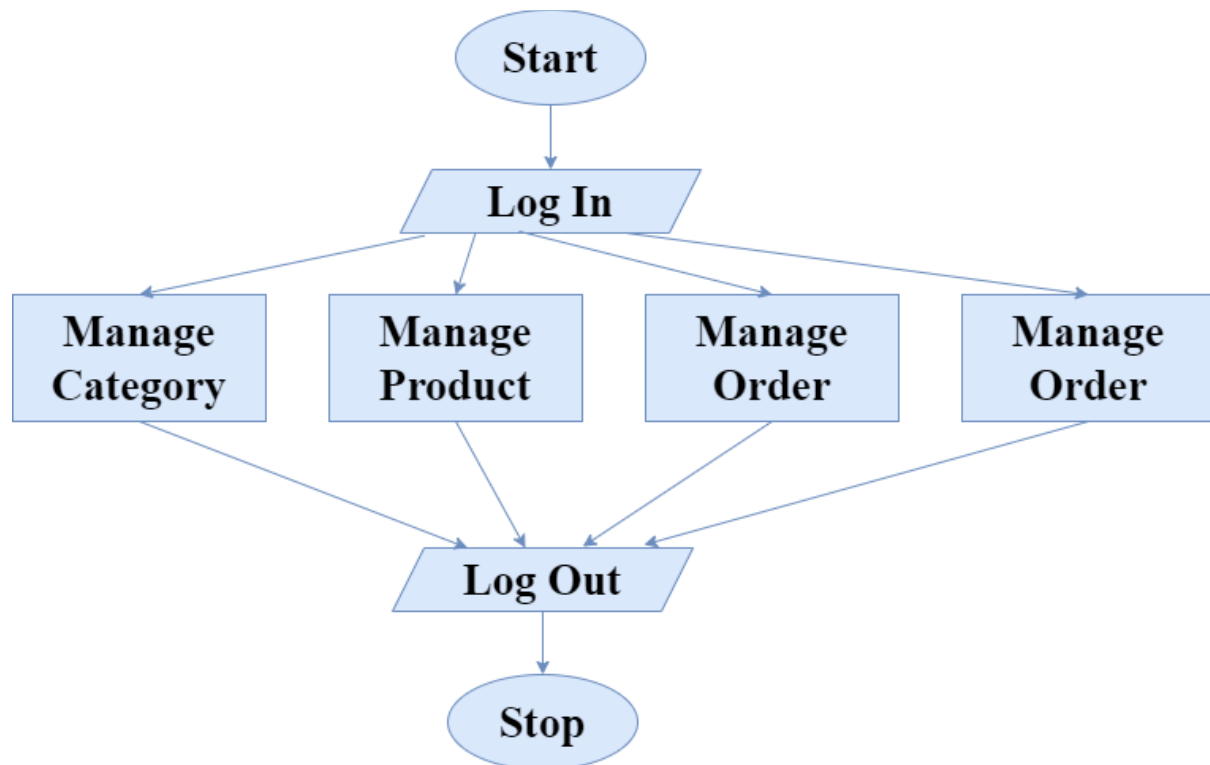
Login-in:-





PLANTREE E-COMMERCE APPLICATION

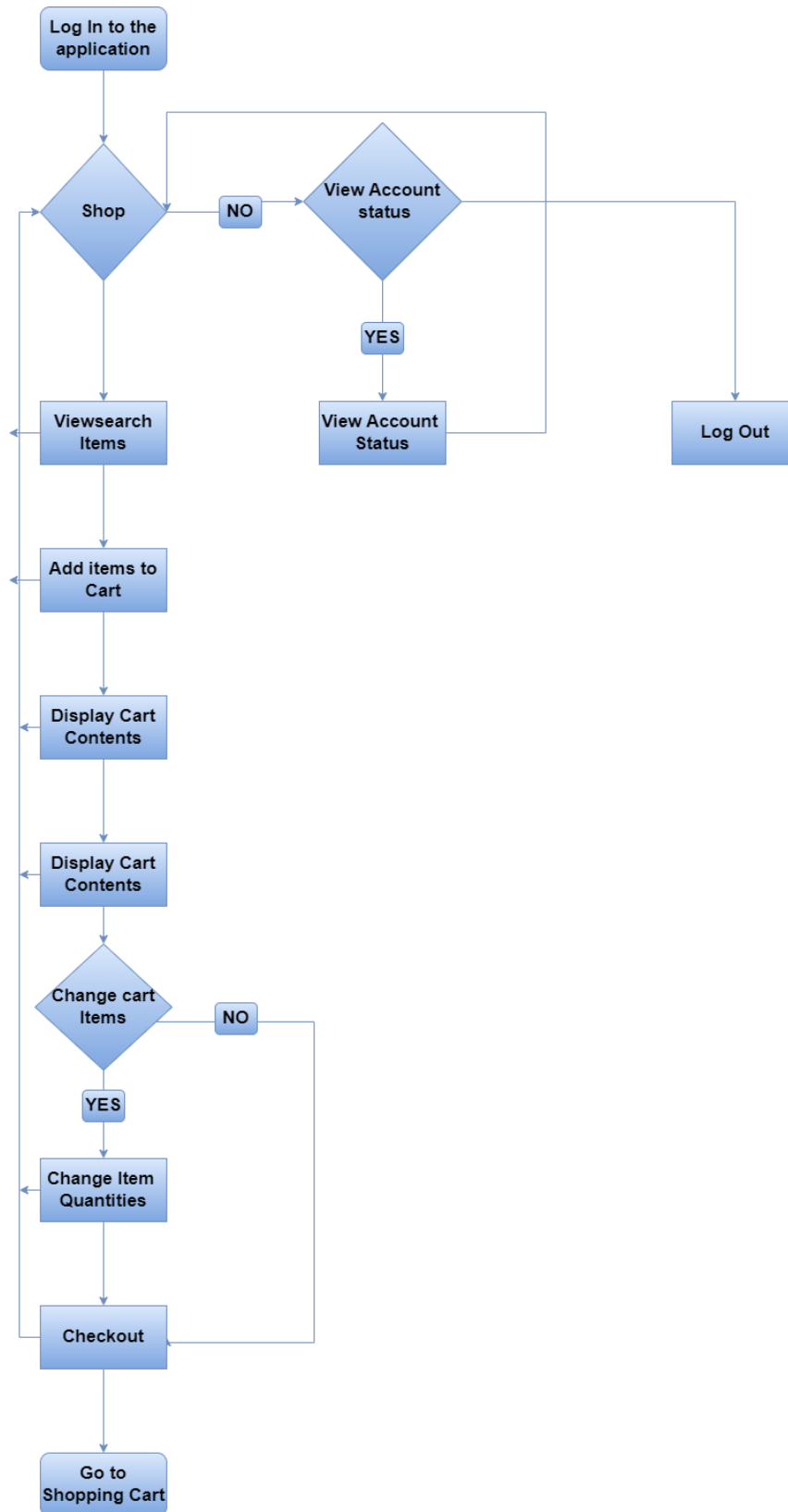
Admin :-





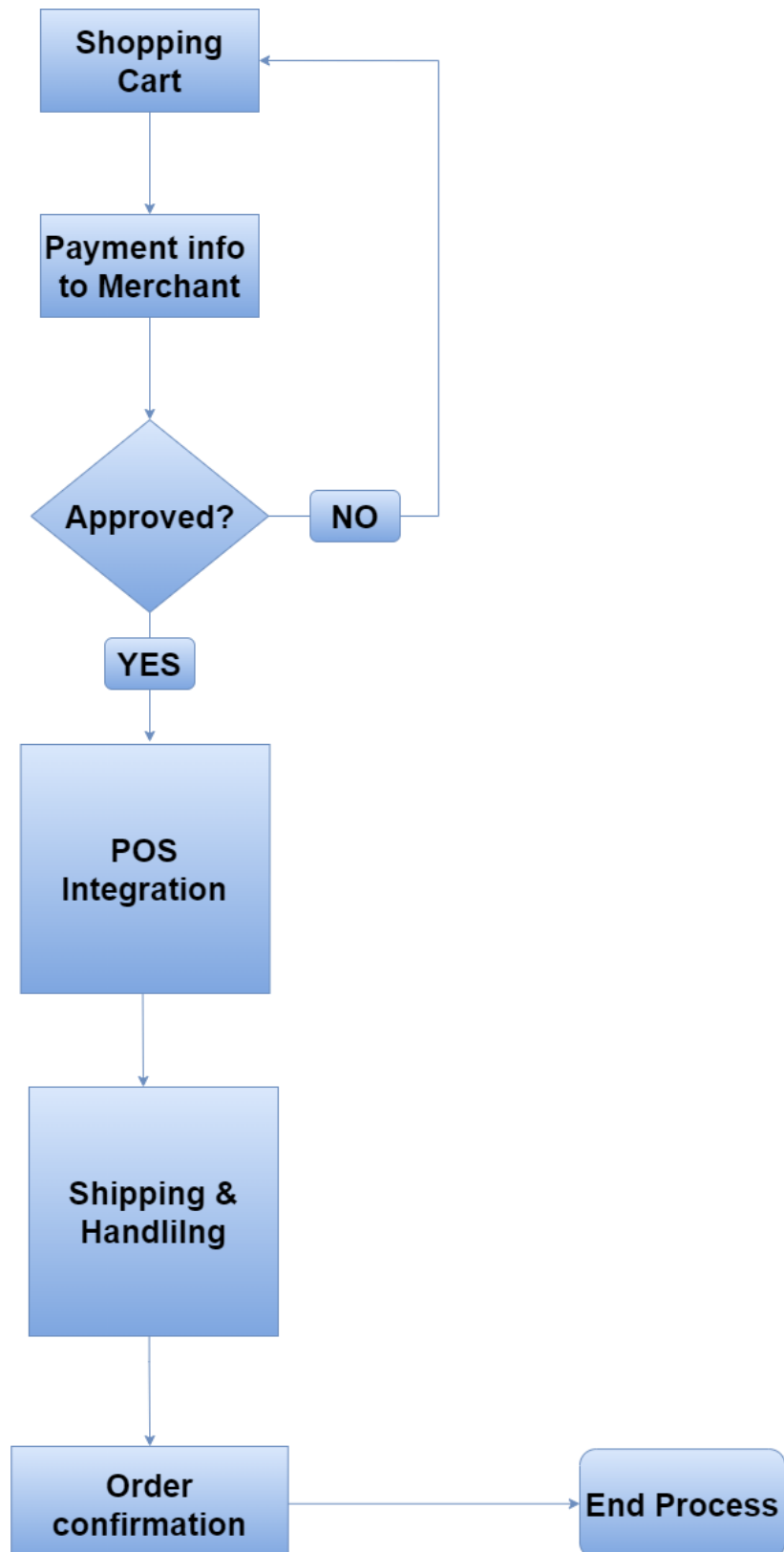
PLANTREE E-COMMERCE APPLICATION

Shopping:-





Shopping cart:-





4.4.5 DFD and/or UML

Data Flow Diagrams

A Data Flow Diagram (DFD) is a traditional visual representation of the information flows within a system. A neat and clear DFD can depict the right amount of the system requirement graphically. It can be manual, automated, or a combination of both.

It shows how data enters and leaves the system, what changes the information, and where data is stored.

The objective of a DFD is to show the scope and boundaries of a system as a whole. It may be used as a communication tool between a system analyst and any person who plays a part in the order that acts as a starting point for redesigning a system. The DFD is also called as a data flow graph or bubble chart.

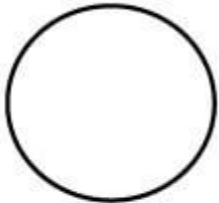
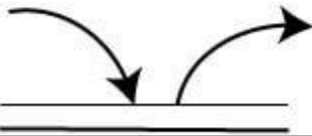
The following observations about DFDs are essential:

names should be unique. This makes it easier to refer to elements in the DFD.

1. Remember that DFD is not a flow chart. Arrows in a flow chart that represents the order of events; arrows in DFD represents flowing data. A DFD does not involve any order of events.
2. Suppress logical decisions. If we ever have the urge to draw a diamond-shaped box in a DFD, suppress that urge! A diamond-shaped box is used in flow charts to represents decision points with multiple exists paths of which the only one is taken. This implies an ordering of events, which makes no sense in a DFD.
3. Do not become bogged down with details. Defer error conditions and error handling until the end of the analysis.

Standard symbols for DFDs are derived from the electric circuit diagram analysis and are shown in fig:



Symbol	Name	Function
	Data flow	Used to Connect Processes to each other, to sources or Sinks; the arrow head indicates direction of data flow.
	Process	Performs Some transformation of Input data to yield output data.
	Source of Sink (External Entity)	A Source of System inputs or Sink of System outputs.
	Data Store	A repository of data; the arrow heads indicate net inputs and net outputs to store.

Symbols for Data Flow Diagrams

Circle: A circle (bubble) shows a process that transforms data inputs into data outputs.

Data Flow: A curved line shows the flow of data into or out of a process or data store.

Data Store: A set of parallel lines shows a place for the collection of data items. A data store indicates that the data is stored which can be used at a later stage or by the other processes in a different order. The data store can have an element or group of elements.

Source or Sink: Source or Sink is an external entity and acts as a source of system inputs or sink of system outputs.

Levels in Data Flow Diagrams (DFD)

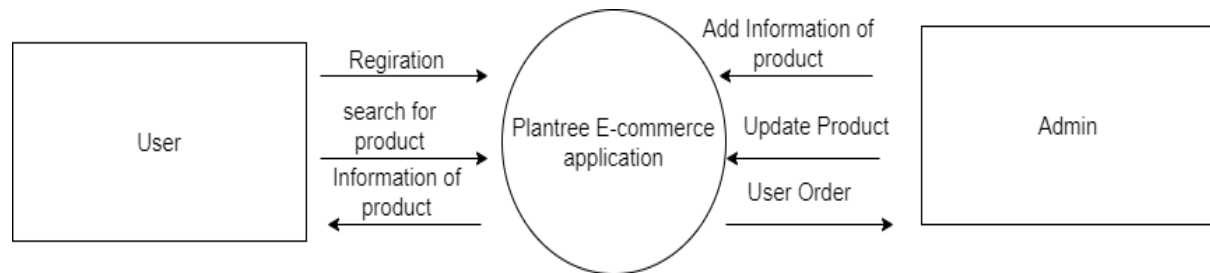
The DFD may be used to perform a system or software at any level of abstraction. Infact, DFDs may be partitioned into levels that represent increasing information flow and functional detail. Levels in DFD are numbered 0, 1, 2 or beyond. Here, we will see primarily three levels in the data flow diagram, which are: 0-level DFD, 1-level DFD, and 2-level DFD.



PLANTREE E-COMMERCE APPLICATION

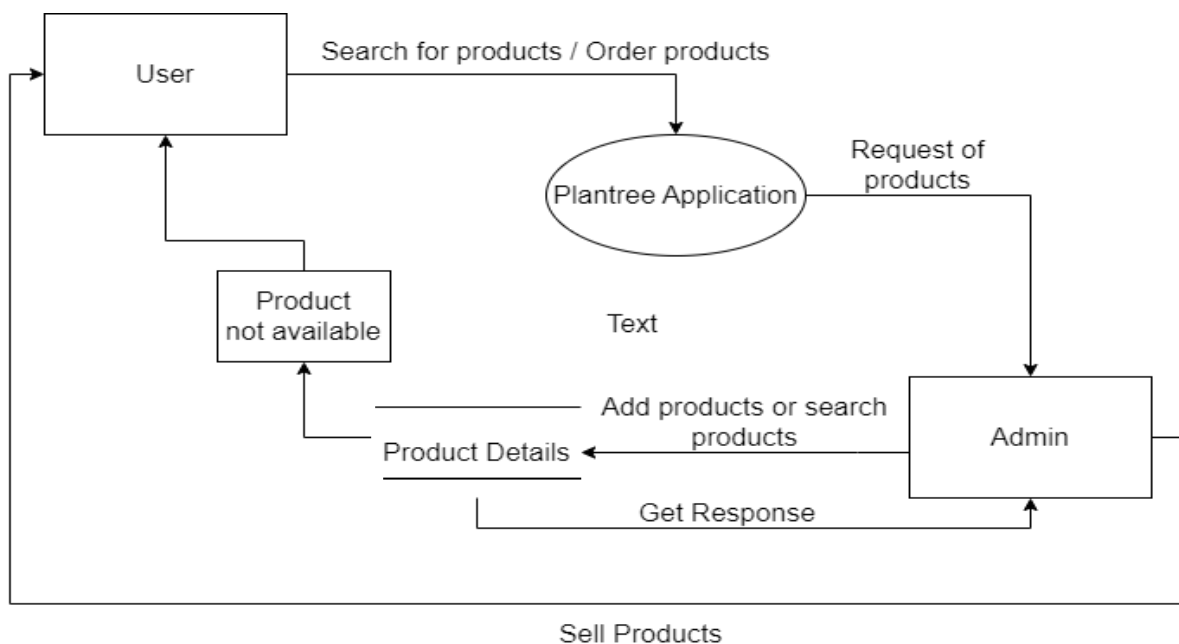
0-level DFD

The Level-0 DFD, also called context diagram of the result management system is shown in fig. As the bubbles are decomposed into less and less abstract bubbles, the corresponding data flow may also be needed to be decomposed.



1-level DFD

In 1-level DFD, a context diagram is decomposed into multiple bubbles/processes. In this level, we highlight the main objectives of the system and breakdown the high-level process of 0-level DFD into subprocesses.

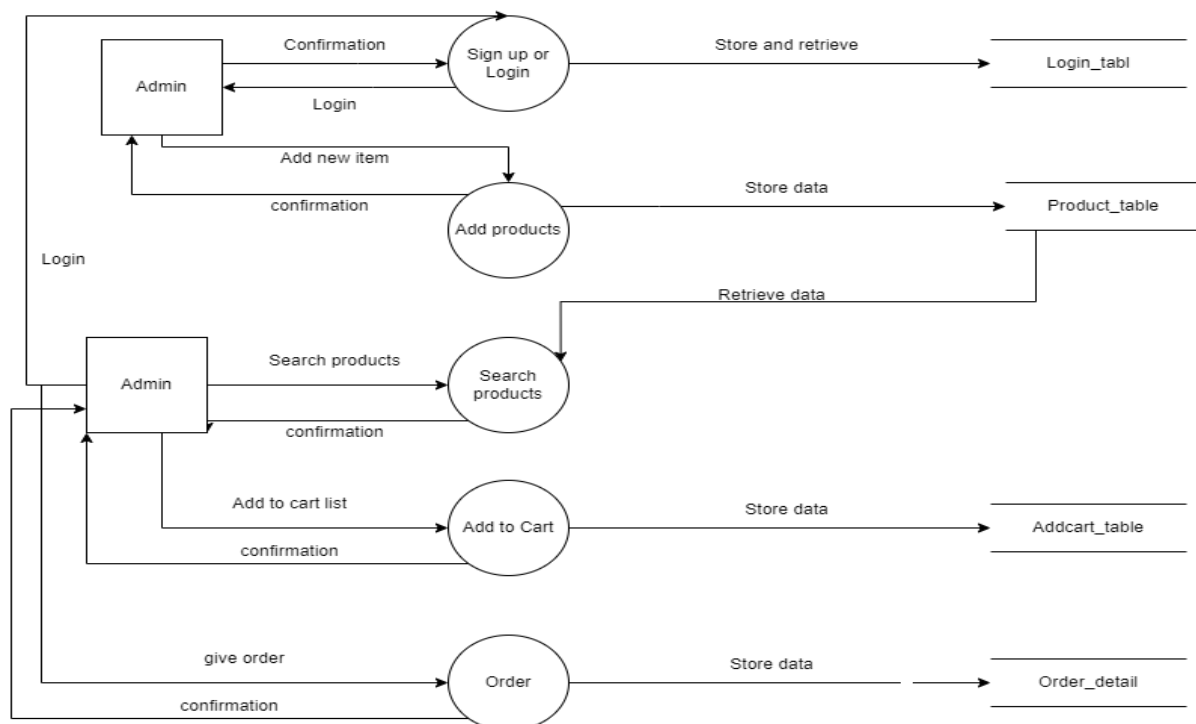




PLANTREE E-COMMERCE APPLICATION

2-Level DFD

2-level DFD goes one process deeper into parts of 1-level DFD. It can be used to project or record the specific/necessary detail about the system's functioning.



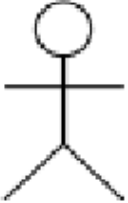
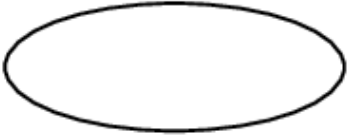
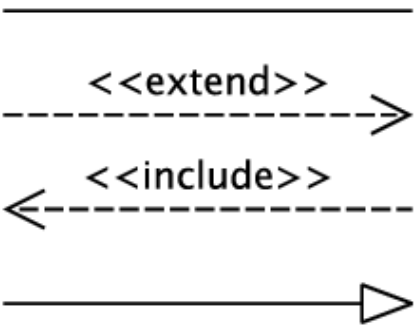
Unified Modeling Language

UML stands for Unified Modeling Language, which is a standardized visual modeling language used to design and document software systems. UML is used by software engineers, analysts, and designers to create blueprints or diagrams that represent different aspects of a software system, including its structure, behavior, and interactions between components.

UML diagrams can be used to model software systems at different levels of abstraction, from high-level overviews to detailed specifications of individual components. The most common types of UML diagrams include class diagrams, use case diagrams, sequence diagrams, activity diagrams, and state machine diagrams.

UML is a powerful tool for communicating complex software designs and requirements to stakeholders and development teams, and it is widely used in software development processes such as Agile, Scrum, and Waterfall.

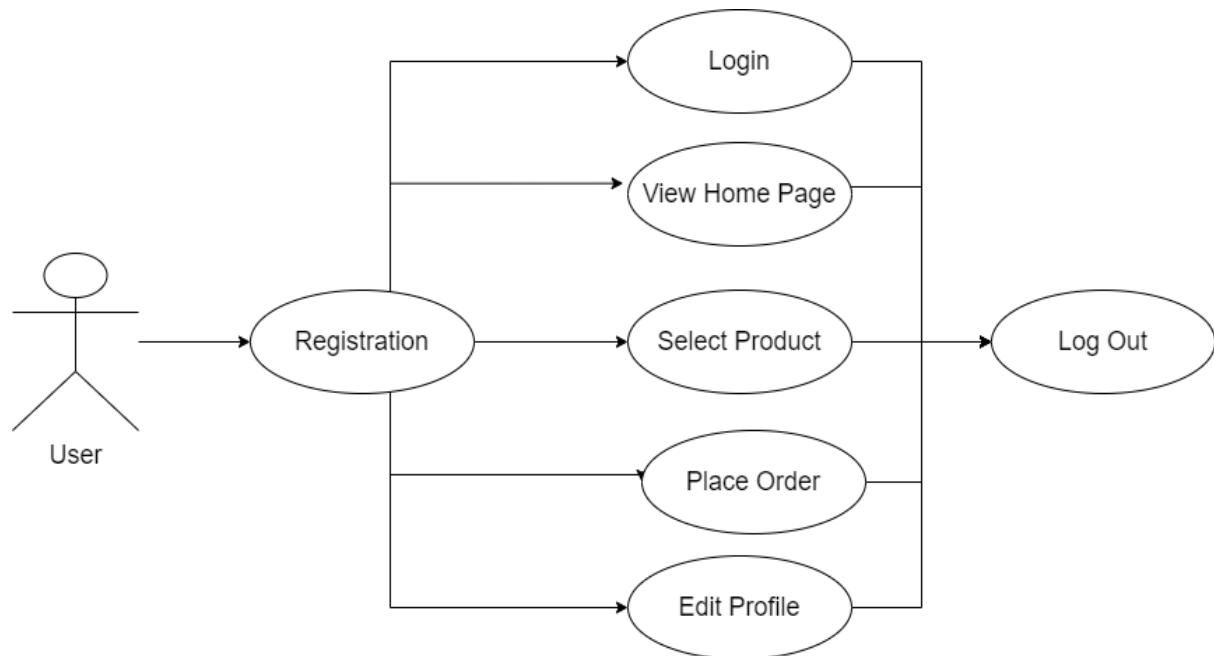


Symbol	Reference Name
	Actor
	Use case
	Relationship

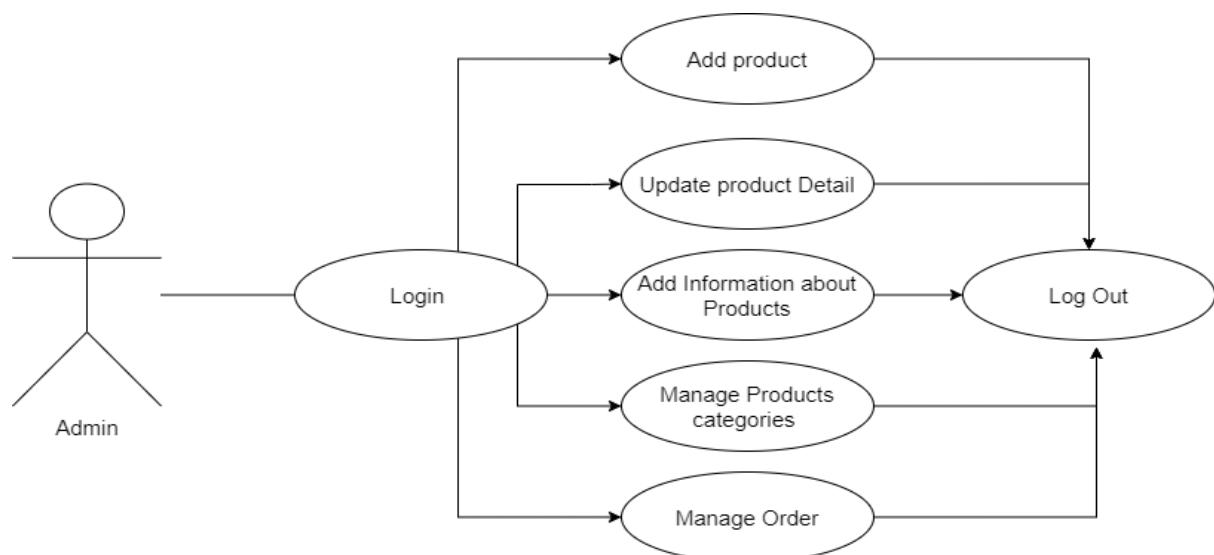


PLANTREE E-COMMERCE APPLICATION

UML FOR USER:-



UML FOR ADMIN :-





4.4.6 Process / Control Specification

Process/control specification refers to the process of defining the requirements for controlling a process or system. This involves identifying the key parameters that need to be monitored and controlled, specifying the appropriate control algorithms and hardware, and defining the overall control strategy.

Process/control specification is a critical step in the design of automated systems, as it ensures that the system will function reliably and efficiently in the intended application. The process/control specification process typically involves several steps, including:

1. Defining the objectives of the control system or process
2. Identifying the key parameters to be monitored and controlled
3. Developing a mathematical model of the process or system
4. Selecting appropriate control algorithms and hardware
5. Designing and implementing the control system
6. Testing and validating the control system

During the process/control specification process, engineers must carefully consider the specific requirements and constraints of the process or system being controlled. This may involve selecting sensors and actuators that are appropriate for the application, designing control algorithms that are robust and efficient, and implementing safety measures to prevent system failures.

Effective process/control specification can result in improved efficiency, safety, and reliability of the controlled process or system, and is an important part of modern engineering design.

Process/control specification for a Plantree e-commerce application :-

Process/control specification for a Plantree e-commerce application would involve defining the requirements for controlling the e-commerce system to ensure it functions effectively and efficiently in the intended application. The following steps could be taken:

1. Defining the objectives of the e-commerce system: This would involve defining the overall purpose of the e-commerce system, such as providing a platform for customers to purchase products and for vendors to sell products.
2. Identifying the key parameters to be monitored and controlled: This would involve identifying the critical aspects of the e-commerce system that must be monitored and controlled, such as inventory levels, product pricing, order processing, and payment processing.



PLANTREE E-COMMERCE APPLICATION

3. Developing a mathematical model of the e-commerce system: A mathematical model of the system could be developed to help engineers understand how the different components of the system interact and to develop control algorithms that can optimize the performance of the system.
4. Selecting appropriate control algorithms and hardware: Control algorithms could be developed to ensure that the system operates within defined parameters, such as maintaining inventory levels within a specific range, ensuring that prices remain competitive, and processing orders in a timely manner. Appropriate hardware, such as servers and databases, must also be selected to support the system.
5. Designing and implementing the e-commerce system: Based on the process/control specification, the e-commerce system could be designed and implemented. This would involve integrating different hardware and software components, creating user interfaces, and configuring the system to operate efficiently.
6. Testing and validating the e-commerce system: Finally, the system would be tested to ensure that it meets the desired specifications and operates correctly. This would involve testing the system under different conditions, such as high traffic volumes and varying user demand, and validating the system's performance against the defined requirements.

Overall, effective process/control specification for a Plantree e-commerce application can result in improved efficiency, reliability, and customer satisfaction.



PLANTREE E-COMMERCE APPLICATION

4.4.7 Data Dictionary

A data dictionary is a structured repository of information about data elements that are used within an information system or database. It provides a clear definition of the data elements and their relationships, as well as information about how the data is structured, stored, and used within the system.

Data Dictionary: -

COLLECTION NAME :- Plantree

Description:- Use for storing all data of application

Sr.no	Name	Data type	Data Size	Constraint	Description
1.	Admin view	Object	Object	-	Contain admin data
2.	User view	Object	Object	-	Contain user data
3.	Orders	Object	Object	-	Contain order detail
4.	products	Object	Object	-	Contain products detail
5.	Users	object	Object	-	Contain user login data

COLLECTION NAME :- Plantree

Node :- Admin view

Description:- Use for storing of Admin data

Sr.no	name	Data type	Data size	Constraint	Description
1.	Phone number	Object	Object	-	Phone number of Admin
2.	Products	Object	Object	-	Product detail which admin add

COLLECTION NAME :- Plantree

Sub Node of admin view :- Products

Composite key :- admin view + phone number + products + products reg. date

Description:- Use for storing of user order.



PLANTREE E-COMMERCE APPLICATION

Sr.no	Name	Data Type	Data Size	Constraint	Description
1.	Date	Object	Object	-	Product entry date
2.	Discount	Object	Object	-	Discount given to the customer
3.	Pid	Object	Object	-	Product id
4.	Pname	Object	Object	-	Product name
5.	Price	Object	Object	-	Price of product
6.	Quantity	Object	Object	-	Quantity of product
7.	Time	Object	Object	-	Time of order

COLLECTION NAME :- Plantree

Node :- User view

Description:- Use for storing of User data.

Sr.no	Name	Data type	Data size	Constraint	Description
1.	Phone number	Object	Object	-	Phone number of user
2.	Products	Object	Object	-	Product detail which user purchase

COLLECTION NAME :- Plantree

Sub Node of User view :- Products

Composite key :- User view + phone number + products + products reg. date

Description:- Use for storing of user data.

Sr.no	Name	Data Type	Data Size	Constraint	Description
1.	Date	Object	Object	-	Product entry date
2.	Discount	Object	Object	-	Discount given to the customer
3.	Pid	Object	Object	-	Product id
4.	Pname	Object	Object	-	Product name
5.	Price	Object	Object	-	Price of product
6.	Quantity	Object	Object	-	Quantity of product
7.	Time	Object	Object	-	Time of order

COLLECTION NAME :- Plantree

Node :- Order

composite key :- Orders + phone number

Description:- Use for storing of user order data.



PLANTREE E-COMMERCE APPLICATION

Sr.no	Name	Data Type	Data Size	Constraint	Description
1.	Address	Object	Object	-	Address of order delivery
2.	City	Object	Object	-	City of order delivery
3.	Date	Object	Object	-	Date of order
4.	Name	Object	Object	-	User name
5.	Phone	Object	Object	-	User phone number
6.	status	Object	Object	-	Status of order
7.	Time	Object	Object	-	Time of order
8.	totalamount	Object	Object	-	Total amount of purchase

COLLECTION NAME :- Plantree

Node :- Products

Description:- Use for storing of products data.

Sr.no	Name	Data Type	Data Size	Constraint	Description
1.	Category	Object	Object	-	Category of products
2.	date	Object	Object	-	Date of products
3.	description	Object	Object	-	Description of products
4.	image	Object	Object	-	Image of products
5.	Pid	Object	Object	-	Products Id
6.	pname	Object	Object	-	Products name
7.	price	Object	Object	-	Products Price
8.	Time	Object	Object	-	Products time

COLLECTION NAME :- Plantree

Node :- User

Description:- Use for storing of user login data.

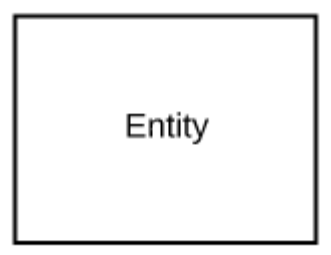
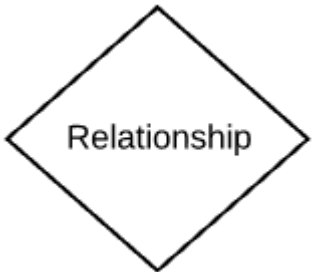
Sr.no	name	Data type	Data size	constraint	description
1	name	Object	Object	-	User name use for login
2	phone	object	object	-	User phone number use for login
3	Password	Object	object	-	Password for login



4.4.8 E-R Diagram

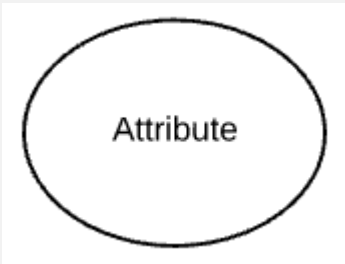
An ER (Entity-Relationship) diagram is a visual representation of the relationships between entities in a database. It is a graphical model used to represent the information or data requirements and the relationships between entities in a system or database.

1. **Entities:** An entity is a thing, object, or concept in the real world that is represented in the database. For example, in a university database, entities could include students, courses, and professors.
2. **Attributes:** An attribute is a characteristic or property of an entity. For example, for a student entity, attributes could include name, student ID, major, and GPA.
3. **Relationships:** A relationship is a connection or association between two or more entities. For example, a student can be enrolled in multiple courses, and a course can have multiple students enrolled.

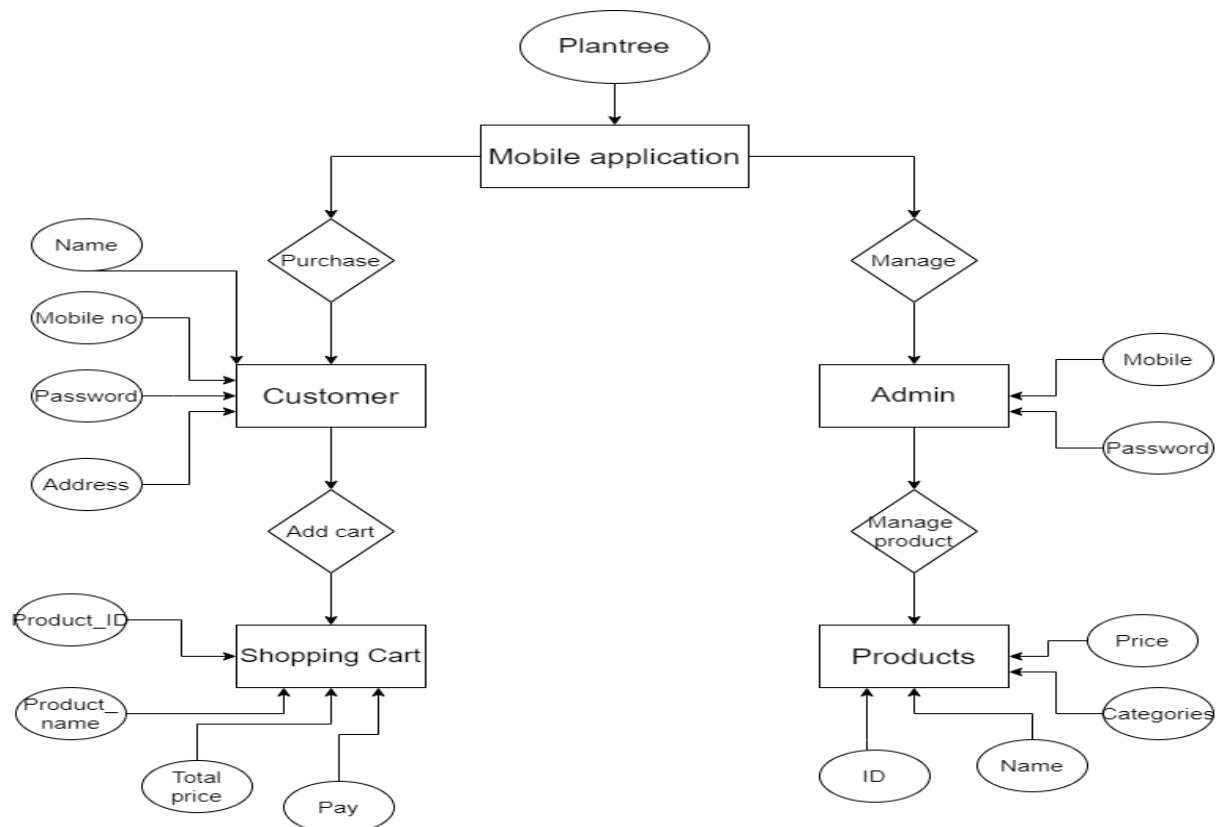
Entity Symbol	Name	Description
	Strong entity	These shapes are independent from other entities, and are often called parent entities, since they will often have weak entities that depend on them. They will also have a primary key, distinguishing each occurrence of the entity.
	Relationship	Relationships are associations between or among entities.



PLANTREE E-COMMERCE APPLICATION

	Attribute	Attributes are characteristics of an entity, a many-to-many relationship, or a one-to-one relationship.
---	-----------	---

ER Diagram of plantree e-commerce application: -





4.4.9 Data Object Description

Plantree is an e-commerce application that allows customers to purchase plants and gardening supplies online. The application is designed to be user-friendly and intuitive, with a focus on providing a seamless shopping experience for customers. Here are some examples of data objects that might be used in the Plantree application:

Product: The Product object represents a specific plant or gardening supply that is available for purchase on the Plantree website. This object contains information such as the product name, description, price, image, and inventory level.

Customer: The Customer object represents an individual who has created an account on the Plantree website. This object contains information such as the customer's name, email address, shipping address, and payment information.

Order: The Order object represents a customer's purchase of one or more products from the Plantree website. This object contains information such as the order number, customer information, product information, total price, and shipping information.

Cart: The Cart object represents a customer's current shopping cart on the Plantree website. This object contains information such as the products in the cart, the quantity of each product, and the total price.

Review: The Review object represents a customer's review of a specific product on the Plantree website. This object contains information such as the customer's name, rating, and comments.

Category: The Category object represents a specific category of plants or gardening supplies on the Plantree website. This object contains information such as the category name and a list of products that belong to the category.

Promotion: The Promotion object represents a discount or special offer that is available to customers on the Plantree website. This object contains information such as the promotion code, discount amount, and expiration date.

These data objects are just a few examples of the many types of data that might be used in the Plantree e-commerce application. By using objects to represent different types of data, the application can be designed to be more modular, flexible, and scalable. This makes it easier to add new features, modify existing features, and handle large amounts of data.



5. SOFTWARE DESIGN

5.1 Project Design Process Hierarchy

The Project Design Process Hierarchy is a structured approach to organizing the various stages and components of a project design process. This hierarchy typically consists of several levels, each of which represents a different level of detail and abstraction in the design process.

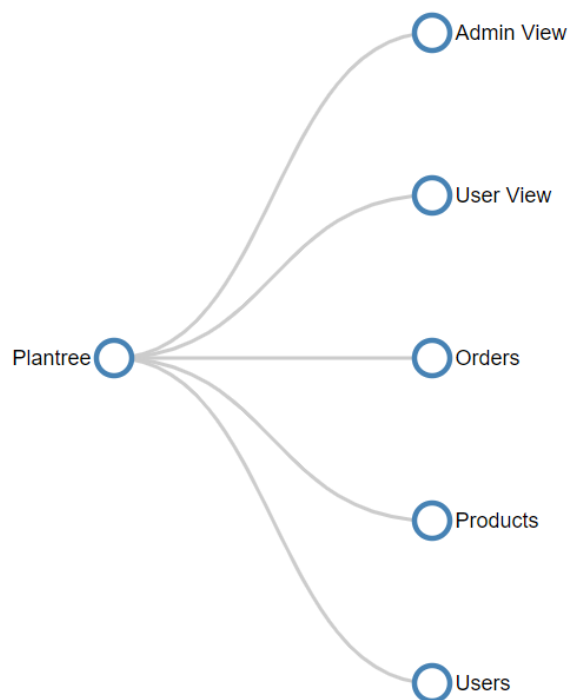
- **Conceptualization:** In this stage, the goals and objectives of the e-commerce application are defined. This may include identifying the target market, the types of products to be sold, and the business model for the application.
- **Planning:** The planning stage involves developing a detailed plan for how the e-commerce application will be developed, tested, launched, and maintained. This may include creating a project timeline, defining project milestones, and identifying the resources required to complete the project.
- **Design:** In this stage, the actual design of the e-commerce application is developed, including its user interface, database schema, and backend architecture. The design should take into account the needs and preferences of the target market, as well as any technical constraints or requirements.
- **Implementation:** The implementation stage involves building the e-commerce application according to the design specifications. This may include developing the frontend and backend code, setting up the database and server infrastructure, and integrating payment processing and other third-party services.
- **Testing:** Once the e-commerce application has been developed, it must be thoroughly tested to ensure that it meets the functional and non-functional requirements defined in the design stage. This may include testing the application for performance, scalability, security, and usability.
- **Deployment:** After testing is complete, the e-commerce application can be deployed to a production environment. This may involve setting up a web server, configuring DNS settings, and uploading the application code and database to the server.
- **Maintenance:** Once the e-commerce application is live, it must be regularly maintained to ensure that it remains up-to-date and secure. This may include applying security patches, fixing bugs, and adding new features or functionality as needed.

By following a structured Project Design Process Hierarchy, the Plantree e-commerce application project team can ensure that the application is developed efficiently and effectively, and that it meets the needs of its intended users.



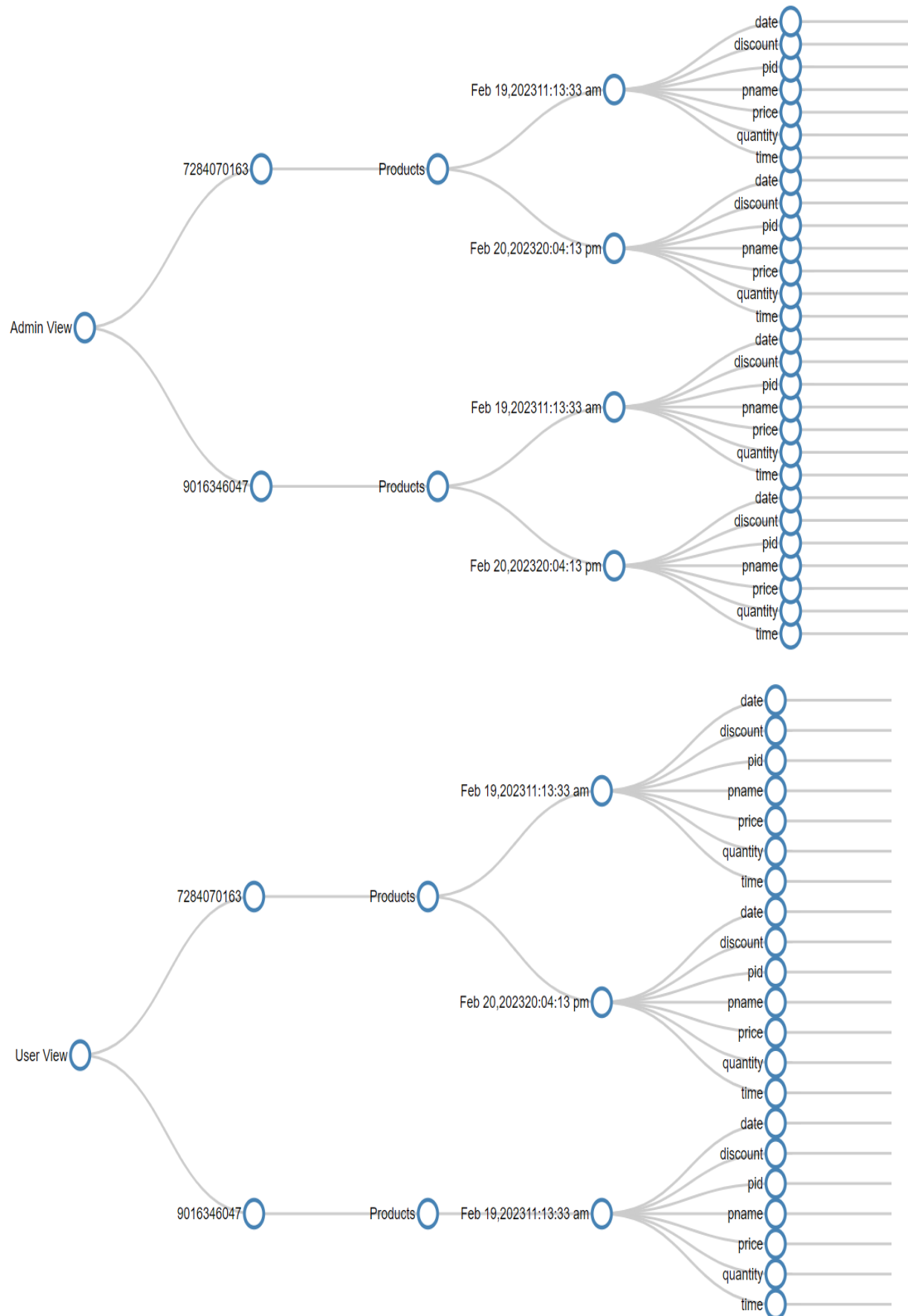
5.2 Database Design

Database design is the process of defining the structure, relationships, and rules that govern the organization and storage of data in a database management system (DBMS). It involves creating a logical model of the database schema, which outlines the tables, fields, and relationships between them, as well as defining the constraints and rules that govern the data stored in the database.



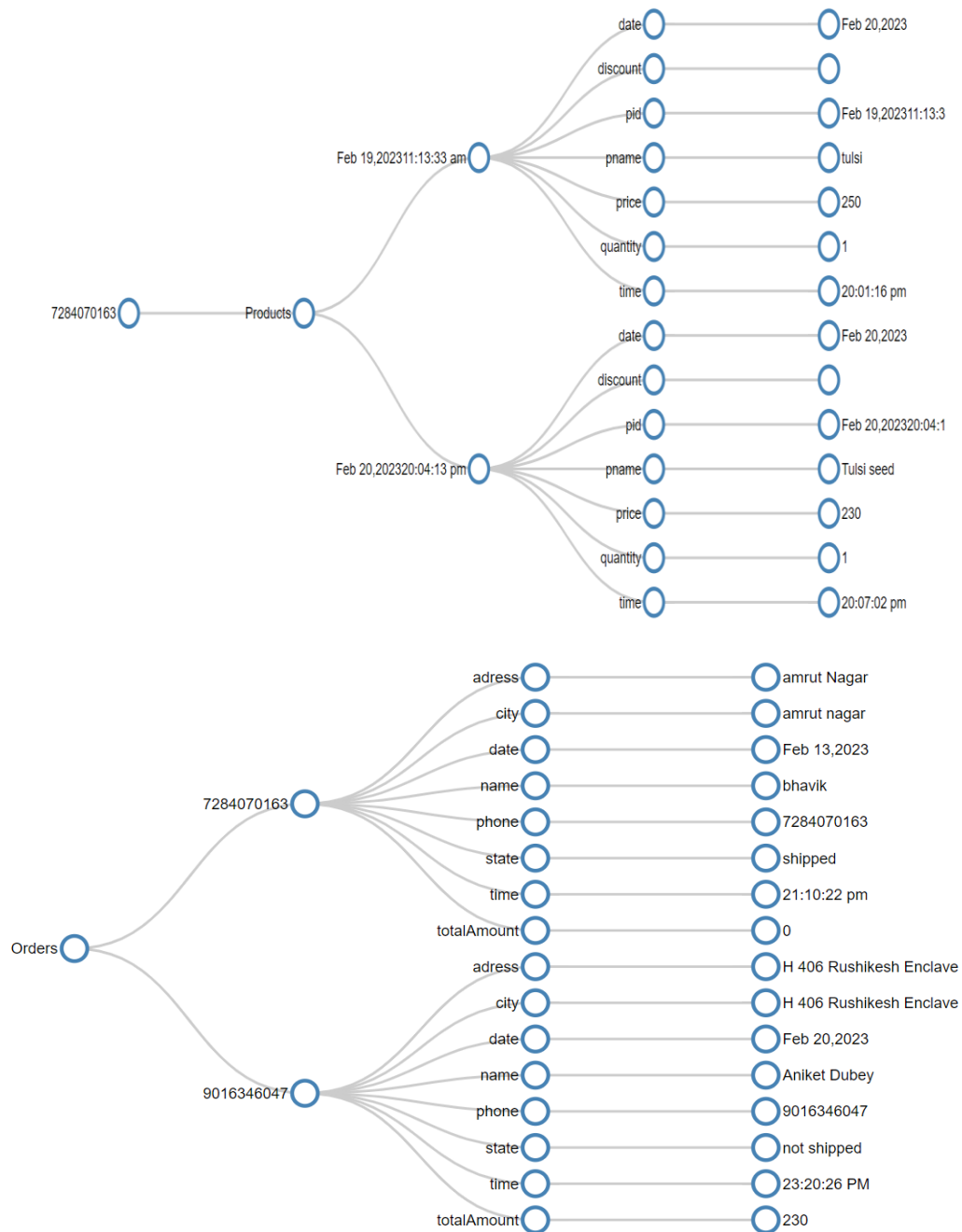


PLANTREE E-COMMERCE APPLICATION



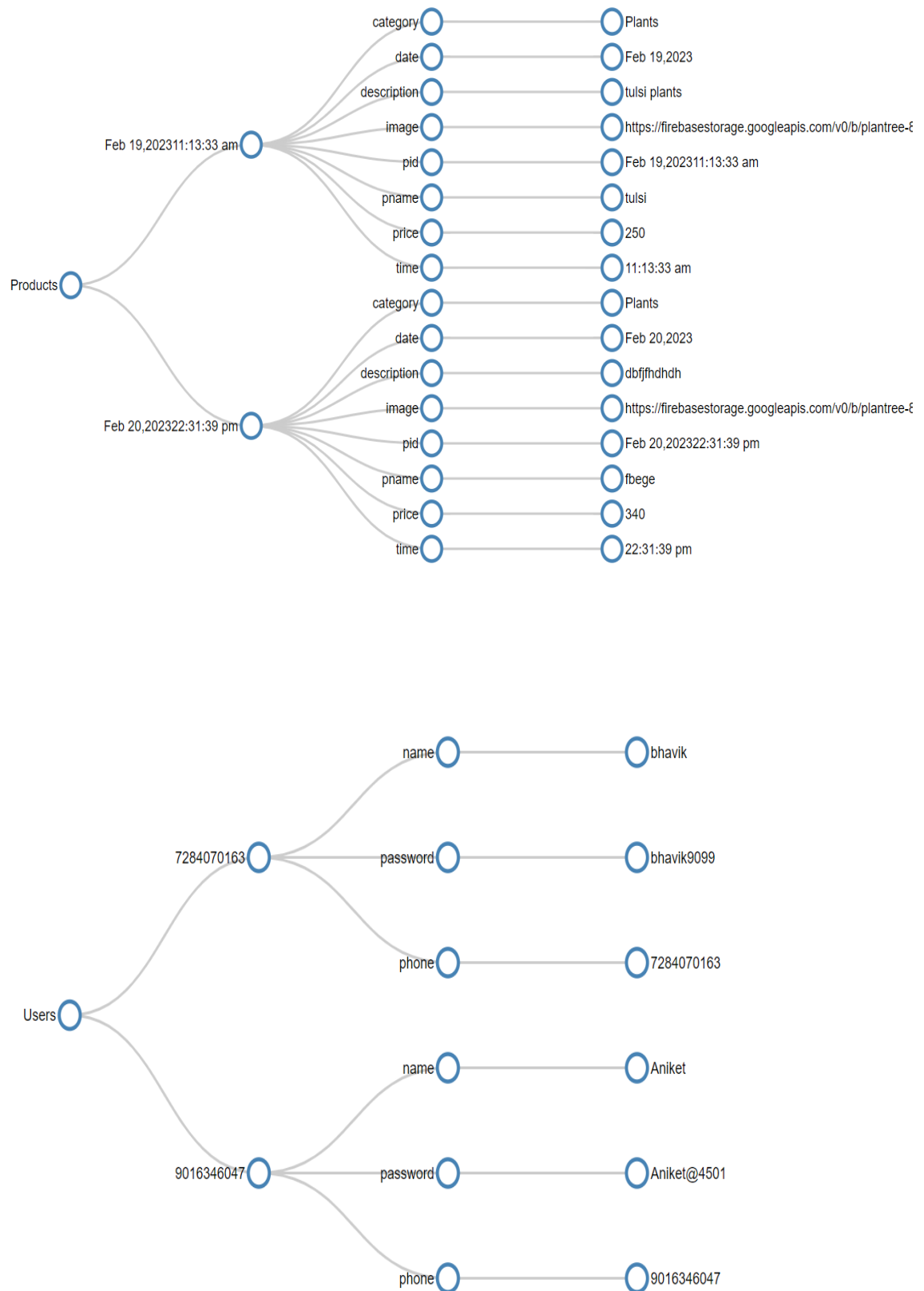


PLANTREE E-COMMERCE APPLICATION





PLANTREE E-COMMERCE APPLICATION





5.2.1 Justification of Normalization

Normalization

Normalization is a process of organizing data in a database to reduce redundancy and improve data integrity. It involves breaking down a larger table into smaller, related tables, and creating relationships between them. Normalization is usually applied to structured data, which is data that is organized into a specific format or schema.

Unstructured data, on the other hand, refers to data that is not organized in a predefined manner and does not follow a specific schema. Examples of unstructured data include text documents, images, audio files, and social media posts. Unstructured data cannot be easily organized using normalization techniques since it lacks a predefined structure.

However, there are other techniques that can be used to organize unstructured data, such as data clustering, categorization, and classification. These techniques can help to group similar data together and identify patterns and relationships within the data.

In summary, normalization is not possible in unstructured data since it lacks a predefined structure. However, there are other techniques that can be used to organize unstructured data and improve its usability and relevance.

What is Normalization?

- Normalization is the process of organizing the data in the database.
- Normalization is used to minimize the redundancy from a relation or set of relations. It is also used to eliminate undesirable characteristics like Insertion, Update, and Deletion Anomalies.
- Normalization divides the larger table into smaller and links them using relationships.
- The normal form is used to reduce redundancy from the database table.

Why do we need Normalization?

The main reason for normalizing the relations is removing these anomalies. Failure to eliminate anomalies leads to data redundancy and can cause data integrity and other problems as the database grows. Normalization consists of a series of guidelines that helps to guide you in creating a good database structure.

Types of Normal Forms:

Normalization works through a series of stages called Normal forms. The normal forms apply to individual relations. The relation is said to be in particular normal form if it satisfies constraints.



Normal Form	Description
<u>1NF</u>	A relation is in 1NF if it contains an atomic value.
<u>2NF</u>	A relation will be in 2NF if it is in 1NF and all non-key attributes are fully functional dependent on the primary key.
<u>3NF</u>	A relation will be in 3NF if it is in 2NF and no transition dependency exists.
BCNF	A stronger definition of 3NF is known as Boyce Codd's normal form.
<u>4NF</u>	A relation will be in 4NF if it is in Boyce Codd's normal form and has no multi-valued dependency.
<u>5NF</u>	A relation is in 5NF. If it is in 4NF and does not contain any join dependency, joining should be lossless.

Advantages of Normalization

- Normalization helps to minimize data redundancy.
- Greater overall database organization.
- Data consistency within the database.
- Much more flexible database design.
- Enforces the concept of relational integrity.

Disadvantages of Normalization

- You cannot start building the database before knowing what the user needs.
- The performance degrades when normalizing the relations to higher normal forms, i.e., 4NF, 5NF.
- It is very time-consuming and difficult to normalize relations of a higher degree.
- Careless decomposition may lead to a bad database design, leading to serious problems.



5.3 Architectural Design

The architectural design of a Plantree e-commerce application would typically involve identifying the system requirements, breaking down the system into its constituent parts or components, and designing the interfaces and interactions between those components.

Here is a possible high-level architecture for the Plantree e-commerce application:

Presentation Layer: This layer would be responsible for the user interface of the application, including web pages, mobile apps, and other interfaces that users would interact with.

Application Layer: This layer would contain the business logic of the application, including the processing of user input, validation of user data, and the execution of business rules and workflows.

Data Access Layer: This layer would be responsible for managing the data storage and retrieval functions of the application, including database management and access, caching, and other data-related functions.

Infrastructure Layer: This layer would provide the underlying infrastructure and services that support the other layers of the application, including cloud hosting, security, monitoring, and other related functions.

In addition, the following components or subsystems may be included within the architecture:

Catalog Management: This subsystem would be responsible for managing the product catalog of the application, including the creation, updating, and deletion of products, categories, and other related information.

Order Management: This subsystem would be responsible for managing the order fulfillment process of the application, including the processing of orders, tracking of shipments, and other related functions.

Payment Gateway Integration: This subsystem would be responsible for integrating with third-party payment gateways to enable users to make payments for purchases made through the application.

Customer Management: This subsystem would be responsible for managing the customer data and related functions, including registration, login, profile management, and other related functions.



5.4 Algorithm Development / Pseudo-code

Algorithm development for a Plantree e-commerce application involves designing and implementing a set of logical steps that will allow the application to function as intended. The algorithms would be specific to each feature or task in the application, such as user authentication, product search and filtering, shopping cart management, and order processing.

Here are some examples of algorithms that might be used in a Plantree e-commerce application:

User authentication algorithm:

- a. Prompt the user to enter their login credentials.
- b. Compare the entered credentials with the credentials stored in the user database.
- c. If the entered credentials match a user's credentials, grant access to the user.
- d. If the entered credentials do not match any user's credentials, display an error message and prompt the user to try again.

Product search and filtering algorithm:

- a. Allow the user to enter a search query.
- b. Compare the search query with the name and description of all products in the database.
- c. Display the products that match the search query.
- d. Allow the user to filter the displayed products by category, price range, etc.

Shopping cart management algorithm:

- a. Allow the user to add a product to their cart.
- b. If the product is already in the cart, increase its quantity by one.
- c. If the product is not in the cart, add it to the cart with a quantity of one.
- d. Allow the user to remove a product from their cart.
- e. Allow the user to update the quantity of a product in their cart.

Order processing algorithm:

- a. Retrieve the user's cart and shipping and billing information.
- b. Calculate the total price of the items in the cart.
- c. Process the payment using the selected payment method.
- d. Generate an order confirmation with the details of the order and send it to the user.

These are just a few examples of algorithms that might be used in a Plantree e-commerce application. The actual algorithms used would depend on the specific requirements and functionality of the application.



PLANTREE E-COMMERCE APPLICATION

Here's a sample pseudo-code for a Plantree e-commerce application:

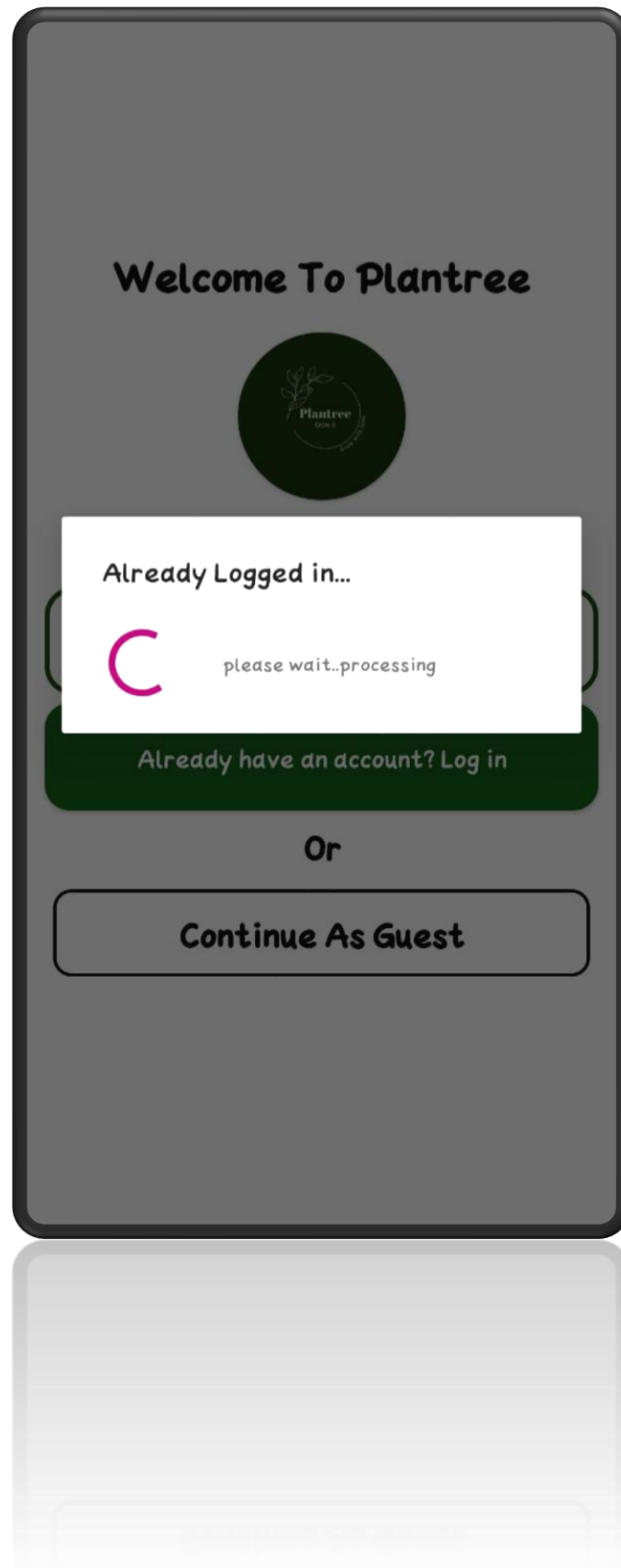
1. Start
2. Display the welcome message and available options to the user.
3. Prompt the user to choose an option (e.g. login, register, browse products, view cart, etc.)
4. If the user selects "login":
 - a. Prompt the user to enter their login credentials.
 - b. Authenticate the user's credentials and grant access if they are valid.
 - c. If the authentication fails, display an error message and prompt the user to try again or register.
5. If the user selects "register":
 - a. Prompt the user to enter their personal information, such as name, email address, password, etc.
 - b. Validate the information entered by the user.
 - c. Create a new user account with the provided information and grant access.
6. If the user selects "browse products":
 - a. Display a list of available products for sale, including their name, description, price, and image.
 - b. Allow the user to filter the products by category, price range, etc.
 - c. If the user selects a product, display its details and allow them to add it to their cart.
7. If the user selects "view cart":
 - a. Display the items currently in the user's cart, including their name, description, price, and quantity.
 - b. Allow the user to update the quantity of items in their cart or remove items from their cart.
 - c. Display the total price of all items in the cart.
8. If the user selects "checkout":
 - a. Prompt the user to enter their shipping and billing information.
 - b. Validate the information entered by the user.
 - c. Process the payment and generate an order confirmation for the user.
9. If the user selects "logout":
 - a. End the user's session and return them to the login screen.
10. End

Note that this is just a sample pseudo-code, and the actual implementation may vary depending on the specific requirements and functionality of your Plantree e-commerce application.



5.5 User Interface Design







Welcome To Plantree



Plantree! Grow With Love

Join Now

Already have an account? Log in

Or

Continue As Guest



Signup With Plantree

Sign Up



Login With Plantree

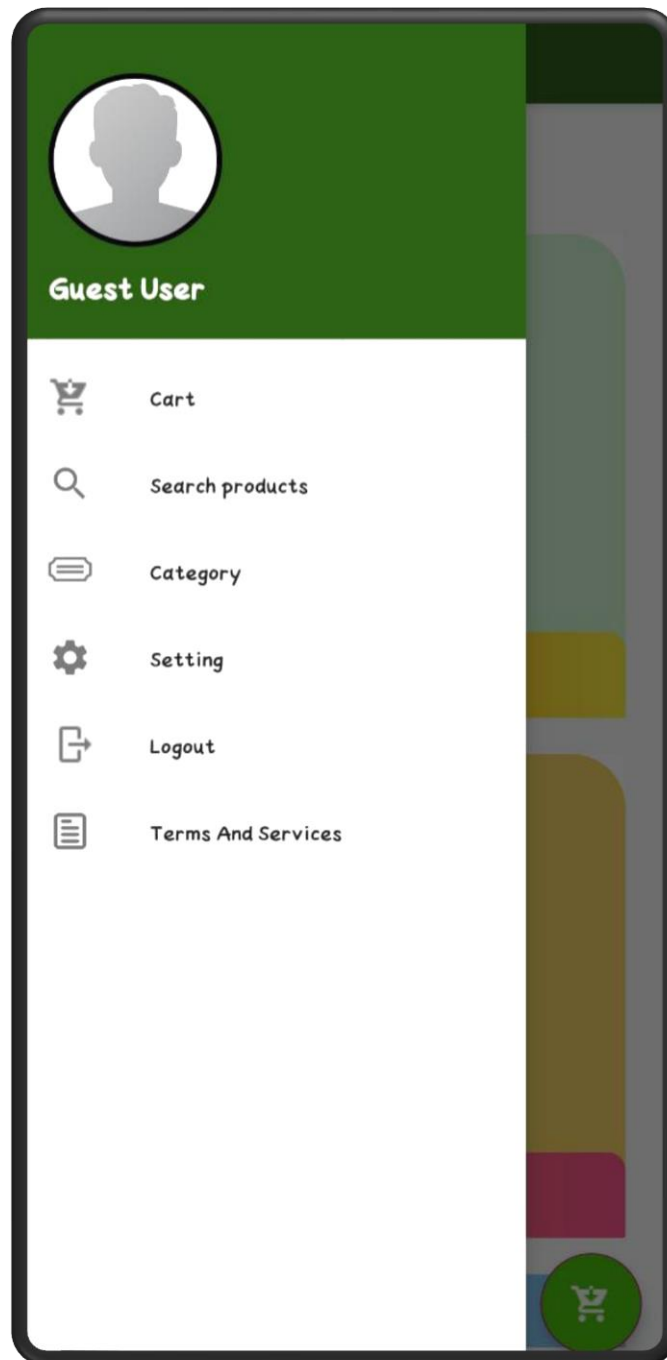
phone number

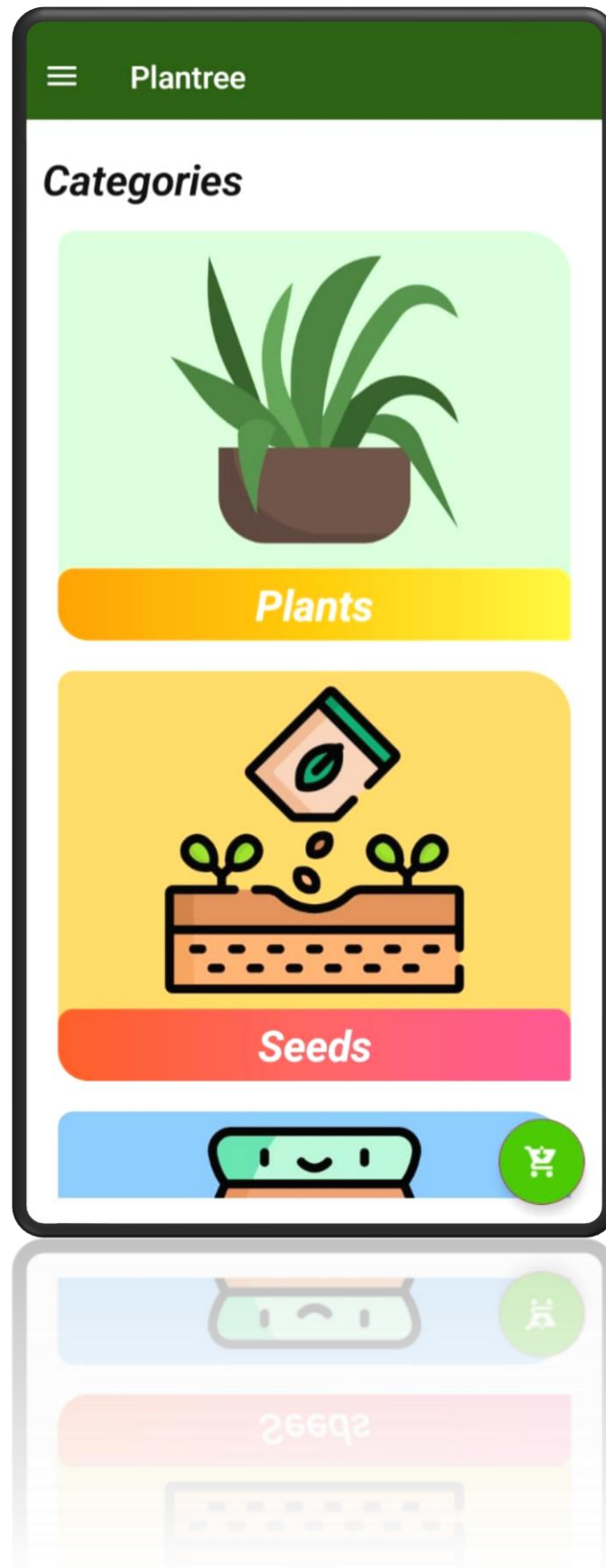
password

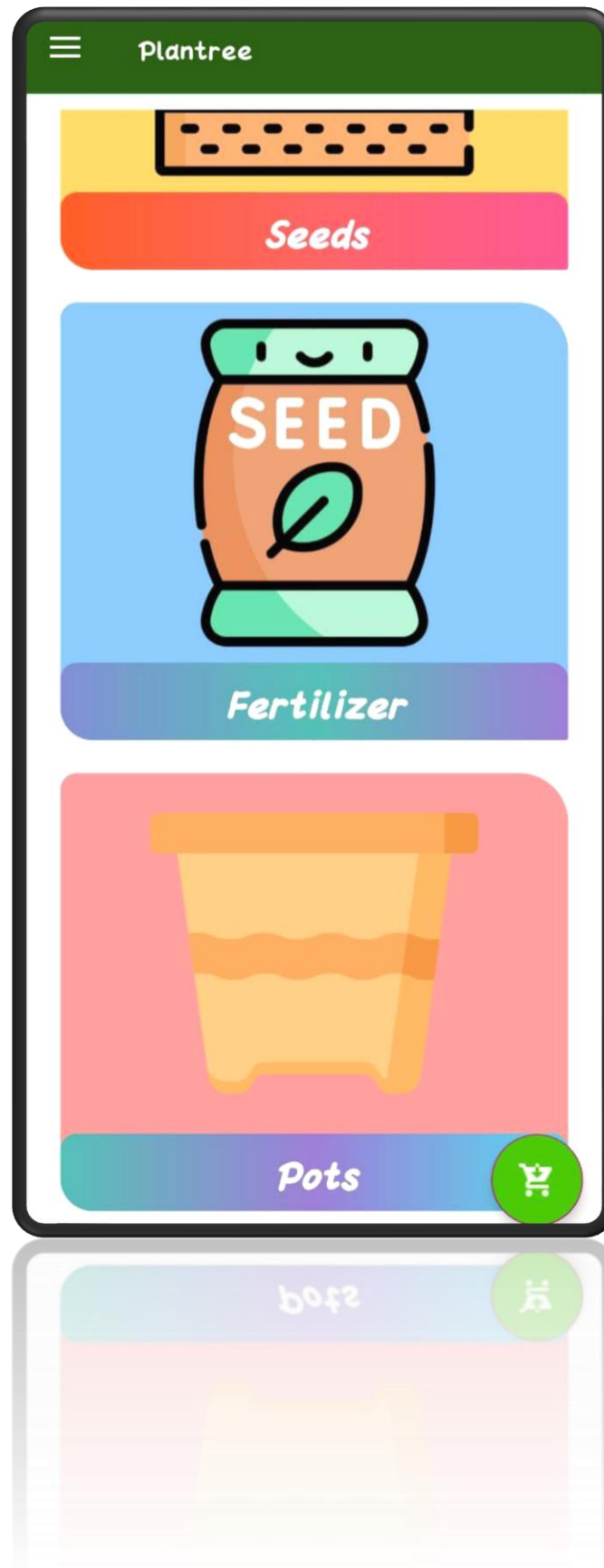
☐ Remember me

[Forget password](#)

Log in









Plantree Terms And Services

Welcome to Plantree! These terms and conditions outline the rules and regulations for the use of Plantree's App. By accessing this App we assume you accept these terms and conditions. Do not continue to use Plantree if you do not agree to take all of the terms and conditions stated on this Screen.

License

Unless otherwise stated, Plantree and/or its licensors own the intellectual property rights for all material on Plantree. All intellectual property rights are reserved. You may access this from Plantree for your own personal use subjected to restrictions set in these terms and conditions. Parts of this App offer an opportunity for users to Buy the different types of plants and also to sell the plants at a

time. Plantree does not filter
any of the content posted by
users. Plantree is not responsible
for any content posted by users.
Plantree reserves the right to
remove any content that is
in violation of its terms and
conditions. Plantree reserves
the right to suspend or terminate
any user who violates its terms
and conditions. Plantree reserves
the right to take any action it
deems fit in order to protect its
rights and the rights of its users.

OK



Close**Update**

Update image

Set security question

Become Seller



Search product

SEARCH



Mogra Plant, *Jasminum sambac*, Arabian Jasmine- Plant

Price in Rs.299

Description



Mogra plant, also known as *Jasminum sambac* or Arabian jasmine, is a beautiful and fragrant flowering plant that belongs to the family Oleaceae. It is native to South Asia, particularly India, and is widely cultivated for its ornamental and medicinal properties.

The Mogra plant grows as a shrub or a small tree, reaching up to 3 meters in height. It has dark green, shiny leaves that are oval-shaped and arranged in pairs. The plant produces small, white, star-shaped flowers that are highly fragrant and bloom



Search product



**Mogra Plant,
Jasminum
sambac,
Arabian
Jasmine- Plant**

Price in Rs.299



**Miniature
Rose, Button
Rose(Any
Color)-Plant**

Price in Rs.259



Peace Lily,



Parijat Tree,

Peace Lily

Parijat Tree



Search product



**Cherry Tomato
Seeds**

Price in Rs.175



**Italian Basil
Seeds**

Price in Rs.199



**Basil Purple
Color | Herb
Seeds**



Dill Herb Seeds

Price in Rs.166

Seeds
Color | Herb
Basil Purple

Price in Rs.166
Dill Herb Seeds



Search product



**PerfoSil Ecocert
Certified Plant
Immunity &
Yield Booster
of Plants,
Bio Available
Stabilized Silica
3%, Sorbitol 15%**

Price in Rs.514



View V

**Utkarsh
Phosphoz
(Phosphate
Solubilizing
Bacteria – PSB)
Bio Fertilizers**

Price in Rs.450





Search product



**Ugaoo Krish
Self Watering
Planter -7.5 inch
(Set of 5)**

Price in Rs.1299



**Phoenix
Ceramic Pot
(12.5 Inch
Diameter)**

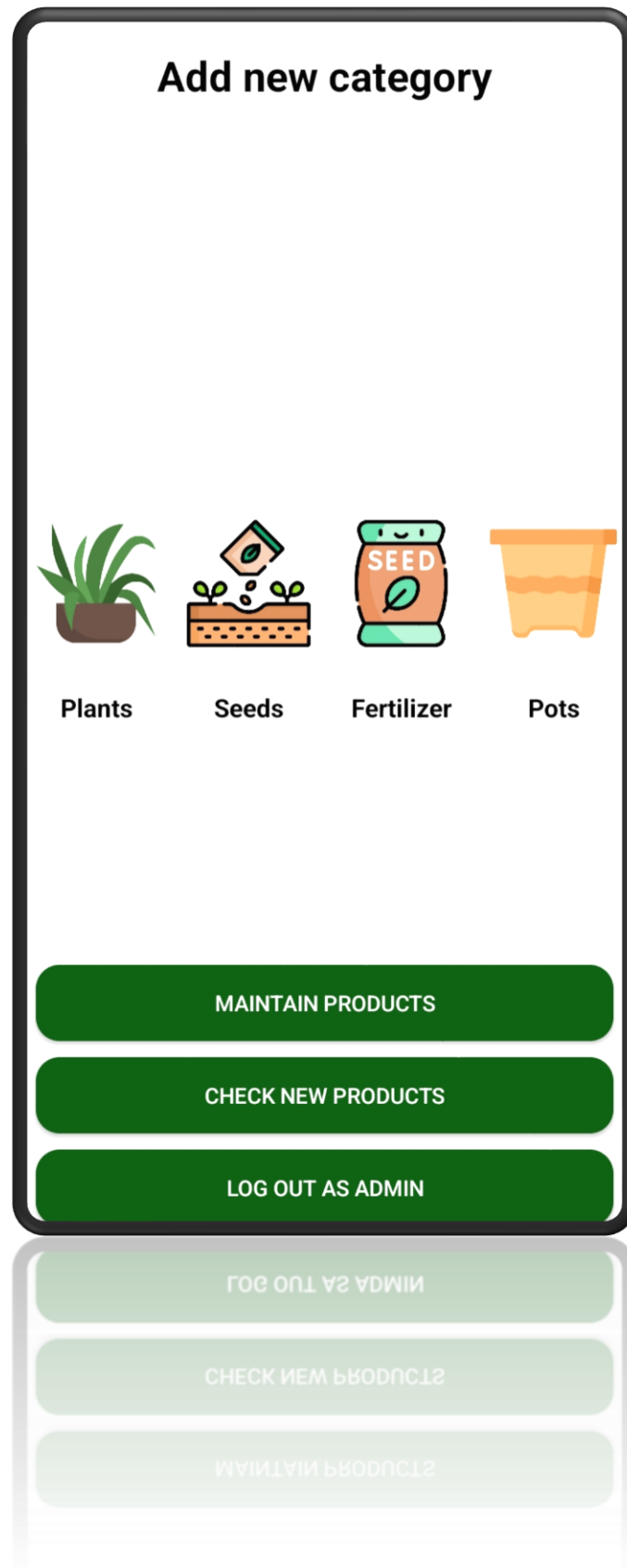
Price in Rs.2999



**Six Tier Durable
Pot Stand**



**Natures Hum
604XS Ceramic**





Enter product name

Enter product description

Enter product price

add product

add product



Plase confirm your shipping detail

CONFIRM

CONFIRM



Total Paying price= Rs 598

**Mogra Plant, Jasminum
sambac, Arabian Jasmine-
Plant**

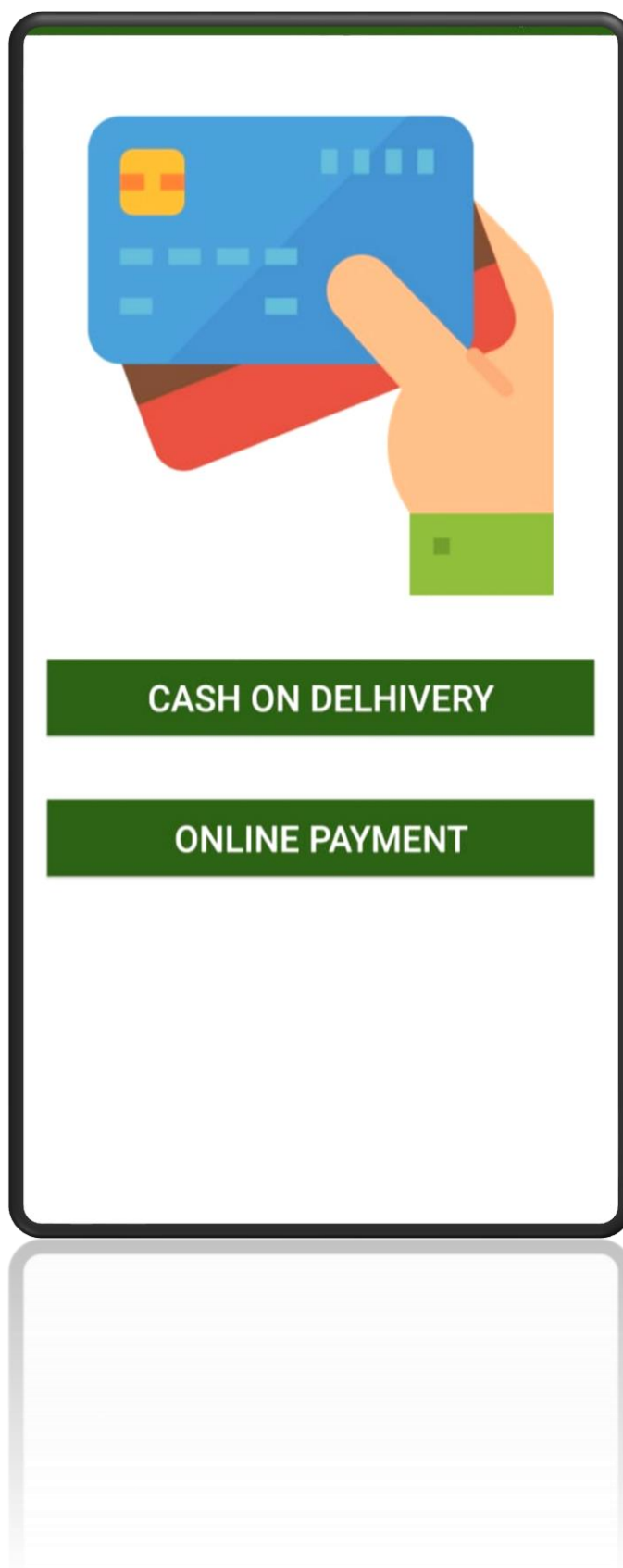
Quantity: 1

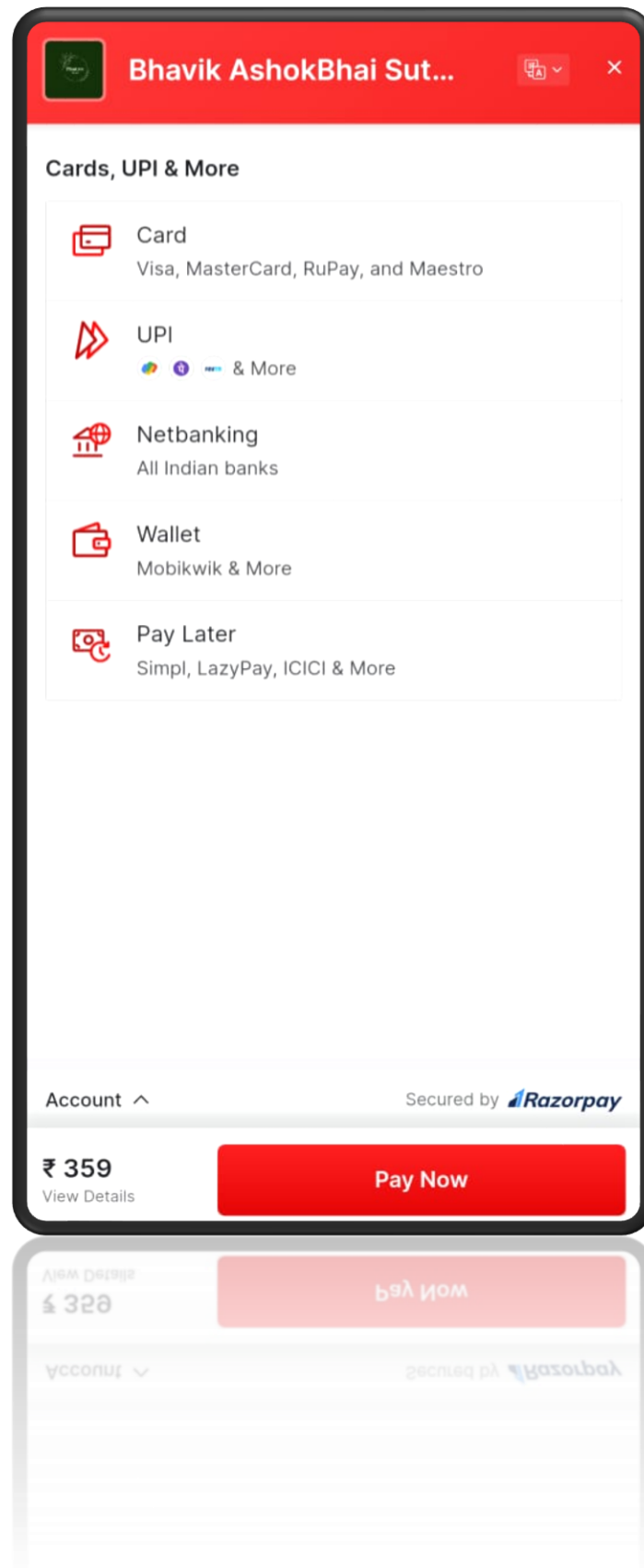
Price: Rs299

**Congratulation your order has
been placed succesfully soon you
will receive that at your door steps**

PROCEED TO NEXT

PROCEED TO NEXT







5.6 Security Issues

Plantree e-commerce application may face several security issues that could compromise the confidentiality, integrity, and availability of its users' data. Some of the security issues that may arise in this application are:

Cross-Site Scripting (XSS): This is a security vulnerability that allows attackers to inject malicious code into a web page viewed by other users. The malicious code can steal sensitive information such as passwords, credit card details, or session tokens.

SQL Injection: This is an attack that allows an attacker to inject malicious SQL commands into a database query. The attacker can manipulate, delete, or steal sensitive information from the database.

Cross-Site Request Forgery (CSRF): This is a type of attack where an attacker tricks a user into performing an action on a website without their knowledge or consent. For example, an attacker could trick a user into transferring money from their bank account to the attacker's account.

Broken Authentication and Session Management: This is a vulnerability that occurs when authentication and session management mechanisms are poorly implemented or configured. Attackers can exploit this vulnerability to gain access to user accounts or hijack user sessions.

Insufficient Authorization: This is a vulnerability that occurs when access controls are poorly implemented or configured. Attackers can exploit this vulnerability to gain unauthorized access to sensitive resources or functionality.

Insecure Communication: This is a vulnerability that occurs when sensitive information is transmitted over unencrypted channels, allowing attackers to intercept and read the information.

Payment System Security: Plantree must ensure that its payment system is secure and PCI DSS compliant, protecting customers' credit card information.

Denial of Service (DoS) Attacks: Plantree should be prepared to mitigate DoS attacks that can affect its website's availability.

Mobile Application Security: Plantree must ensure that its mobile application is secure and free from vulnerabilities that could be exploited by attackers.

Social Engineering Attacks: Plantree should be aware of social engineering attacks that can trick users into divulging sensitive information, such as passwords or credit card details.



5.7 Quality / Reliability Measures

Here are some general quality and reliability measures that apply to the plantree e-commerce application:

Functionality: The application should be able to handle all e-commerce functions such as product catalogue, shopping cart, payment processing, order management, and shipment tracking.

Usability: The application should have a user-friendly interface that is easy to navigate and provides clear information about products, prices, and shipping options.

Security: The application should use secure protocols such as SSL/TLS encryption for data transfer and storage, and implement measures to prevent hacking, data breaches, and fraud.

Performance: The application should be able to handle a large number of simultaneous users and transactions, with fast loading times and low latency.

Availability: The application should be available 24/7, with minimal downtime for maintenance or upgrades.

Scalability: The application should be designed to handle increased traffic and transactions as the business grows, without impacting performance or user experience.

Compatibility: The application should work across multiple platforms and devices, including desktop and mobile devices, and support multiple browsers and operating systems.

Reliability: The application should be stable and reliable, with minimal bugs and errors that could affect user experience or data integrity.

Maintainability: The application should be easy to maintain and update, with good documentation and a well-organized codebase.

Customer support: The application should have an efficient and responsive customer support system that can handle customer inquiries, complaints, and feedback.



5.8 System Map

User Interface: This is the front-end of the application that the users interact with. It includes web pages, menus, buttons, and forms that allow users to browse products, add items to their shopping cart, checkout, and make payments.

Product Catalogue: This is the database that contains all the products available for purchase on the application. It includes product details such as images, descriptions, prices, and stock levels.

Shopping Cart: This is the section of the application that stores the items selected by the user for purchase. It allows users to view, edit, and remove items from their cart before proceeding to checkout.

Order Management: This is the back-end system that handles the processing and tracking of orders. It includes order processing, order tracking, and shipment tracking.

Payment Gateway: This is the system that handles the processing of payments made by users. It includes integration with third-party payment processors, such as PayPal or Stripe, and secure encryption of payment information.

User Management: This is the system that manages user accounts, authentication, and authorization. It includes user registration, login, and password reset functionality.

Customer Service: This is the section of the application that provides customer support to users. It includes a help centre, FAQs, and a contact form or live chat system for users to reach out to customer service representatives.

Analytics and Reporting: This is the system that collects and analyses data on user behaviour, sales, and inventory levels. It provides insights into the performance of the application, and allows for optimization and improvement of the application over time.



PLANTREE E-COMMERCE APPLICATION





6. SOFTWARE CODING

6.1 Tools & Techniques

- Front-end development: We have use XML to develop the user interface and front-end functionality of the Plantree e-commerce application.
- Back-end development: We have use a programming language such as JAVA to develop the server-side functionality of the Plantree e-commerce application.
- IDE (Integrated development environment) : Android Studio version Electric Eel | 2022.1.1 Required plugin version - 3.2-7.4
- Database management: JSON (JavaScript Object Notation) is a lightweight, text-based format for representing data. It is often used for data interchange between systems and for storing configuration data. JSON is not a database management system (DBMS) like MySQL or MongoDB, but it can be used to store data in a file or in-memory data structure.
- API integration: We will integrate third-party APIs such as payment gateways, shipping providers, and social media platforms to enable secure payment processing, efficient order fulfillment, and social media integration.
- Version control: We will use Git as our version control system to track changes to the Plantree e-commerce application codebase and collaborate with team members.

By utilizing these tools and techniques, we will be able to develop a robust and scalable e-commerce application that provides a seamless shopping experience for our customers, enables secure payment processing and efficient order fulfillment, and drives more sales for the Plantree online store.



6.2 Business Logic

A Business logic in software coding refers to the code that implements the set of rules, processes, and workflows that define how a business application operates and makes decisions. This code is responsible for processing data, enforcing business rules, performing calculations, and making decisions based on the inputs and outputs of the system.

The business logic code is typically written using a programming language, and is often separated from other components of the application, such as the user interface or data storage layers. This separation promotes modularity and flexibility, allowing developers to update and modify the business logic independently of other components.

Effective business logic code requires a clear understanding of the business requirements and goals, as well as the technical skills necessary to implement the logic in code. Business analysts, software architects, and developers work together to identify the business rules and workflows, design the architecture of the application, and write the code that implements the logic.

Login Activity: -

```
package com.weare.plantree;

import androidx.annotation.NonNull;
import androidx.appcompat.app.AppCompatActivity;

import android.app.ProgressDialog;
import android.content.Intent;
import android.os.Bundle;
import android.text.TextUtils;
import android.view.View;
import android.widget.Button;
import android.widget.EditText;
import android.widget.TextView;
import android.widget.Toast;

import com.google.firebase.database.DataSnapshot;
import com.google.firebase.database.DatabaseError;
import com.google.firebase.database.DatabaseReference;
import com.google.firebase.database.FirebaseDatabase;
import com.google.firebase.database.ValueEventListener;
import com.rey.material.widget.CheckBox;
import com.weare.plantree.Admin.AdminCategoryActivity;
import com.weare.plantree.Buyers.ResetPasswordActivity;
import com.weare.plantree.Model.Users;
import com.weare.plantree.Prevalent.Prevalent;

import io.paperdb.Paper;

public class LoginActivity extends AppCompatActivity {
    private Button loginButton;
    private EditText phoneEditText,passwordEditText;
```




PLANTREE E-COMMERCE APPLICATION

```
private CheckBox remember_me_check_box;
private ProgressDialog loadingbar;
private String parentDbName="Users";

private TextView admin_panel_link;
private TextView no_admin_panel_text;
private TextView forgetPassword;

@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_login);
    loadingbar = new ProgressDialog(this);
    loginButton=findViewById(R.id.login_now_btn);
    phoneEditText=findViewById(R.id.login_phone_number_input);
    passwordEditText=findViewById(R.id.login_password_input);
    remember_me_check_box=findViewById(R.id.remember_me_check_box);

    loginButton.setOnClickListener(new View.OnClickListener() {
        @Override
        public void onClick(View v) {
            login();
        }
    });

    //paper io for storage
    Paper.init(this);

    //for admin
    admin_panel_link=findViewById(R.id.admin_panel_link);
    no_admin_panel_text=findViewById(R.id.not_admin_panel_link);

    admin_panel_link.setOnClickListener(new View.OnClickListener()
    {
        @Override
        public void onClick(View v) {
            loginButton.setText(R.string.log_in_as_admin);
            admin_panel_link.setVisibility(View.INVISIBLE);
            no_admin_panel_text.setVisibility(View.VISIBLE);
            parentDbName="Admins";
            remember_me_check_box.setVisibility(View.GONE);
        }
    });
    no_admin_panel_text.setOnClickListener(new
    View.OnClickListener() {
        @Override
        public void onClick(View v) {
            loginButton.setText(R.string.login_text);
            admin_panel_link.setVisibility(View.VISIBLE);
            no_admin_panel_text.setVisibility(View.INVISIBLE);

            parentDbName="Users";
            remember_me_check_box.setVisibility(View.GONE);
        }
    });
}
```



```
//forgot pass
forgotPassword=findViewById(R.id.forget_password_link);
forgotPassword.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {
        Intent intent=new Intent(LoginActivity.this,
ResetPasswordActivity.class);
        intent.putExtra("check","login");
        startActivity(intent);
    }
});

private void logIn() {
    String phone = phoneEditText.getText().toString();
    String password = passwordEditText.getText().toString();

    if (TextUtils.isEmpty(phone)) {
        Toast.makeText(this, R.string.enter_phone_number_text,
Toast.LENGTH_SHORT).show();
    } else if (TextUtils.isEmpty(password)) {
        Toast.makeText(this, R.string.enter_password_text,
Toast.LENGTH_SHORT).show();
    } else {
        loadingbar.setTitle("login...");

        loadingbar.setMessage(getString(R.string.please_wait_text));
        loadingbar.setCanceledOnTouchOutside(false);
        loadingbar.show();
        AllowAscessToAccount(phone, password);
    }
}

private void AllowAscessToAccount(final String phone, final String
password) {

    final DatabaseReference rootRef;
    rootRef = FirebaseDatabase.getInstance().getReference();

    rootRef.addListenerForSingleValueEvent(new ValueEventListener()
{
        @Override
        public void onDataChange(@NonNull DataSnapshot snapshot) {

            if
(snapshot.child(parentDbName).child(phone).exists()) {
                Users
users=snapshot.child(parentDbName).child(phone).getValue(Users.class);
                assert users != null;
                if (users.getPhone().equals(phone)) {
                    if (users.getPassword().equals(password)) {
```



PLANTREE E-COMMERCE APPLICATION

```
Toast.makeText(LoginActivity.this, "logged
in succesfully", Toast.LENGTH_SHORT).show();
loadingbar.dismiss();
if (remember_me_check_box.isChecked())
{
Paper.book().write(Prevalent.UserphoneKey, phone);
Paper.book().write(Prevalent.UserPasswordKey, password);

}

if (parentDbName.equals("Admins"))
{
Intent intent=new
Intent(LoginActivity.this, AdminCategoryActivity.class);
startActivity(intent);
finish();
}
else if (parentDbName.equals("Users"))
{
Paper.book().write(Prevalent.currentOnlineUser, users);
//
Paper.book().write(Prevalent.currentOnlineUser, users);
Intent intent=new
Intent(LoginActivity.this, TermsActivity.class);
startActivity(intent);
finish();
}
}
else
{
loadingbar.dismiss();
Toast.makeText(LoginActivity.this,
"Incorrect password", Toast.LENGTH_SHORT).show();
}
}
else {
loadingbar.dismiss();
}
}
else
{
Toast.makeText(LoginActivity.this, "account with
this phone number "+phone+" does not exists",
Toast.LENGTH_SHORT).show();
loadingbar.dismiss();
}
}

@Override
public void onCancelled(@NonNull DatabaseError error) {
```



PLANTREE E-COMMERCE APPLICATION

```
    }  
    });  
}  
  
}
```

Register Activity : -

```
package com.weare.plantree;  
  
import androidx.annotation.NonNull;  
import androidx.appcompat.app.AppCompatActivity;  
  
import android.app.ProgressDialog;  
import android.content.Intent;  
import android.os.Bundle;  
import android.text.TextUtils;  
import android.view.View;  
import android.widget.Button;  
import android.widget.EditText;  
import android.widget.Toast;  
  
import com.google.android.gms.tasks.OnCompleteListener;  
import com.google.android.gms.tasks.Task;  
import com.google.firebase.database.DataSnapshot;  
import com.google.firebase.database.DatabaseError;  
import com.google.firebase.database.DatabaseReference;  
import com.google.firebase.database.FirebaseDatabase;  
import com.google.firebase.database.ValueEventListener;  
  
import java.util.HashMap;  
  
public class RegisterActivity extends AppCompatActivity {  
  
    private Button signUpButton;  
    private EditText phoneEditText, userNameEditText, passwordEditText;  
    private ProgressDialog loadingbar;  
  
    @Override  
    protected void onCreate(Bundle savedInstanceState) {  
        super.onCreate(savedInstanceState);  
        setContentView(R.layout.activity_register);  
        signUpButton = findViewById(R.id.sign_up);  
        phoneEditText = findViewById(R.id.register_phone_number_input);  
        passwordEditText = findViewById(R.id.register_password_input);  
        userNameEditText = findViewById(R.id.register_user_name_input);  
        loadingbar = new ProgressDialog(this);  
  
        signUpButton.setOnClickListener(new View.OnClickListener() {  
            @Override  
            public void onClick(View v) {  
                createAccount();  
            }  
        })  
    }  
}
```



PLANTREE E-COMMERCE APPLICATION

```
    });  
  
    }  
  
    private void createAccount() {  
        String phone = phoneEditText.getText().toString();  
        String userName = userNameEditText.getText().toString();  
        String password = passwordEditText.getText().toString();  
  
        if (TextUtils.isEmpty(userName)) {  
            Toast.makeText(this, "please enter name",  
Toast.LENGTH_SHORT).show();  
        } else if (TextUtils.isEmpty(phone)) {  
            Toast.makeText(this, "please enter phone number",  
Toast.LENGTH_SHORT).show();  
        } else if (TextUtils.isEmpty(password)) {  
            Toast.makeText(this, "please enter password",  
Toast.LENGTH_SHORT).show();  
  
        } else {  
            loadingbar.setTitle("Creating account");  
            loadingbar.setMessage("please wait..processing");  
            loadingbar.setCanceledOnTouchOutside(false);  
            loadingbar.show();  
            ValidatPhoneNumber(userName, phone, password);  
        }  
  
    }  
  
    private void ValidatPhoneNumber(final String userName, final String  
phone, final String password) {  
        final DatabaseReference rootRef;  
        rootRef = FirebaseDatabase.getInstance().getReference();  
  
        rootRef.addListenerForSingleValueEvent(new ValueEventListener()  
{  
            @Override  
            public void onDataChange(@NonNull DataSnapshot snapshot) {  
                if ((snapshot.child("Users").child(phone).exists())) {  
                    loadingbar.dismiss();  
                    Toast.makeText(RegisterActivity.this, "Phone number  
is already in use\nplease login or create account eith new phonenumber", Toast.LENGTH_SHORT).show();  
                    startActivity(new Intent(RegisterActivity.this,  
MainActivity.class));  
                    finish();  
  
                } else {  
                    HashMap<String, Object> userDataMap = new  
HashMap<>();  
  
                    userDataMap.put("phone", phone);  
                    userDataMap.put("password", password);  
                    userDataMap.put("name", userName);  
  
                    rootRef.child("Users").child(phone).updateChildren(userDataMap)
```



PLANTREE E-COMMERCE APPLICATION

```
.addOnCompleteListener(new
OnCompleteListener<Void>() {
    @Override
    public void onComplete(@NonNull
Task<Void> task) {
        if (task.isSuccessful()) {
            loadingbar.dismiss();

            Toast.makeText(RegisterActivity.this, "User Create",
            Toast.LENGTH_SHORT).show();

            startActivity(new
            Intent(RegisterActivity.this, LoginActivity.class));
            finish();
        } else {
            loadingbar.dismiss();

            Toast.makeText(RegisterActivity.this, "something went wrong\n please
            try again", Toast.LENGTH_SHORT).show();
        }
    }
});
}

@Override
public void onCancelled(@NonNull DatabaseError error) {
}
});
}

@Override
public void onBackPressed() {
    this.finish();
}
}
```

Reset Password Activity : -

```
package com.weare.plantree.Buyers;

import androidx.annotation.NonNull;
import androidx.appcompat.app.AlertDialog;
import androidx.appcompat.app.AppCompatActivity;

import android.content.DialogInterface;
import android.content.Intent;
import android.os.Bundle;
import android.text.TextUtils;
import android.view.View;
import android.widget.Button;
import android.widget.EditText;
import android.widget.TextView;
```



PLANTREE E-COMMERCE APPLICATION

```
import android.widget.Toast;

import com.google.android.gms.tasks.OnCompleteListener;
import com.google.android.gms.tasks.Task;
import com.google.firebase.database.DataSnapshot;
import com.google.firebase.database.DatabaseError;
import com.google.firebase.database.DatabaseReference;
import com.google.firebase.database.FirebaseDatabase;
import com.google.firebase.database.ValueEventListener;
import com.weare.plantree.LoginActivity;
import com.weare.plantree.Model.Users;
import com.weare.plantree.Prevalent.Prevalent;
import com.weare.plantree.R;

import java.util.HashMap;

import io.paperdb.Paper;

public class ResetPasswordActivity extends AppCompatActivity {
    private String check;
    private TextView title;
    private EditText phoneEditText, qn1EditText, qn2EditText;
    private Button verifyBtn;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_reset_password);
        check = getIntent().getStringExtra("check");
        title = findViewById(R.id.header_text_reset_password);
        phoneEditText = findViewById(R.id.phone_edit_reset);
        qn1EditText = findViewById(R.id.question_1);
        qn2EditText = findViewById(R.id.question_2);
        verifyBtn = findViewById(R.id.verifyBtn);
    }

    @Override
    protected void onStart() {
        super.onStart();
        if (check.equals("setting")) {
            Users mUsers =
                Paper.book().read(Prevalent.currentOnlineUser);
            final DatabaseReference ref1 =
                FirebaseDatabase.getInstance().getReference().child("Users").child(mUsers.getPhone());
            Toast.makeText(this, "From setting",
                Toast.LENGTH_SHORT).show();
            title.setText("Set Question");
            phoneEditText.setVisibility(View.GONE);
            verifyBtn.setText("Set");

            ref1.child("Security Questions").addValueEventListener(new
```



PLANTREE E-COMMERCE APPLICATION

```
ValueEventListener() {
    @Override
    public void onDataChange(@NonNull DataSnapshot
snapshot) {
        if (snapshot.exists()) {
            String qn1 =
snapshot.child("answer1").getValue().toString();
            String qn2 =
snapshot.child("answer2").getValue().toString();
            qn1EditText.setText(qn1);
            qn2EditText.setText(qn2);
        }
    }

    @Override
    public void onCancelled(@NonNull DatabaseError error) {
    }
});
verifyBtn.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {
        String qn1 = qn1EditText.getText().toString();
        String qn2 = qn2EditText.getText().toString();

        if
(TextUtils.isEmpty(qn1EditText.getText().toString()) &&
TextUtils.isEmpty(qn2EditText.getText().toString())) {
            Toast.makeText(ResetPasswordActivity.this,
"please answer two question", Toast.LENGTH_SHORT).show();
        } else {
            Paper.init(ResetPasswordActivity.this);

            HashMap<String, Object> userDataMap = new
HashMap<>();

            userDataMap.put("answer1", qn1.toLowerCase());
            userDataMap.put("answer2", qn2.toLowerCase());

            ref1.child("Security Questions").
                updateChildren(userDataMap)
                .addOnCompleteListener(new
OnCompleteListener<Void>() {
                    @Override
                    public void onComplete(@NonNull
Task<Void> task) {
                        if (task.isSuccessful()) {
                            Toast.makeText(ResetPasswordActivity.this, "Security question updated
succesfully", Toast.LENGTH_SHORT).show();
                            finish();
                        } else {
                            Toast.makeText(ResetPasswordActivity.this, "Somehing went wrong",
Toast.LENGTH_SHORT).show();
                        }
                    }
                })
            }
        }
    }
});
```




```
        }
    });
}

});
} else if (check.equals("login")) {
    title.setText("Reset Password");
    phoneEditText.setVisibility(View.VISIBLE);
    verifyBtn.setText("Verify");
    verifyBtn.setOnClickListener(new View.OnClickListener() {
        @Override
        public void onClick(View v) {
            ResetPassword();
        }
    });
    // ResetPassword();
}

}

private void ResetPassword() {
    final String phone = phoneEditText.getText().toString();
    final String qn1 = qn1EditText.getText().toString();
    final String qn2 = qn2EditText.getText().toString();
    if (TextUtils.isEmpty(qn1EditText.getText().toString()) &&
    TextUtils.isEmpty(qn2EditText.getText().toString()) &&
    TextUtils.isEmpty(phoneEditText.getText().toString())) {
        Toast.makeText(ResetPasswordActivity.this, "enter all
        required field", Toast.LENGTH_SHORT).show();
    } else {

        final DatabaseReference ref1 =
        FirebaseDatabase.getInstance().getReference().child("Users").child(phone);
        ref1.addValueEventListener(new ValueEventListener() {
            @Override
            public void onDataChange(@NonNull DataSnapshot
            snapshot) {
                if (snapshot.exists()) {
                    if (snapshot.hasChild("Security Questions")) {
                        String answer1 = snapshot.child("Security
                        Questions").child("answer1").getValue().toString();
                        String answer2 = snapshot.child("Security
                        Questions").child("answer2").getValue().toString();

                        if (!qn1.equals(answer1)) {
                            Toast.makeText(ResetPasswordActivity.this, "first answer incorrect",
                            Toast.LENGTH_SHORT).show();
                        } else if (!qn2.equals(answer2)) {
                            Toast.makeText(ResetPasswordActivity.this, "second answer incorrect",
                            Toast.LENGTH_SHORT).show();
                        } else {
```



PLANTREE E-COMMERCE APPLICATION

```
AlertDialog.Builder builder = new
AlertDialog.Builder(ResetPasswordActivity.this);
builder.setTitle("Enter new password");
final EditText passwordEditText = new
EditText(ResetPasswordActivity.this);
passwordEditText.setHint("Enter new
password");

passwordEditText.setPadding(20,20,20,20);
builder.setView(passwordEditText);
builder.setPositiveButton("Change", new
DialogInterface.OnClickListener() {
    @Override
    public void onClick(final
DialogInterface dialog, int which) {
        if
(TextUtils.isEmpty(passwordEditText.getText().toString())) {
Toast.makeText(ResetPasswordActivity.this, "password field cannot be
empty", Toast.LENGTH_SHORT).show();
        } else {
HashMap<String, Object>
hashMap = new HashMap<>();
hashMap.put("password",
passwordEditText.getText().toString());

refl.child("password").setValue(passwordEditText.getText().toString()).
addOnCompleteListener(new OnCompleteListener<Void>() {
    @Override
    public void
onComplete(@NonNull Task<Void> task) {
        if
(task.isSuccessful()) {
dialog.dismiss();

Toast.makeText(ResetPasswordActivity.this, "Password changed
succesfully", Toast.LENGTH_SHORT).show();

startActivity(new Intent(ResetPasswordActivity.this,
LoginActivity.class));

finish();
        } else {
Toast.makeText(ResetPasswordActivity.this, "something went wrong",
Toast.LENGTH_SHORT).show();

dialog.dismiss();

        }
    }
});
    }
}
});
```



PLANTREE E-COMMERCE APPLICATION

```
builder.setNegativeButton("Cancel", new
DialogInterface.OnClickListener() {
    @Override
    public void onClick(DialogInterface
dialog, int which) {
        dialog.dismiss();
    }
});
builder.show();
}
}

} else {
    Toast.makeText(ResetPasswordActivity.this, "No
user found with this phone :" + phone, Toast.LENGTH_SHORT).show();
}

@Override
public void onCancelled(@NonNull DatabaseError error) {
}
});
}
}
}
```

Search Product Activity : -

```
package com.weare.plantree.Buyers;

import androidx.annotation.NonNull;
import androidx.appcompat.app.AppCompatActivity;
import androidx.recyclerview.widget.LinearLayoutManager;
import androidx.recyclerview.widget.RecyclerView;

import android.content.Intent;
import android.os.Bundle;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.Button;
import android.widget.EditText;

import com.firebase.ui.database.FirebaseRecyclerAdapter;
import com.firebase.ui.database.FirebaseRecyclerOptions;
import com.google.firebase.database.DatabaseReference;
import com.google.firebase.database.FirebaseDatabase;
import com.weare.plantree.Admin.AdminMaintainProductsActivity;
import com.weare.plantree.Model.ProductsModal;
```



PLANTREE E-COMMERCE APPLICATION

```
import com.weare.plantree.R;
import com.weare.plantree.ViewHolder.ProductViewHolder;
import com.squareup.picasso.Picasso;

public class SearchProductActivity extends AppCompatActivity {

    private Button searchProductBtn;
    private EditText searchEditText;
    private RecyclerView recyclerViewSearch;
    private String searchInput;
    private String typeOfUser="";

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_search_product);
        searchProductBtn=findViewById(R.id.search_btn);
        searchEditText=findViewById(R.id.search_product_name);
        recyclerViewSearch=findViewById(R.id.search_recycler_view);
        recyclerViewSearch.setLayoutManager(new
LinearLayoutManager(this));
        recyclerViewSearch.setHasFixedSize(true);

        searchProductBtn.setOnClickListener(new View.OnClickListener()
{
            @Override
            public void onClick(View v) {
                searchInput=searchEditText.getText().toString();
                onStart();
            }
        });

        Intent intent=getIntent();
        Bundle bundle=intent.getExtras();
        if (bundle!=null)
        {
            typeOfUser=bundle.getString("Admin");
        }

    }

    @Override
    protected void onStart() {
        super.onStart();
        DatabaseReference reference=
FirebaseDatabase.getInstance().getReference().child("Products");

        FirebaseRecyclerOptions<ProductsModal> options=new
FirebaseRecyclerOptions.Builder<ProductsModal>()

        .setQuery(reference.orderByChild("pname").startAt(searchInput),Products
Modal.class)
        .build();
        FirebaseRecyclerAdapter<ProductsModal, ProductViewHolder>
```



PLANTREE E-COMMERCE APPLICATION

```
adapter=new FirebaseRecyclerAdapter<ProductsModal,
ProductViewHolder>(options) {
    @Override
    protected void onBindViewHolder(@NonNull ProductViewHolder
holder, int position, @NonNull final ProductsModal model) {
        holder.productTitle.setText(model.getPname());

holder.productDescription.setText(model.getDescription());
        holder.productPrice.setText(String.format("Price $%s ",
model.getPrice()));

Picasso.get().load(model.getImage()).into(holder.productImage);
        holder.itemView.setOnClickListener(new
View.OnClickListener() {
            @Override
            public void onClick(View v) {
                if (typeOfUser.equals("Admin"))
                {

                    String id = model.getPid();
                    Intent intent = new
Intent(SearchProductActivity.this,
AdminMaintainProductsActivity.class);
                    intent.putExtra("pid", id);
                    startActivity(intent);
                }
                else
                {
                    String id = model.getPid();
                    Intent intent = new
Intent(SearchProductActivity.this, ProductDetailsActivity.class);
                    intent.putExtra("pid", id);
                    startActivity(intent);
                }
            }
        });
    }

    @NonNull
    @Override
    public ProductViewHolder onCreateViewHolder(@NonNull
ViewGroup parent, int viewType) {
        return new
ProductViewHolder(LayoutInflater.from(parent.getContext()).inflate(R.la
yout.product_item_layout,parent,false));
    }

};
recyclerViewSearch.setAdapter(adapter);
adapter.startListening();
// adapter.
}
```



PLANTREE E-COMMERCE APPLICATION

Add to Cart Activity : -

```
package com.weare.plantree.Buyers;

import androidx.annotation.NonNull;
import androidx.appcompat.app.AppCompatActivity;

import android.app.Dialog;
import android.content.Intent;
import android.os.Bundle;
import android.util.Log;
import android.view.View;
import android.widget.Button;
import android.widget.ImageView;
import android.widget.TextView;
import android.widget.Toast;

import com.cepheuen.elegantnumberbutton.view.ElegantNumberButton;
import com.google.android.gms.tasks.OnCompleteListener;
import com.google.android.gms.tasks.Task;
import com.google.firebase.database.DataSnapshot;
import com.google.firebase.database.DatabaseError;
import com.google.firebase.database.DatabaseReference;
import com.google.firebase.database.FirebaseDatabase;
import com.google.firebase.database.ValueEventListener;
import com.weare.plantree.LoginActivity;
import com.weare.plantree.Model.ProductsModal;
import com.weare.plantree.Model.Users;
import com.weare.plantree.Prevalent.Prevalent;
import com.weare.plantree.R;
import com.squareup.picasso.Picasso;
import com.weare.plantree.RegisterActivity;

import java.text.SimpleDateFormat;
import java.util.Calendar;
import java.util.HashMap;

import io.paperdb.Paper;

public class ProductDetailsActivity extends AppCompatActivity {

    private String pid;
    DatabaseReference mReference;
    private ImageView productImage;
    private TextView nameTextView,descriptionTextView,priceTextView;
    private ElegantNumberButton mNumberButton;
    private Button addToCartBtn;

    String name,price,description;
    private String state="";
    private Users currentUser;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
```



PLANTREE E-COMMERCE APPLICATION

```
        setContentView(R.layout.activity_product_details);
        Intent intent=getIntent();
        pid=intent.getStringExtra("pid");
        Paper.init(this);
        currentUser= Paper.book().read(Prevalent.currentOnlineUser);

        mReference=
        FirebaseDatabase.getInstance().getReference().child("Products").child(p
        id);

        productImage=findViewById(R.id.imageView_detail);
        nameTextView=findViewById(R.id.product_name_detail);

        descriptionTextView=findViewById(R.id.product_description_detail);
        priceTextView=findViewById(R.id.product_price_detail);
        mNumberButton=findViewById(R.id.number_btn);
        addToCartBtn=findViewById(R.id.add_to_cart_button);

        updateProduct();

        addToCartBtn.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                if (state.equals("Item shipped" )| state.equals("Item
                is ready to be shipped")) {
                    Toast.makeText(ProductDetailsActivity.this, "You
                    can purchase item once the order is placed succesfully",
                    Toast.LENGTH_SHORT).show();
                }
                else
                {
                    addToCartList();
                }
            }
        });

        private void addToCartList() {
            if (currentUser == null){

                Dialog dialog = new Dialog(this);
                dialog.setContentView(R.layout.loginorregisterdialog);

                Button login = dialog.findViewById(R.id.gotologin);
                Button register = dialog.findViewById(R.id.gotoregister);
                Button cancel = dialog.findViewById(R.id.cancel);
                dialog.show();

                login.setOnClickListener(v->{
                    startActivity(new Intent(ProductDetailsActivity.this,
                    LoginActivity.class));
                    finish();
                });
            }
        }
    }
}
```



PLANTREE E-COMMERCE APPLICATION

```
        register.setOnClickListener(v->{
            startActivity(new Intent(ProductDetailsActivity.this,
RegisterActivity.class));
            finish();
        });

        cancel.setOnClickListener(v -> {
            dialog.dismiss();
        });
    }
    else {
        String saveCurrentTime, saveCurrentDate;
        Calendar calendarForDate = Calendar.getInstance();
        SimpleDateFormat currentDate = new SimpleDateFormat("MMM
dd,yyyy");
        saveCurrentDate =
currentDate.format(calendarForDate.getTime());

        Calendar calendarForTime = Calendar.getInstance();
        SimpleDateFormat currentTime = new
SimpleDateFormat("HH:mm:ss a");
        saveCurrentTime =
currentTime.format(calendarForTime.getTime());

        final DatabaseReference cartRef =
FirebaseDatabase.getInstance().getReference().child("Cart List");
        final HashMap<String, Object> productHashMap = new
HashMap<>();
        productHashMap.put("pid", pid);
        productHashMap.put("pname", name);
        productHashMap.put("price", price);
        // productHashMap.put("description",description);
        productHashMap.put("date", saveCurrentDate);
        productHashMap.put("time", saveCurrentTime);
        productHashMap.put("quantity", mNumberButton.getNumber());
        productHashMap.put("discount", "");
        //productHashMap.put("pid",pid);

        cartRef.child("User View")
            .child(currentUser.getPhone())
            .child("Products")
            .child(pid)
            .updateChildren(productHashMap)
            .addOnCompleteListener(new
OnCompleteListener<Void>() {
                @Override
                public void onComplete(@NonNull Task<Void>
task) {
                    if (task.isSuccessful()) {
                        cartRef.child("Admin View")
                            .child(currentUser.getPhone())
                            .child("Products")
                            .child(pid)
                            .updateChildren(productHashMap)
```




PLANTREE E-COMMERCE APPLICATION

```
.addOnCompleteListener(new
OnCompleteListener<Void>() {
    @Override
    public void
onComplete(@NonNull Task<Void> task) {
        if
        (task.isSuccessful()) {
            Toast.makeText(ProductDetailsActivity.this, "Added to cart sucesfully",
            Toast.LENGTH_SHORT).show();
        }
        finish();
    }
});

    } else {
        Toast.makeText(ProductDetailsActivity.this, "Something went wrong!!!
        try again", Toast.LENGTH_SHORT).show();
    }
});
    }
}

private void updateProduct() {
    mReference.addValueEventListener(new ValueEventListener() {
        @Override
        public void onDataChange(@NonNull DataSnapshot snapshot) {
            ProductsModal modal =
            snapshot.getValue(ProductsModal.class);
            if (modal!=null)
            {
                Picasso.get().load(modal.getImage()).into(productImage);
                name=modal.getPname();
                price=modal.getPrice();
                description=modal.getDescription();
                nameTextView.setText(modal.getPname());

                descriptionTextView.setText(modal.getDescription());
                priceTextView.setText("Price in
                Rs."+modal.getPrice());

            }
            else
            {
                Toast.makeText(ProductDetailsActivity.this,
                "Error", Toast.LENGTH_SHORT).show();
            }
        }
    }
}
```



```
        @Override
        public void onCancelled(@NonNull DatabaseError error) {
            Toast.makeText(ProductDetailsActivity.this, "Database
Error", Toast.LENGTH_SHORT).show();
        }
    });

    @Override
    protected void onStart()
    {
        super.onStart();
        checkOrdersState();
    }

    private void checkOrdersState() {
        if (currentUser == null) {
            Log.d("TAG", "checkOrdersState: ");
        } else {
            Paper.init(this);
            Users curentUser =
            Paper.book().read(Prevalent.currentOnlineUser);
            DatabaseReference ordersRef =
            FirebaseDatabase.getInstance().getReference().child("Orders").child(cur
            entUser.getPhone());
            ordersRef.addValueEventListener(new ValueEventListener() {
                @Override
                public void onDataChange(@NonNull DataSnapshot
snapshot) {
                    if (snapshot.exists()) {
                        String state =
                        snapshot.child("state").getValue().toString();
                        if (state.equals("shipped")) {
                            state = "Item shipped";
                        } else if (state.equals("not shipped")) {
                            state = "Item is ready to be shipped";
                        }
                    }
                }

                @Override
                public void onCancelled(@NonNull DatabaseError error) {

                }

            });
        }
    }
}
```



PLANTREE E-COMMERCE APPLICATION

Show Add to Cart : -

```
package com.weare.plantree.Buyers;

import androidx.annotation.NonNull;
import androidx.appcompat.app.AlertDialog;
import androidx.appcompat.app.AppCompatActivity;
import androidx.recyclerview.widget.LinearLayoutManager;
import androidx.recyclerview.widget.RecyclerView;

import android.app.Dialog;
import android.content.DialogInterface;
import android.content.Intent;
import android.os.Bundle;
import android.util.Log;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.Button;
import android.widget.TextView;
import android.widget.Toast;

import com.firebase.ui.database.FirebaseRecyclerAdapter;
import com.firebase.ui.database.FirebaseRecyclerOptions;
import com.google.android.gms.tasks.OnCompleteListener;
import com.google.android.gms.tasks.Task;
import com.google.firebase.auth.FirebaseAuth;
import com.google.firebase.auth.FirebaseUser;
import com.google.firebase.database.DataSnapshot;
import com.google.firebase.database.DatabaseError;
import com.google.firebase.database.DatabaseReference;
import com.google.firebase.database.FirebaseDatabase;
import com.google.firebase.database.ValueEventListener;
import com.weare.plantree.LoginActivity;
import com.weare.plantree.Model.CartModel;
import com.weare.plantree.Model.Users;
import com.weare.plantree.Prevalent.Prevalent;
import com.weare.plantree.R;
import com.weare.plantree.RegisterActivity;
import com.weare.plantree.ViewHolder.CartViewHolder;

import io.paperdb.Paper;

public class CartActivity extends AppCompatActivity {

    private TextView totalAmountTextView,msg1;
    private Button nextButton;
    private RecyclerView mRecyclerView;
    DatabaseReference cartRef;
    FirebaseAuth auth;
    FirebaseUser user;
    int totaPrice=0;
    Users curentUser;
    boolean firstttime=true;
```



PLANTREE E-COMMERCE APPLICATION

```
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_cart);

    totalAmountTextView=findViewById(R.id.total_price_cart_text_view);
    nextButton=findViewById(R.id.next_cartList);
    mRecyclerView=findViewById(R.id.cart_list_recycler_view);
    mRecyclerView.setLayoutManager(new LinearLayoutManager(this));
    //    mRecyclerView.setHasFixedSize(true);
    Paper.init(this);
    curentUser=Paper.book().read(Prevalent.currentOnlineUser);
    msg1=findViewById(R.id.msg1);

    checkOrdersState();

    nextButton.setOnClickListener(new View.OnClickListener() {
        @Override
        public void onClick(View v) {

            if (curentUser == null){
                Dialog dialog = new Dialog(CartActivity.this);

                dialog.setContentView(R.layout.loginorregisterdialog);

                Button login = dialog.findViewById(R.id.gotologin);
                Button register =
                dialog.findViewById(R.id.gotoregister);
                Button cancel = dialog.findViewById(R.id.cancel);
                dialog.show();

                login.setOnClickListener(vi->{
                    startActivity(new Intent(CartActivity.this,
LoginActivity.class));
                    finish();
                });

                register.setOnClickListener(vi->{
                    startActivity(new Intent(CartActivity.this,
RegisterActivity.class));
                    finish();
                });

                cancel.setOnClickListener(vi -> {
                    dialog.dismiss();
                });
            }
            else {
                if (firsttime) {
                    firsttime = false;
                    totalAmountTextView.setText("Total Paying
price= Rs " + String.valueOf(totaPrice));
                }
            }
        }
    });
}
```



```
        } else {

            Intent intent = new Intent(CartActivity.this,
ConfirmFinalOrderActivity.class);
            intent.putExtra("total", totaPrice);
            startActivity(intent);
        }
    }
});

}

@Override
protected void onStart() {
    super.onStart();
    if (curentUser == null){
        Log.d("TAG", "onStart: ");
    }
    else {
        cartRef =
FirebaseDatabase.getInstance().getReference().child("Cart
List").child("User View")
            .child(curentUser.getPhone())
            .child("Products");
        FirebaseRecyclerOptions<CartModel> options = new
FirebaseRecyclerOptions.Builder<CartModel>().setQuery(cartRef,
CartModel.class).build();

        FirebaseRecyclerAdapter<CartModel, CartViewHolder> adapter
= new FirebaseRecyclerAdapter<CartModel, CartViewHolder>(options) {
            @Override
            protected void onBindViewHolder(@NonNull CartViewHolder
holder, int position, @NonNull final CartModel model) {
                holder.titleTextView.setText(model.getPname());
                holder.quantityTextView.setText("Quantity: " +
model.getQuantity());
                holder.priceTextView.setText("Price: Rs" +
model.getPrice());

                int quantity =
Integer.parseInt(model.getQuantity());
                int p = Integer.parseInt(model.getPrice());
                int tp = (quantity * p);
                totaPrice = totaPrice + tp;
                holder.itemView.setOnClickListener(new
View.OnClickListener() {
                    @Override
                    public void onClick(View v) {
                        CharSequence option[] = new CharSequence[]{
                            "Edit"
                            , "Delete"
                        }
                    }
                })
            }
        }
    }
}
```



PLANTREE E-COMMERCE APPLICATION

```
};
final AlertDialog.Builder builder = new
AlertDialog.Builder(CartActivity.this);
builder.setTitle("Cart option:");
builder.setItems(option, new
DialogInterface.OnClickListener() {
@Override
public void onClick(DialogInterface
dialog, int which) {
if (which == 0) {
Intent intent = new
Intent(CartActivity.this, ProductDetailsActivity.class);
intent.putExtra("pid",
model.getPid());
startActivity(intent);
} else if (which == 1) {
cartRef.child(model.getPid()).removeValue()
.addOnCompleteListener(new OnCompleteListener<Void>() {
@Override
public void
onComplete(@NonNull Task<Void> task) {
if
(task.isSuccessful()) {
Toast.makeText(CartActivity.this, "Item removed",
Toast.LENGTH_SHORT).show();
} else {
Toast.makeText(CartActivity.this, "Error occur",
Toast.LENGTH_SHORT).show();
}
}
});
}
});
builder.setCancelable(true);
builder.show();
});
if (totalPrice == 0) {
nextButton.setVisibility(View.INVISIBLE);
} else {
nextButton.setVisibility(View.VISIBLE);
}
}
@NonNull
@Override
```



PLANTREE E-COMMERCE APPLICATION

```
        public CartViewHolder onCreateViewHolder(@NonNull
ViewGroup parent, int viewType) {
            return new
CartViewHolder(LayoutInflater.from(getApplicationContext()).inflate(R.l
ayout.cart_list_item, parent, false));
        }
    };
    mRecyclerView.setAdapter(adapter);
    adapter.startListening();
}

private void checkOrdersState() {
    if (currentUser == null) {
        Log.d("TAG", "checkOrdersState: ");
    } else {
        mRecyclerView.setVisibility(View.VISIBLE);
        DatabaseReference ordersRef =
FirebaseDatabase.getInstance().getReference().child("Orders").child(cur
rentUser.getPhone());
        ordersRef.addValueEventListener(new ValueEventListener() {
            @Override
            public void onDataChange(@NonNull DataSnapshot
snapshot) {
                if (snapshot.exists()) {
                    String state =
snapshot.child("state").getValue().toString();
                    String name =
snapshot.child("name").getValue().toString();
                    if (state.equals("shipped")) {
                        totalAmountTextView.setText("Shipping state
:shipped");

                        msg1.setVisibility(View.VISIBLE);
                        msg1.setText("Dear " + name + " your order
has been shipped sucesfully");
                        nextButton.setVisibility(View.INVISIBLE);
                    } else if (state.equals("not shipped")) {
                        totalAmountTextView.setText("Shipping state
: not shipped");

                        msg1.setVisibility(View.VISIBLE);
                        nextButton.setVisibility(View.INVISIBLE);
                    }
                }
            }

            @Override
            public void onCancelled(@NonNull DatabaseError error) {

            }
        });
    }
}
```



Add new product Activity : -

```
package com.weare.plantree.Admin;

import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import androidx.appcompat.app.AppCompatActivity;

import android.app.ProgressDialog;
import android.content.Intent;

import android.net.Uri;
import android.os.Bundle;
import android.text.TextUtils;
import android.view.View;
import android.widget.Button;
import android.widget.EditText;
import android.widget.ImageView;
import android.widget.Toast;

import com.google.android.gms.tasks.Continuation;
import com.google.android.gms.tasks.OnCompleteListener;
import com.google.android.gms.tasks.OnFailureListener;
import com.google.android.gms.tasks.OnSuccessListener;
import com.google.android.gms.tasks.Task;
import com.google.firebase.database.DatabaseReference;
import com.google.firebase.database.FirebaseDatabase;
import com.google.firebase.storage.FirebaseStorage;
import com.google.firebase.storage.StorageReference;
import com.google.firebase.storage.UploadTask;
import com.weare.plantree.R;

import java.text.SimpleDateFormat;
import java.util.Calendar;
import java.util.HashMap;
import java.util.Objects;

public class AdminAddNewProductActivity extends AppCompatActivity {

    private String categoryName, Description, Price, Pname,
    saveCurrentDate, saveCurrentTime;
    private Button addNewProductBtn;
    private ImageView selectImageView;
    private EditText getName, getDescription, getPrice;
    public static final int GALLERY_PICK_REQUEST_CODE = 1;
    private Uri imageUri;
    private String productRandomKey, downloadImageUrl;
    private StorageReference productImageRef;
    private DatabaseReference productRef;
    private ProgressDialog loadingbar;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
```




PLANTREE E-COMMERCE APPLICATION

```
super.onCreate(savedInstanceState);
setContentView(R.layout.activity_admin_add_new_product);

Intent intent = getIntent();
categoryName = intent.getStringExtra("category");
Toast.makeText(this, "adding " +
intent.getStringExtra("category"), Toast.LENGTH_SHORT).show();

//
productImageRef =
FirebaseStorage.getInstance().getReference().child("Product Images");

productRef=FirebaseDatabase.getInstance().getReference().child("Products");

//
loadingbar = new ProgressDialog(this);
//find
addNewProductBtn = findViewById(R.id.btn_add_new_product);
selectImageView = findViewById(R.id.select_product_image);
getName = findViewById(R.id.product_name);
getDescription = findViewById(R.id.product_description);
getPrice = findViewById(R.id.product_price);

selectImageView.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {
        openGallery();
    }
});

addNewProductBtn.setOnClickListener(new View.OnClickListener()
{
    @Override
    public void onClick(View v) {
        validateProductData();
    }
});

private void validateProductData() {
    Description = getDescription.getText().toString();
    Pname = getName.getText().toString();
    Price = getPrice.getText().toString();

    if (imageUri == null) {
        Toast.makeText(this, "Product image is required",
Toast.LENGTH_SHORT).show();
    } else if (TextUtils.isEmpty(Pname)) {
        Toast.makeText(this, "Product name required",
Toast.LENGTH_SHORT).show();
    } else if (TextUtils.isEmpty(Description)) {
```



PLANTREE E-COMMERCE APPLICATION

```
        Toast.makeText(this, "Product description required",
Toast.LENGTH_SHORT).show();
    } else if (TextUtils.isEmpty(Price)) {
        Toast.makeText(this, "Product price required",
Toast.LENGTH_SHORT).show();
    } else {
        storeProductInformation();
        loadingbar.setTitle("Adding product...");

loadingbar.setMessage(getString(R.string.please_wait_text));
        loadingbar.setCanceledOnTouchOutside(false);
        loadingbar.show();
    }
}

private void storeProductInformation() {
    Calendar calendar = Calendar.getInstance();
    SimpleDateFormat currentDate = new SimpleDateFormat("MMM
dd,yyyy");
    saveCurrentDate = currentDate.format(calendar.getTime());
    SimpleDateFormat currentTime = new SimpleDateFormat("HH:mm:ss
a");
    saveCurrentTime = currentTime.format(calendar.getTime());
    productRandomKey = saveCurrentDate+saveCurrentTime;
    productRandomKey=productRandomKey.replace(".", "
").replace("Rs", " ").replace("#", " ").replace("@", " ");

    //
    final StorageReference filePath =
productImageRef.child(imageUri.getLastPathSegment() + productRandomKey
+ ".jpg");
    final UploadTask uploadTask = filePath.putFile(imageUri);
    uploadTask.addOnFailureListener(new OnFailureListener() {
        @Override
        public void onFailure(@NonNull Exception e) {
            loadingbar.dismiss();
            Toast.makeText(AdminAddNewProductActivity.this, "Error"
+ e.getMessage(), Toast.LENGTH_SHORT).show();
        }
    })

        .addOnSuccessListener(new
OnSuccessListener<UploadTask.TaskSnapshot>() {
            @Override
            public void onSuccess(UploadTask.TaskSnapshot
taskSnapshot) {

                Task<Uri> uriTask =
uploadTask.continueWithTask(new Continuation<UploadTask.TaskSnapshot,
Task<Uri>>() {
                    @Override
                    public Task<Uri> then(@NonNull
Task<UploadTask.TaskSnapshot> task) throws Exception {
                        if (!task.isSuccessful()) {
                            throw
```



PLANTREE E-COMMERCE APPLICATION

```

Objects.requireNonNull(task.getException());
    }
    downloadImageUrl =
filePath.getDownloadUrl().toString();
    return filePath.getDownloadUrl();
    }
    }).addOnCompleteListener(new
OnCompleteListener<Uri>() {
    @Override
    public void onComplete(@NonNull Task<Uri>
task) {

        if (task.isSuccessful())
        {

downloadImageUrl=task.getResult().toString();

Toast.makeText(AdminAddNewProductActivity.this, "got product image",
Toast.LENGTH_SHORT).show();

                saveProductInfoTodatabase();
            }
        }
    });
}

}

private void saveProductInfoTodatabase() {
    HashMap<String, Object> productHashMap=new HashMap<>();
    productHashMap.put("pid",productRandomKey);
    productHashMap.put("date",saveCurrentDate);
    productHashMap.put("time",saveCurrentTime);
    productHashMap.put("description",Description);
    productHashMap.put("image",downloadImageUrl);
    productHashMap.put("category",categoryName);
    productHashMap.put("price",Price);
    productHashMap.put("pname",Pname);

productRef.child(productRandomKey).updateChildren(productHashMap).addOn
CompleteListener(new OnCompleteListener<Void>() {
    @Override
    public void onComplete(@NonNull Task<Void> task) {

        if (task.isSuccessful()){
            loadingbar.dismiss();
            Toast.makeText(AdminAddNewProductActivity.this,
"Product added succesfully", Toast.LENGTH_SHORT).show();
            startActivity(new
Intent(AdminAddNewProductActivity.this, AdminCategoryActivity.class));
            finish();
        }
    }
});
}
}

```



PLANTREE E-COMMERCE APPLICATION

```
        else {
            loadingbar.dismiss();
            Toast.makeText(AdminAddNewProductActivity.this,
"Product cannot added error:"+task.getException(),
Toast.LENGTH_SHORT).show();
        }
    }
});

}

private void openGallery() {
    Intent intent = new Intent();
    intent.setAction(Intent.ACTION_GET_CONTENT);
    intent.setType("image/*");
    startActivityForResult(intent, GALLERY_PICK_REQUEST_CODE);
}

@Override
protected void onActivityResult(int requestCode, int resultCode,
@Nullable Intent data) {
    super.onActivityResult(requestCode, resultCode, data);
    if (requestCode == GALLERY_PICK_REQUEST_CODE && resultCode ==
RESULT_OK) {
        if (data != null) {
            imageUri = data.getData();
            selectImageView.setImageURI(imageUri);
        }
    }
}
}
```

Confirm Final Order Activity : -

```
package com.weare.plantree.Buyers;

import androidx.annotation.NonNull;
import androidx.appcompat.app.AppCompatActivity;

import android.content.Intent;
import android.os.Bundle;
import android.text.TextUtils;
import android.view.View;
import android.widget.Button;
import android.widget.EditText;
import android.widget.Toast;

import com.google.android.gms.tasks.OnCompleteListener;
import com.google.android.gms.tasks.Task;
import com.google.firebase.database.DatabaseReference;
import com.google.firebase.database.FirebaseDatabase;
import com.weare.plantree.HomeActivity;
import com.weare.plantree.Model.Users;
```



PLANTREE E-COMMERCE APPLICATION

```
import com.weare.plantree.Prevalent.Prevalent;
import com.weare.plantree.R;

import java.text.SimpleDateFormat;
import java.util.Calendar;
import java.util.HashMap;

import io.paperdb.Paper;

public class ConfirmFinalOrderActivity extends AppCompatActivity {
    private EditText
phoneEditText,nameEditText,adressEditText,cityEditText,email;
    private Button confirmButton;
    int totalprice;
    int i = 0;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_confirm_final_order);
        phoneEditText=findViewById(R.id.phone_edit_confirm);
        nameEditText=findViewById(R.id.name_edit_confirm);
        adressEditText=findViewById(R.id.adress_edit_confirm);
        cityEditText=findViewById(R.id.city_edit_confirm);
        confirmButton=findViewById(R.id.confirm_button_confirm);
        email = findViewById(R.id.email);
        Paper.init(this);

        totalprice=getIntent().getIntExtra("total",0);
        // Toast.makeText(this, "total: $" +String.valueOf(totalprice),
        Toast.LENGTH_SHORT).show();

        confirmButton.setOnClickListener(new View.OnClickListener()
        {
            @Override
            public void onClick(View v)
            {
                if (checkEditText())
                {
                    confirmOrder();
                }
            }
        });
    }

    private void confirmOrder()
    {
        Intent intent = new
Intent(ConfirmFinalOrderActivity.this,PaymentActivity.class);
        intent.putExtra("total",totalprice);
        intent.putExtra("name",nameEditText.getText().toString());
        intent.putExtra("phone",nameEditText.getText().toString());
        intent.putExtra("address",nameEditText.getText().toString());
        intent.putExtra("city",cityEditText.getText().toString());
    }
}
```



PLANTREE E-COMMERCE APPLICATION

```
        intent.putExtra("email",email.getText().toString());
        startActivity(intent);

    }

    private boolean checkEditText() {
        if (TextUtils.isEmpty(nameEditText.getText().toString())){
            Toast.makeText(this, "enter name",
                Toast.LENGTH_SHORT).show();
            return false;
        }
        else if
            (TextUtils.isEmpty(phoneEditText.getText().toString()))
        {
            Toast.makeText(this, "enter phone",
                Toast.LENGTH_SHORT).show();
            return false;
        }
        else if
            (TextUtils.isEmpty(adressEditText.getText().toString()))
        {
            Toast.makeText(this, "enter adress",
                Toast.LENGTH_SHORT).show();
            return false;
        }
        else if
            (TextUtils.isEmpty(cityEditText.getText().toString()))
        {
            Toast.makeText(this, "enter city",
                Toast.LENGTH_SHORT).show();
            return false;
        }
        else if (TextUtils.isEmpty(email.getText().toString()))
        {
            Toast.makeText(this, "enter email",
                Toast.LENGTH_SHORT).show();
            return false;
        }
        else {
            return true;
        }
    }
}
```



PLANTREE E-COMMERCE APPLICATION

Payment Activity : -

```
package com.weare.plantree.Buyers;

import android.app.Activity;
import android.content.Intent;
import android.os.Bundle;
import android.widget.Button;
import android.widget.Toast;

import androidx.annotation.NonNull;
import androidx.appcompat.app.AppCompatActivity;

import com.google.android.gms.tasks.OnCompleteListener;
import com.google.android.gms.tasks.Task;
import com.google.firebase.database.DatabaseReference;
import com.google.firebase.database.FirebaseDatabase;
import com.razorpay.Checkout;
import com.razorpay.PaymentResultListener;
import com.weare.plantree.HomeActivity;
import com.weare.plantree.Model.Users;
import com.weare.plantree.Prevalent.Prevalent;
import com.weare.plantree.R;

import org.json.JSONObject;

import java.text.SimpleDateFormat;
import java.util.Calendar;
import java.util.HashMap;

import io.paperdb.Paper;

public class PaymentActivity extends AppCompatActivity implements
PaymentResultListener {

    Button onlinePayment, cashOnDelhivery;
    Intent intent;
    String name, phone, address, city, email;
    int price;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_payment);

        onlinePayment = findViewById(R.id.onlineBtn);
        cashOnDelhivery = findViewById(R.id.cashondelhiveryBtn);
        intent = getIntent();
        Paper.init(this);
        Checkout.preload(this);

        name = intent.getStringExtra("name");
        phone = intent.getStringExtra("phone");
        address = intent.getStringExtra("address");
        city = intent.getStringExtra("city");
```



PLANTREE E-COMMERCE APPLICATION

```
email = intent.getStringExtra("email");
price = intent.getIntExtra("total",0);

cashOnDelhivery.setOnClickListener(view-> {
    String saveCurrentTime, saveCurrentDate;
    Calendar calendarForDate = Calendar.getInstance();
    SimpleDateFormat currentDate = new
SimpleDateFormat("MMM dd,yyyy");
    saveCurrentDate =
currentDate.format(calendarForDate.getTime());

    Calendar calendarForTime = Calendar.getInstance();
    SimpleDateFormat currentTime = new
SimpleDateFormat("HH:mm:ss a");
    saveCurrentTime =
currentTime.format(calendarForTime.getTime());
    final Users users =
Paper.book().read(Prevalent.currentOnlineUser);

    final DatabaseReference reference =
FirebaseDatabase.getInstance().getReference()
        .child("Orders").child(users.getPhone());

    final HashMap<String, Object> orderFinalHashMap =
new HashMap<>();
    orderFinalHashMap.put("totalAmount",
String.valueOf(price));
    orderFinalHashMap.put("name", name);
    orderFinalHashMap.put("phone", phone);
    // productHashMap.put("description",description);
    orderFinalHashMap.put("adress", address);
    orderFinalHashMap.put("city", city);
    orderFinalHashMap.put("date", saveCurrentDate);
    orderFinalHashMap.put("email",email);
    orderFinalHashMap.put("time", saveCurrentTime);
    orderFinalHashMap.put("paymentType","Cash On
Delhivery");
    orderFinalHashMap.put("state", "not shipped");

    reference.updateChildren(orderFinalHashMap).addOnCompleteListener(new
OnCompleteListener<Void>() {
        @Override
        public void onComplete(@NonNull Task<Void>
task) {
            if (task.isSuccessful()) {
                DatabaseReference removeRef =
FirebaseDatabase.getInstance().getReference().child("Cart
List").child("User View")
                    .child(users.getPhone())
                    .child("Products");
                removeRef.removeValue()
                    .addOnCompleteListener(new
OnCompleteListener<Void>() {
                        @Override
                        public void
```




Page 119 of 150



PLANTREE E-COMMERCE APPLICATION

```
@Override
public void onPaymentSuccess(String s) {
    String saveCurrentTime, saveCurrentDate;
    Calendar calendarForDate = Calendar.getInstance();
    SimpleDateFormat currentDate = new SimpleDateFormat("MMM
dd,yyyy");
    saveCurrentDate =
currentDate.format(calendarForDate.getTime());

    Calendar calendarForTime = Calendar.getInstance();
    SimpleDateFormat currentTime = new SimpleDateFormat("HH:mm:ss
a");
    saveCurrentTime =
currentTime.format(calendarForTime.getTime());
    final Users users =
Paper.book().read(Prevalent.currentOnlineUser);

    final DatabaseReference reference =
FirebaseDatabase.getInstance().getReference()
        .child("Orders").child(users.getPhone());

    final HashMap<String, Object> orderFinalHashMap = new
HashMap<>();
    orderFinalHashMap.put("totalAmount", String.valueOf(price));
    orderFinalHashMap.put("name", name);
    orderFinalHashMap.put("phone", phone);
    // productHashMap.put("description",description);
    orderFinalHashMap.put("adress", address);
    orderFinalHashMap.put("city", city);
    orderFinalHashMap.put("email",email);
    orderFinalHashMap.put("date", saveCurrentDate);
    orderFinalHashMap.put("time", saveCurrentTime);
    orderFinalHashMap.put("paymentType","Online Payment");
    orderFinalHashMap.put("state", "not shipped");

    reference.updateChildren(orderFinalHashMap).addOnCompleteListener(new
OnCompleteListener<Void>() {
        @Override
        public void onComplete(@NonNull Task<Void> task) {
            if (task.isSuccessful()) {
                DatabaseReference removeRef =
FirebaseDatabase.getInstance().getReference().child("Cart
List").child("User View")
                    .child(users.getPhone())
                    .child("Products");
                removeRef.removeValue()
                    .addOnCompleteListener(new
OnCompleteListener<Void>() {
                        @Override
                        public void onComplete(@NonNull
Task<Void> task) {
                            if (task.isSuccessful()) {
```



PLANTREE E-COMMERCE APPLICATION

```
Toast.makeText(PaymentActivity.this, "Your final order has been
confiirmed", Toast.LENGTH_SHORT).show();

        Intent intent2 = new
Intent(PaymentActivity.this, HomeActivity.class);

intent2.setFlags(Intent.FLAG_ACTIVITY_CLEAR_TASK);
        startActivity(intent2);
        finish();

    }

    });

    } else {
        Toast.makeText(PaymentActivity.this, "Something
went wrong", Toast.LENGTH_SHORT).show();
    }

    });

    Toast.makeText(this, "Payment Successfull",
Toast.LENGTH_SHORT).show();
}

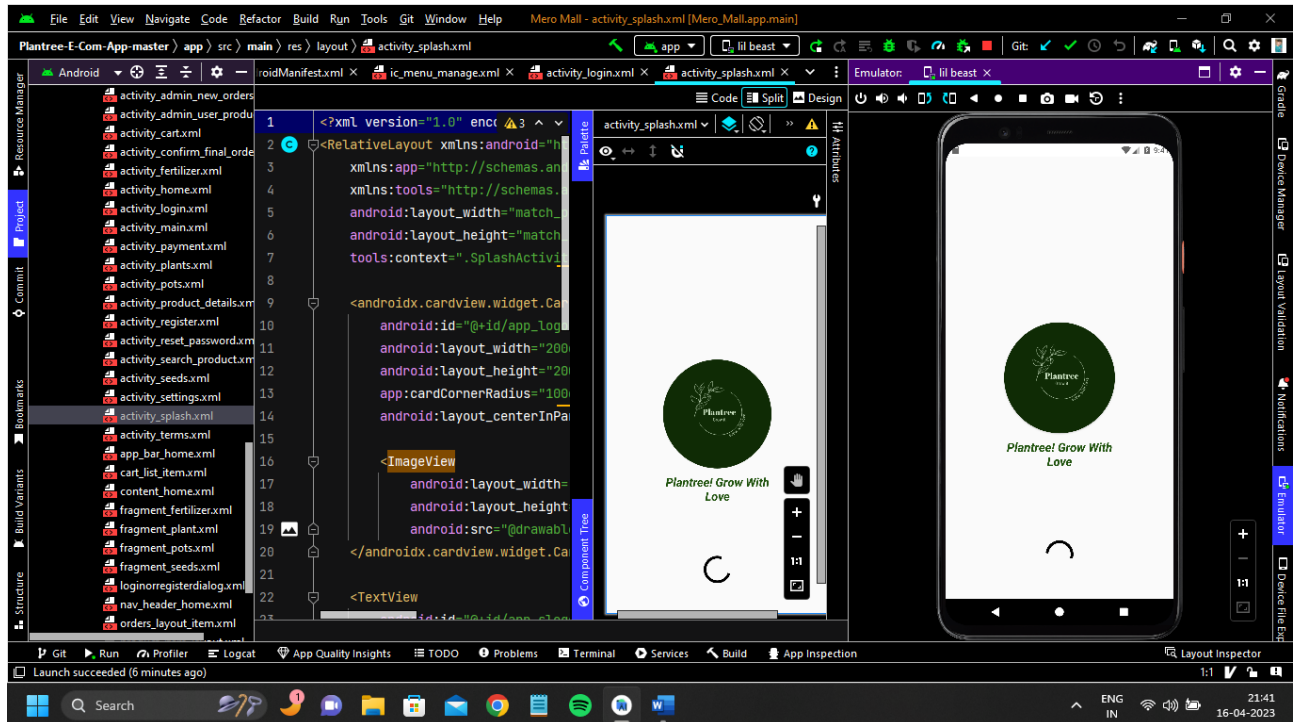
@Override
public void onPaymentError(int i, String s) {
    Toast.makeText(this, "Payment Failed",
Toast.LENGTH_SHORT).show();
}
}
```



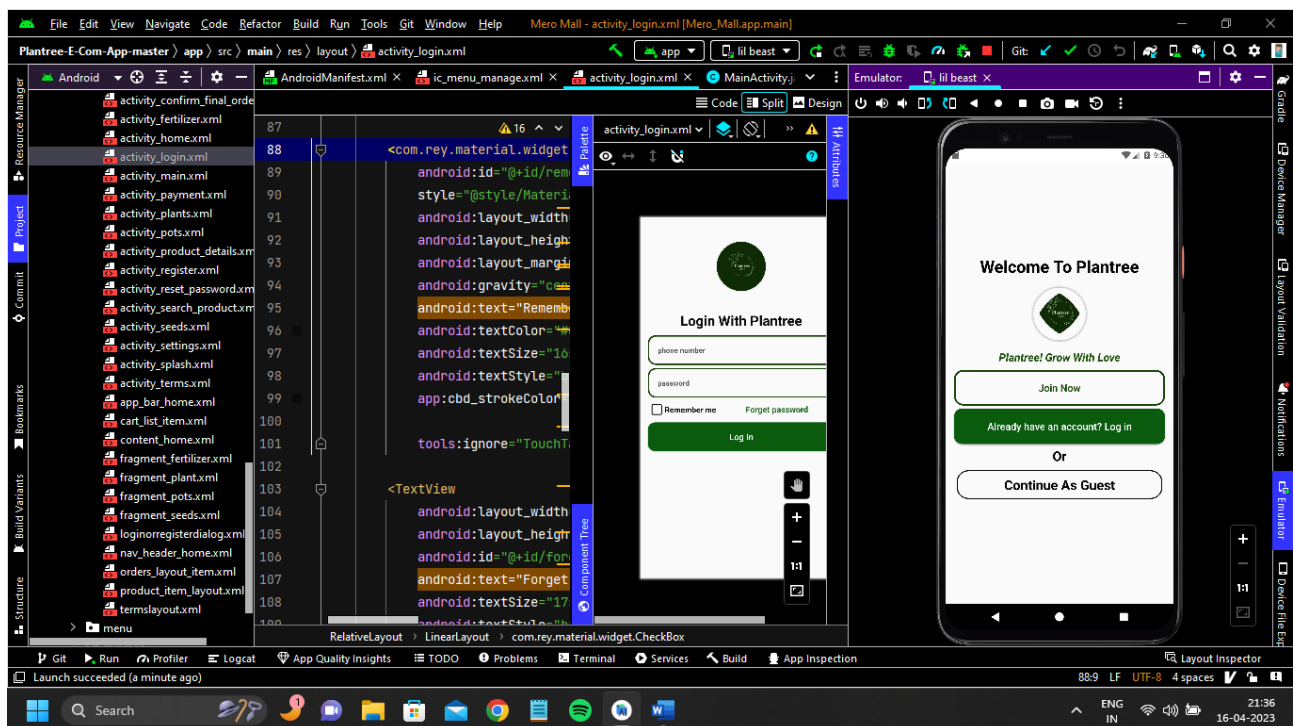
PLANTREE E-COMMERCE APPLICATION

6.3 Result Snapshot

SplashScreen :-



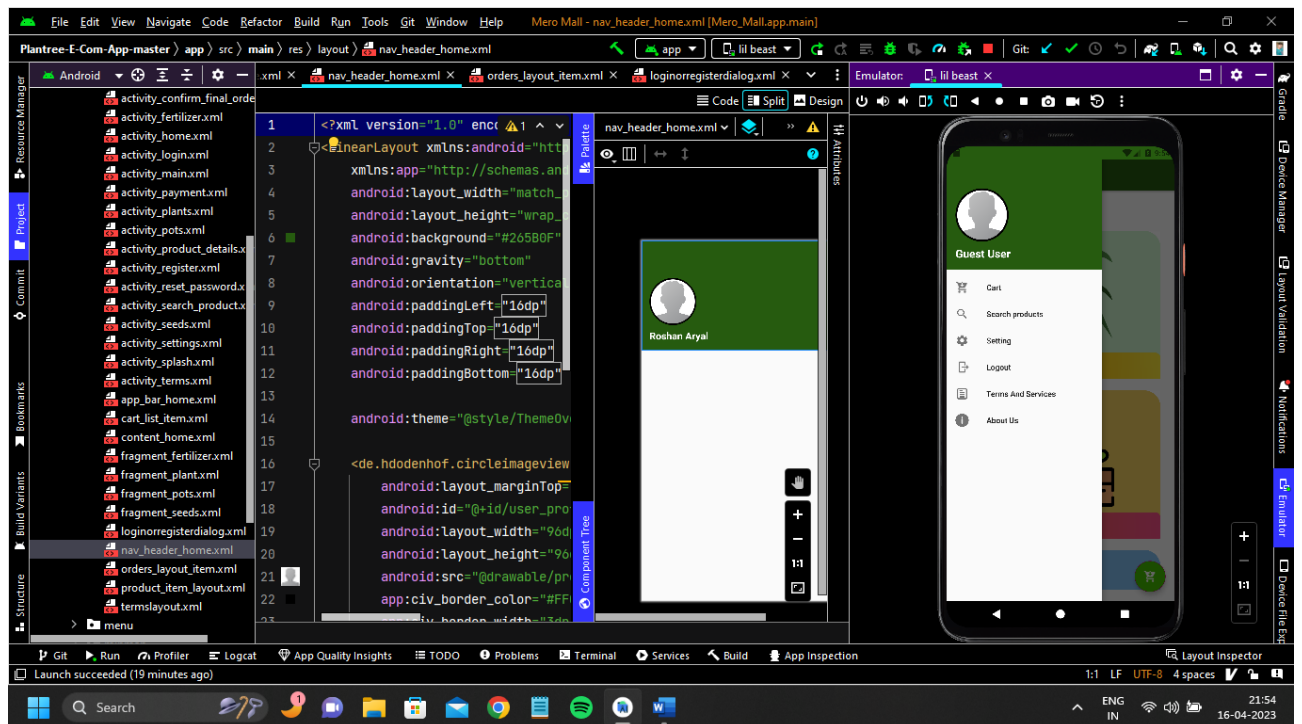
MainPage :-



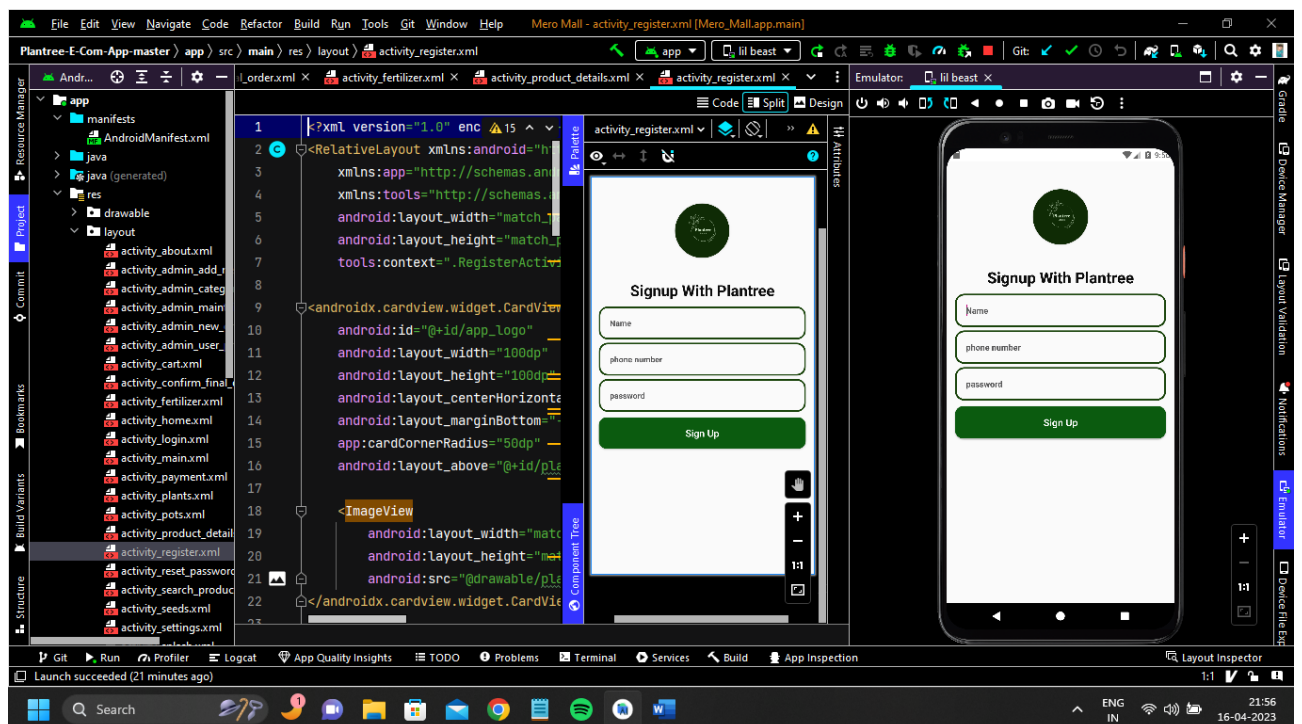


PLANTREE E-COMMERCE APPLICATION

GuestPortal :-



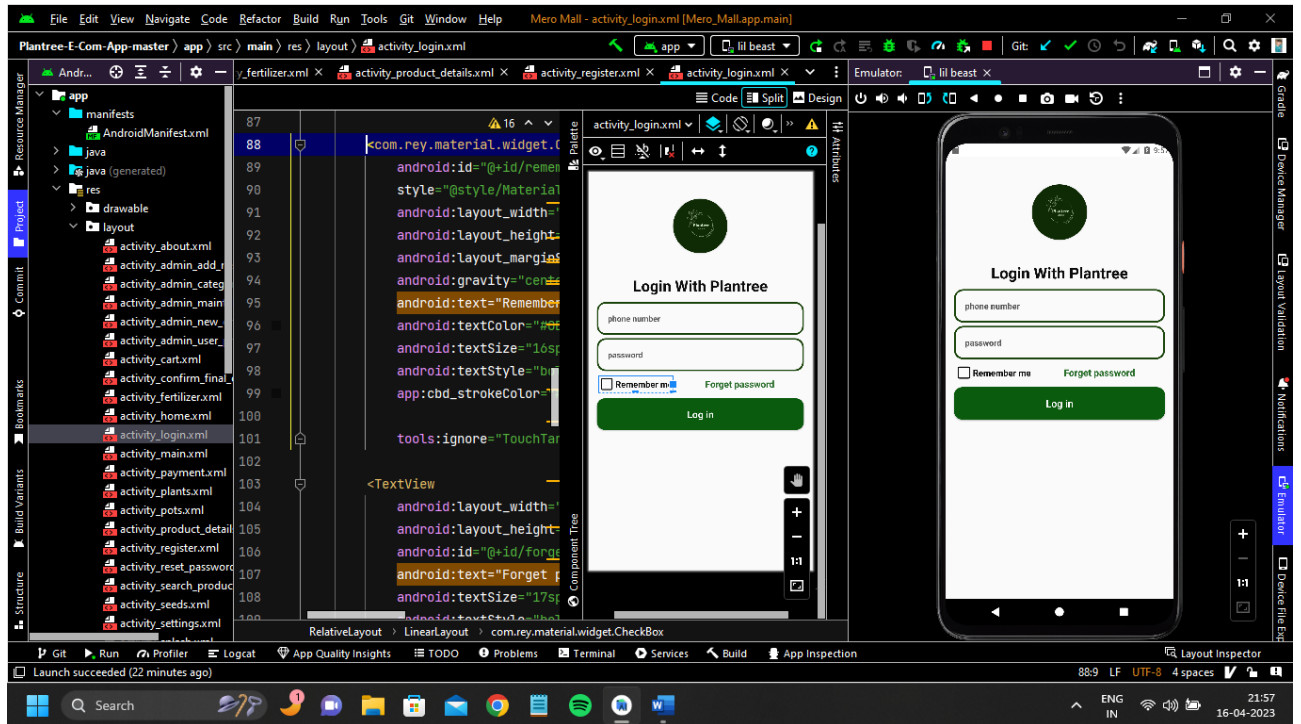
SignUpPage :-



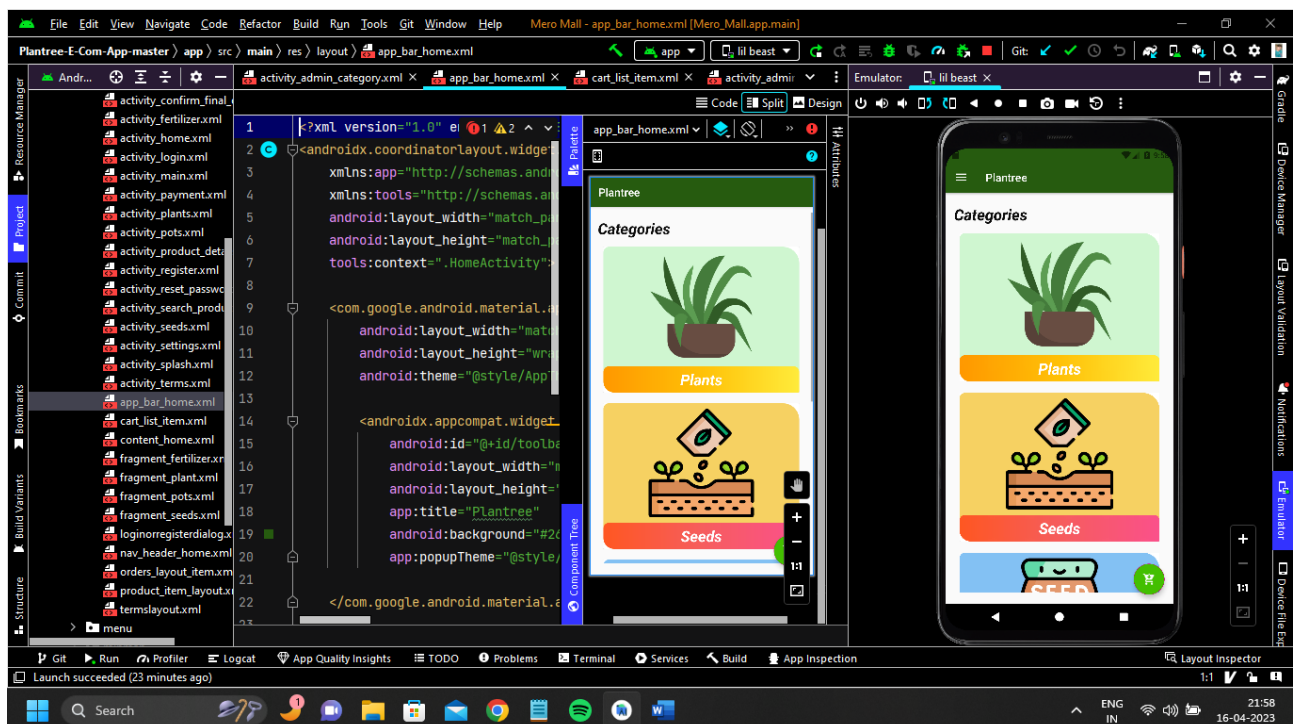


PLANTREE E-COMMERCE APPLICATION

LoginPage :-



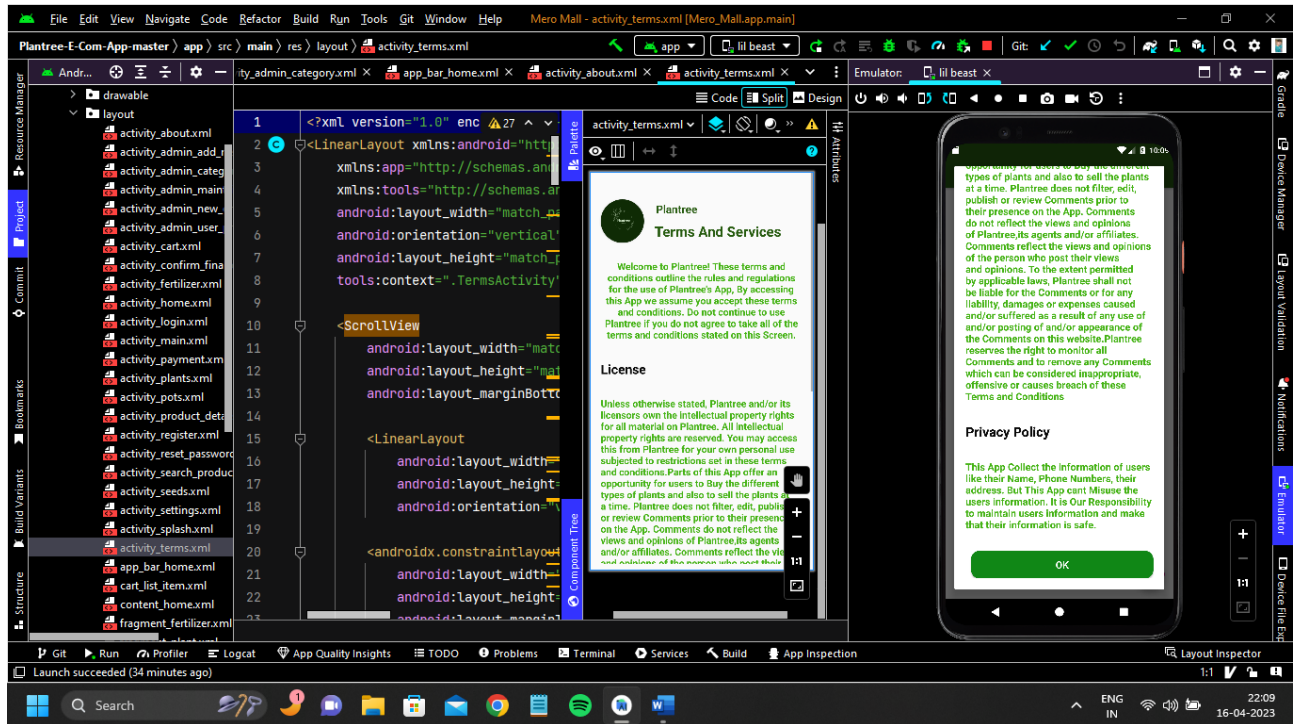
CategoryPage :-



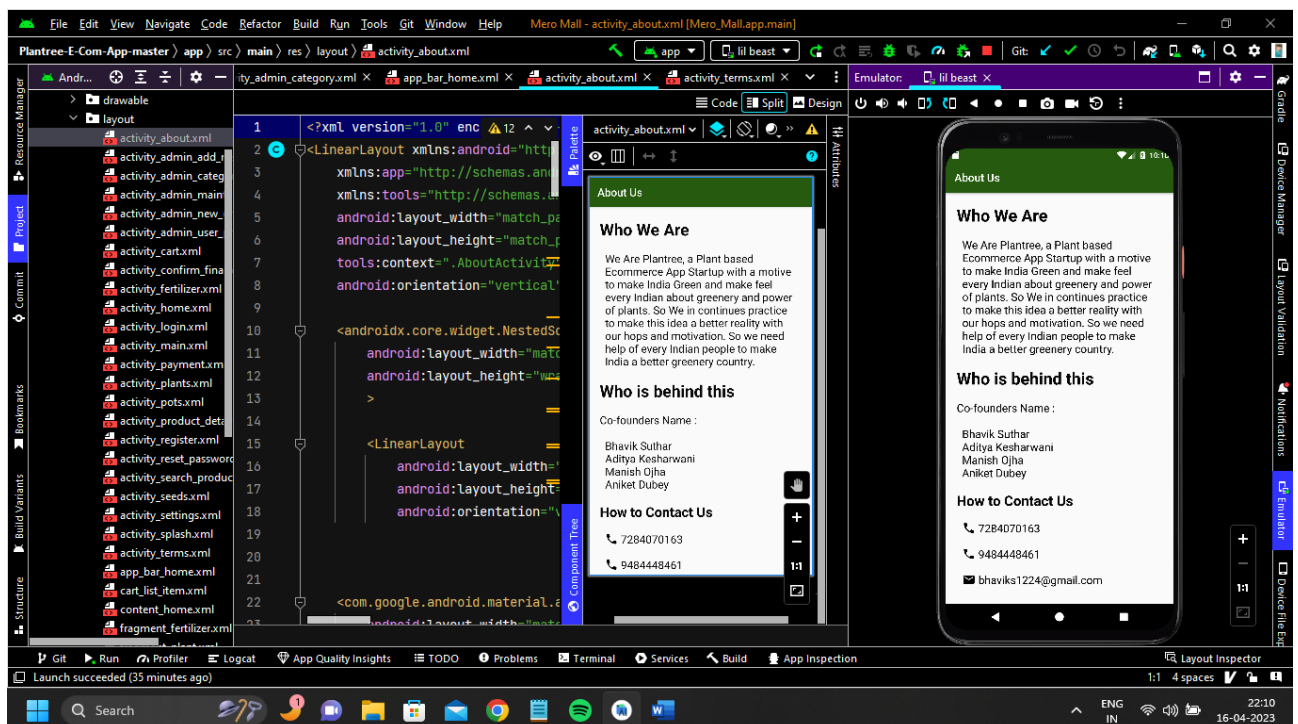


PLANTREE E-COMMERCE APPLICATION

Terms&Services Page :-



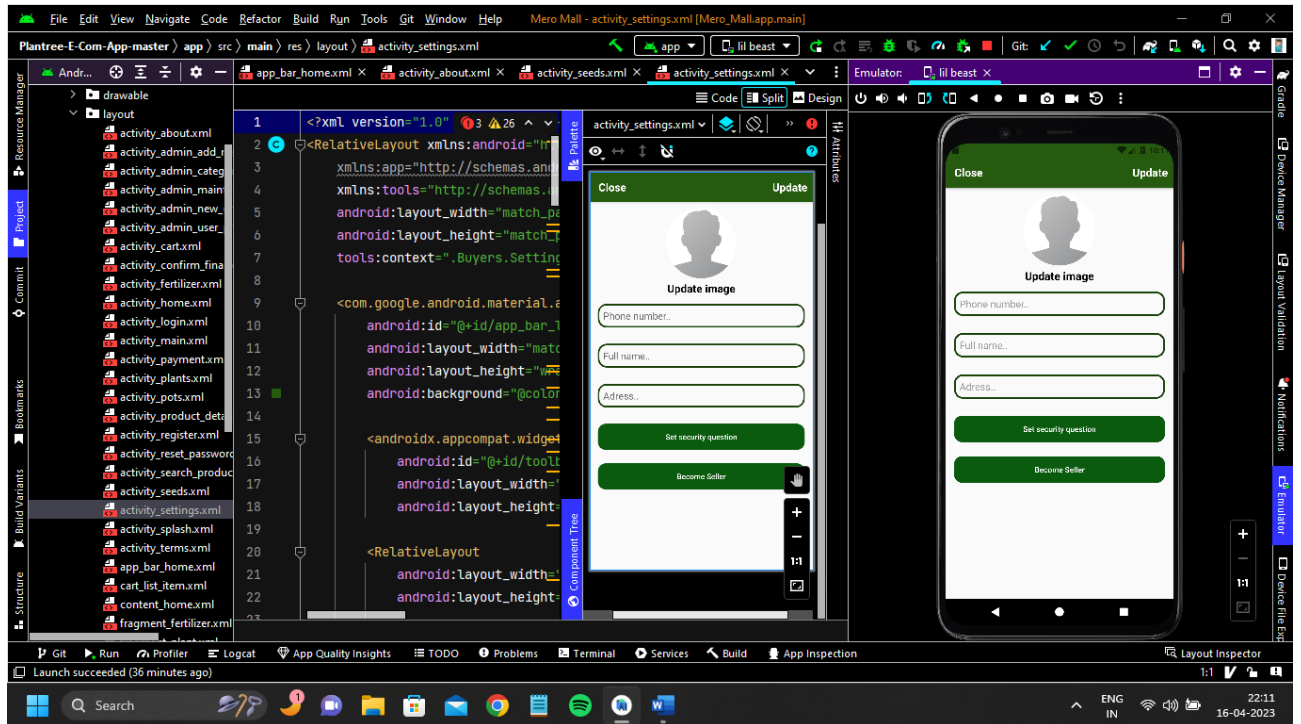
Aboutus Page :-



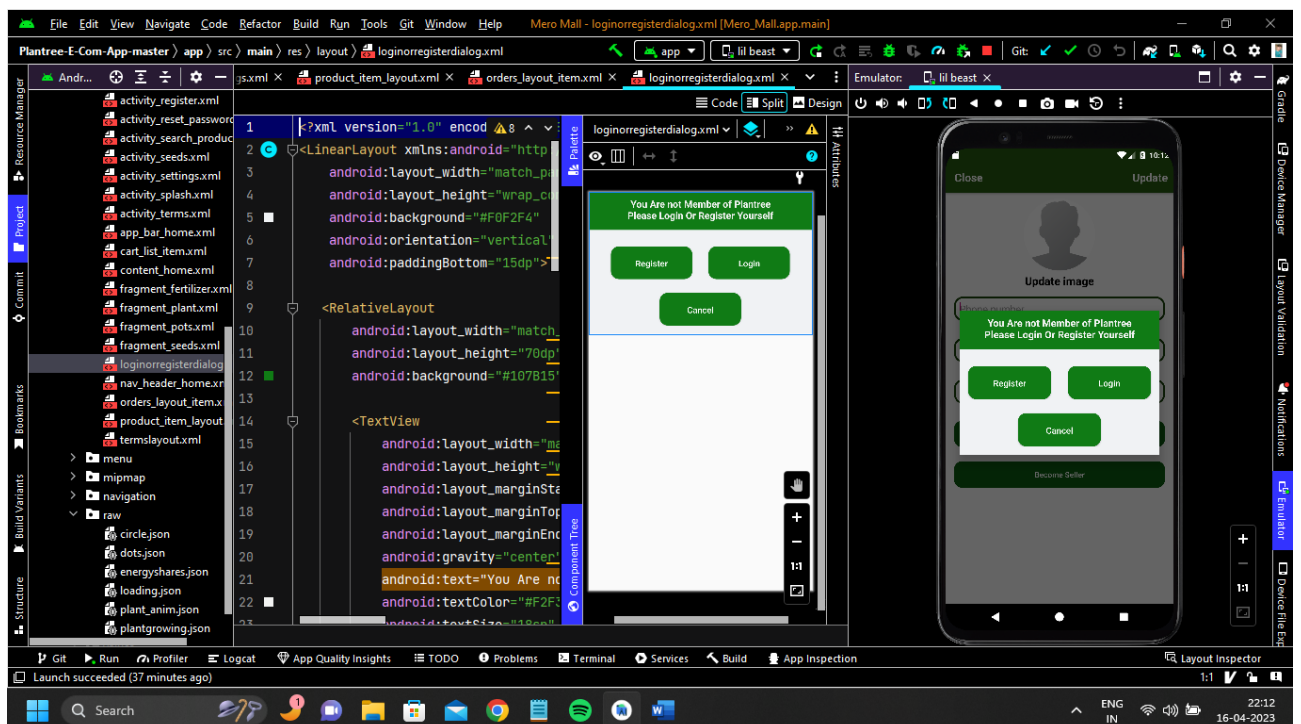


PLANTREE E-COMMERCE APPLICATION

Setting Page: -



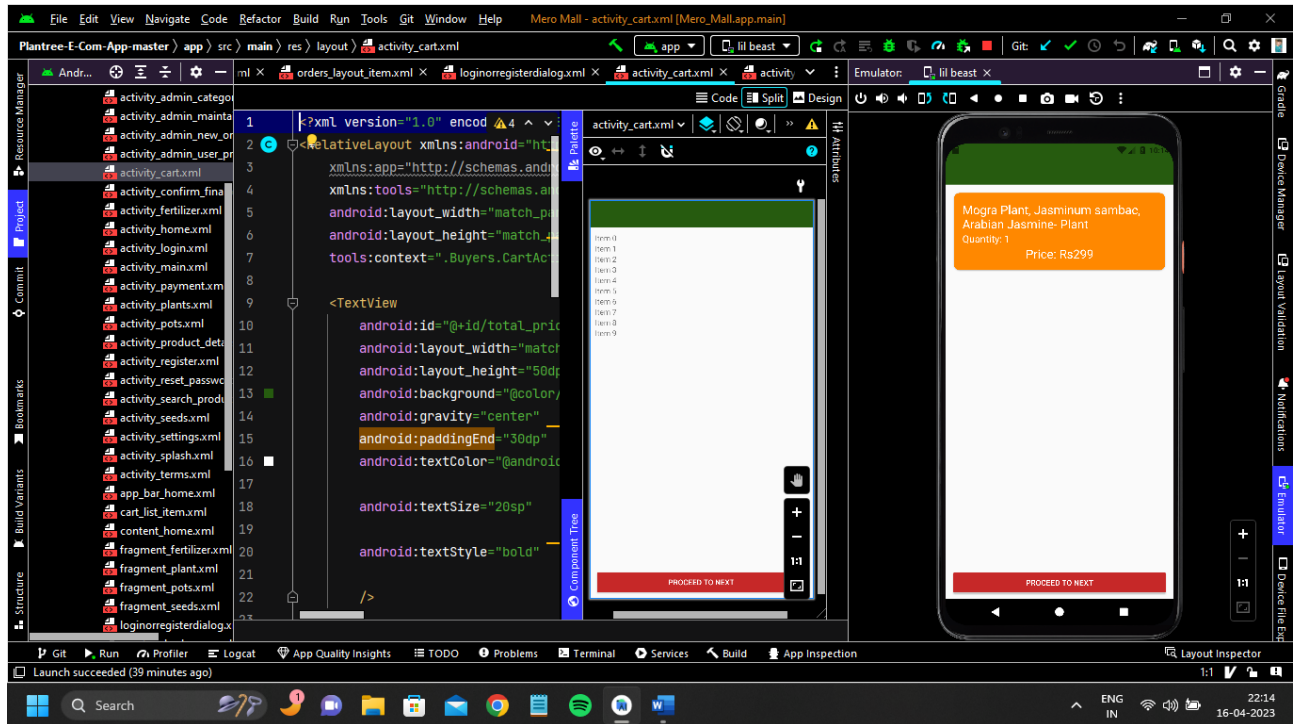
Become Seller Page : -



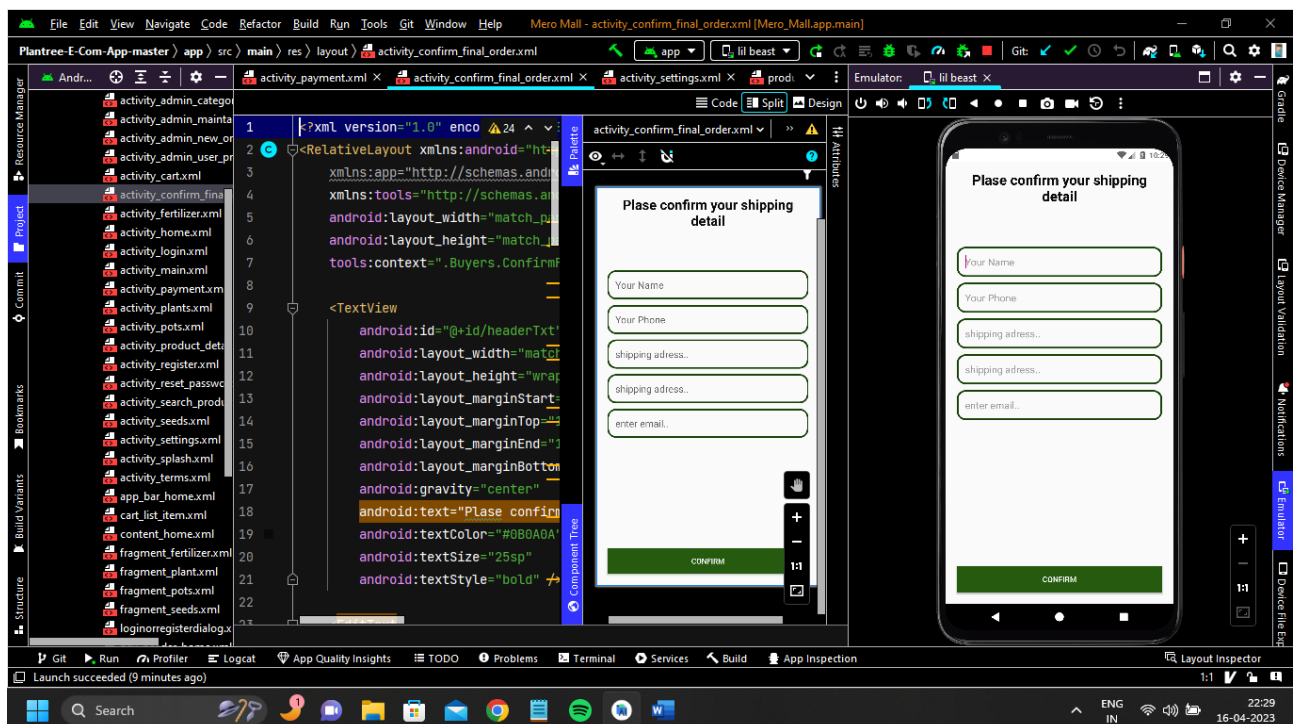


PLANTREE E-COMMERCE APPLICATION

Cart page: -



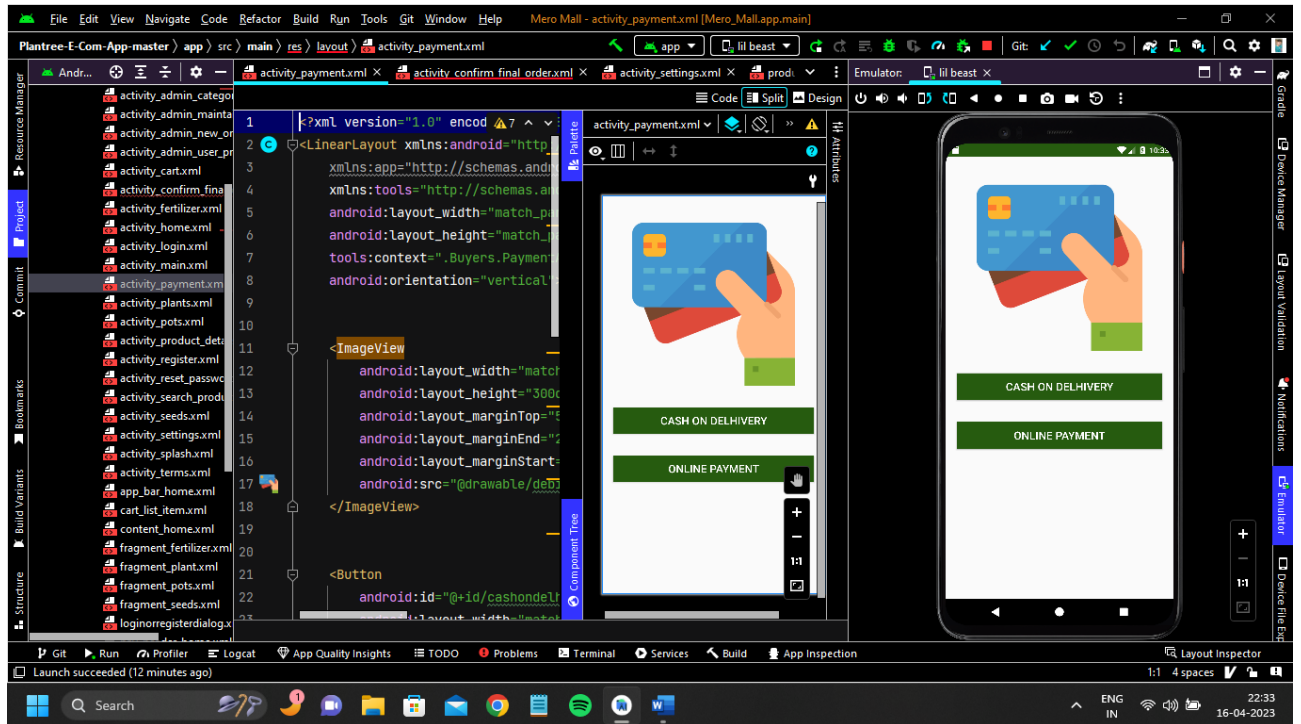
Confirmation Shipping page : -



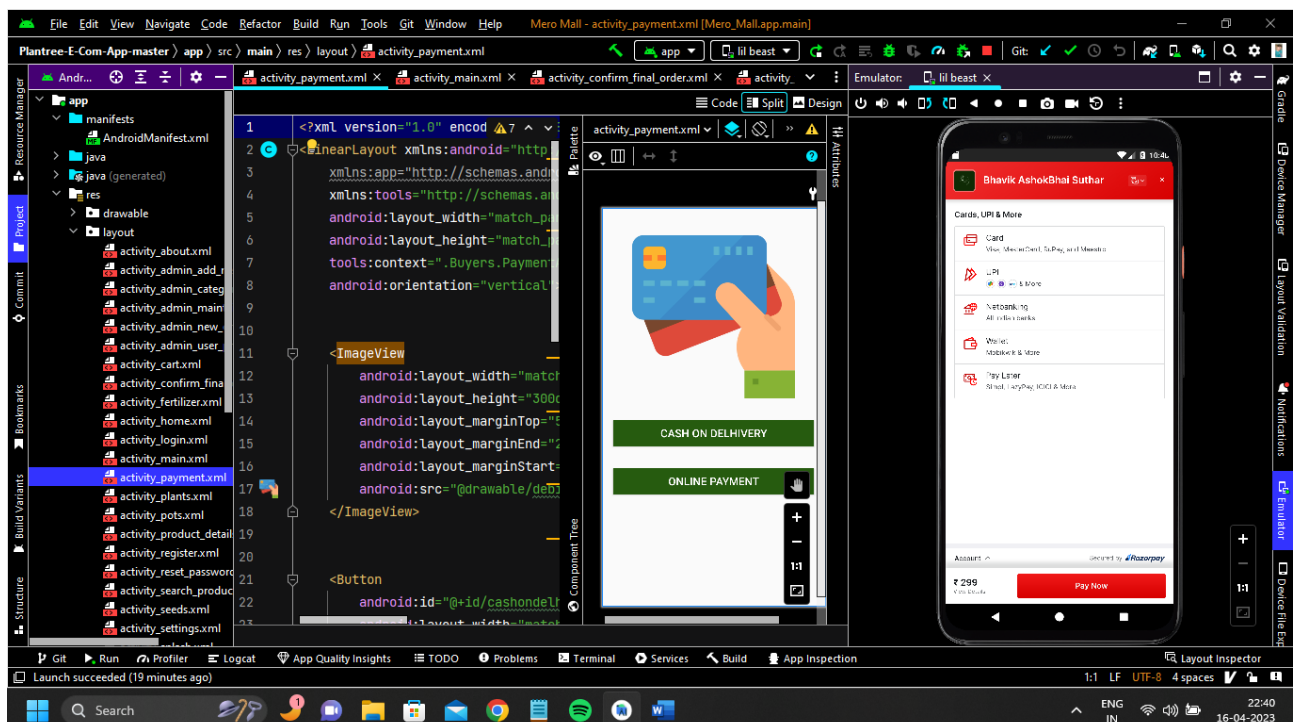


PLANTREE E-COMMERCE APPLICATION

Payment Option Page :-



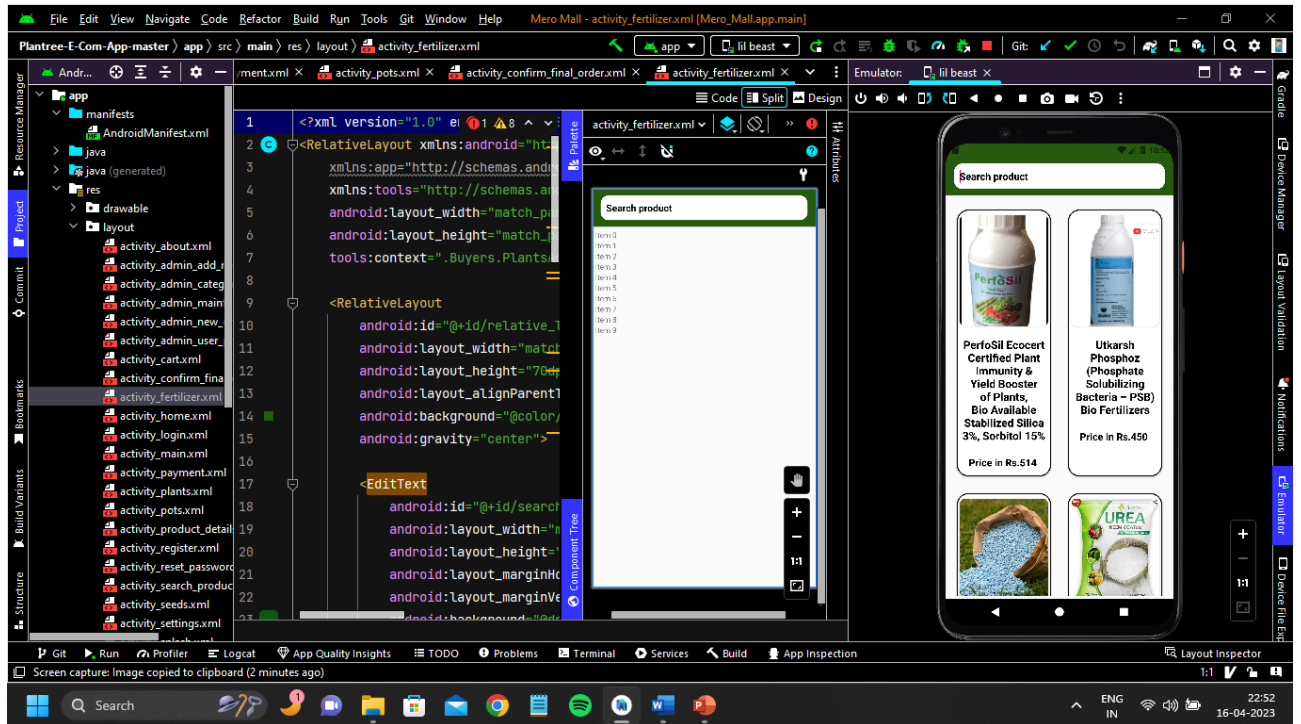
Online Payment Page :-



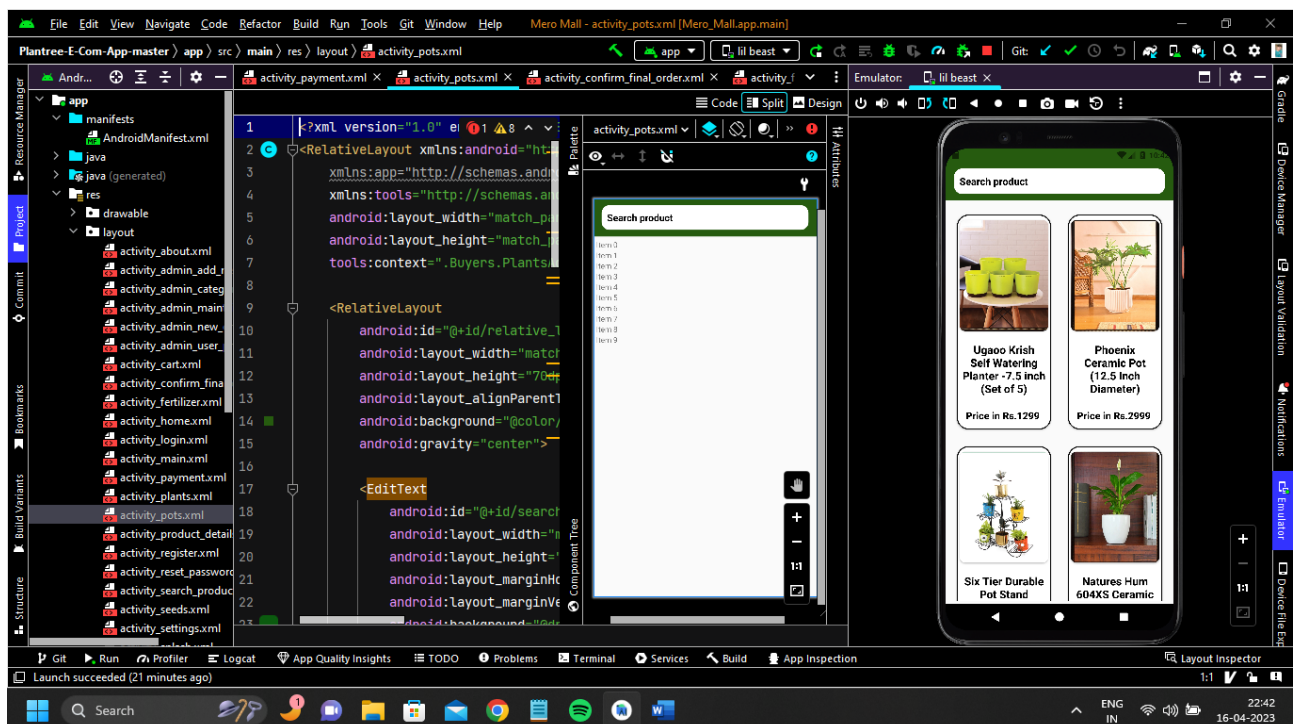


PLANTREE E-COMMERCE APPLICATION

Fertilizer Page :-



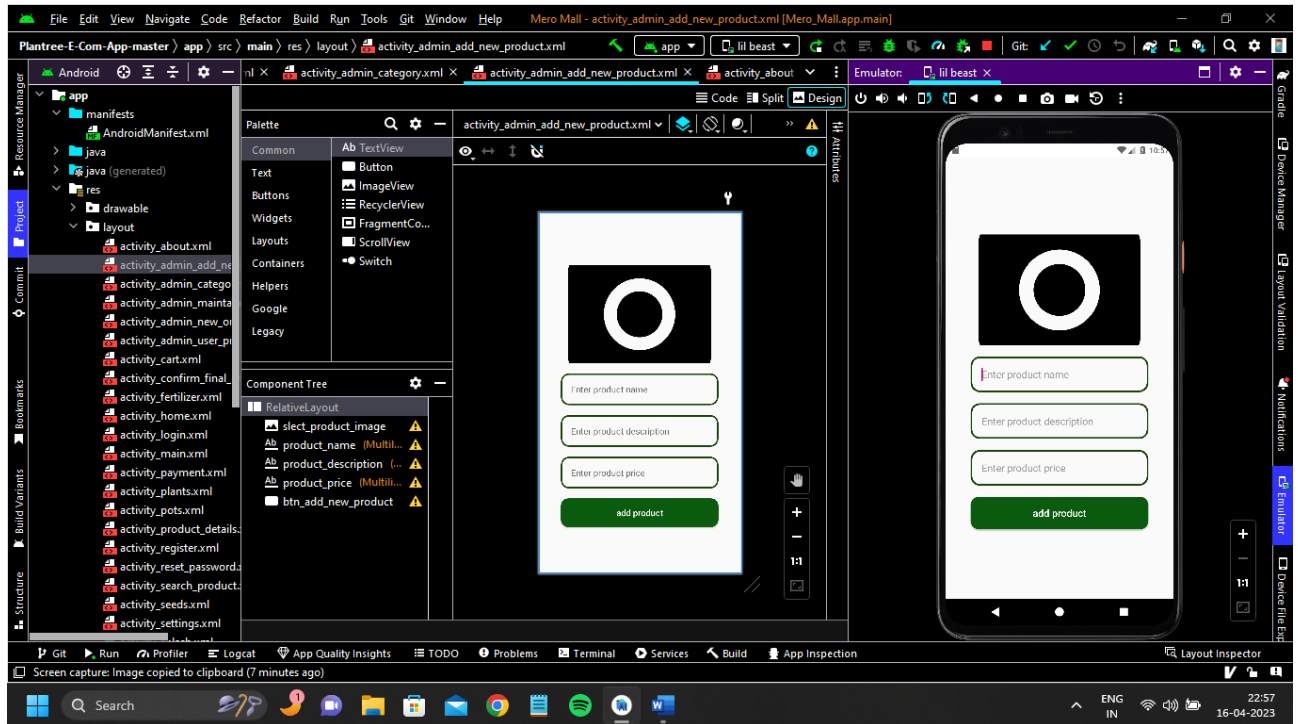
Pots Page :-



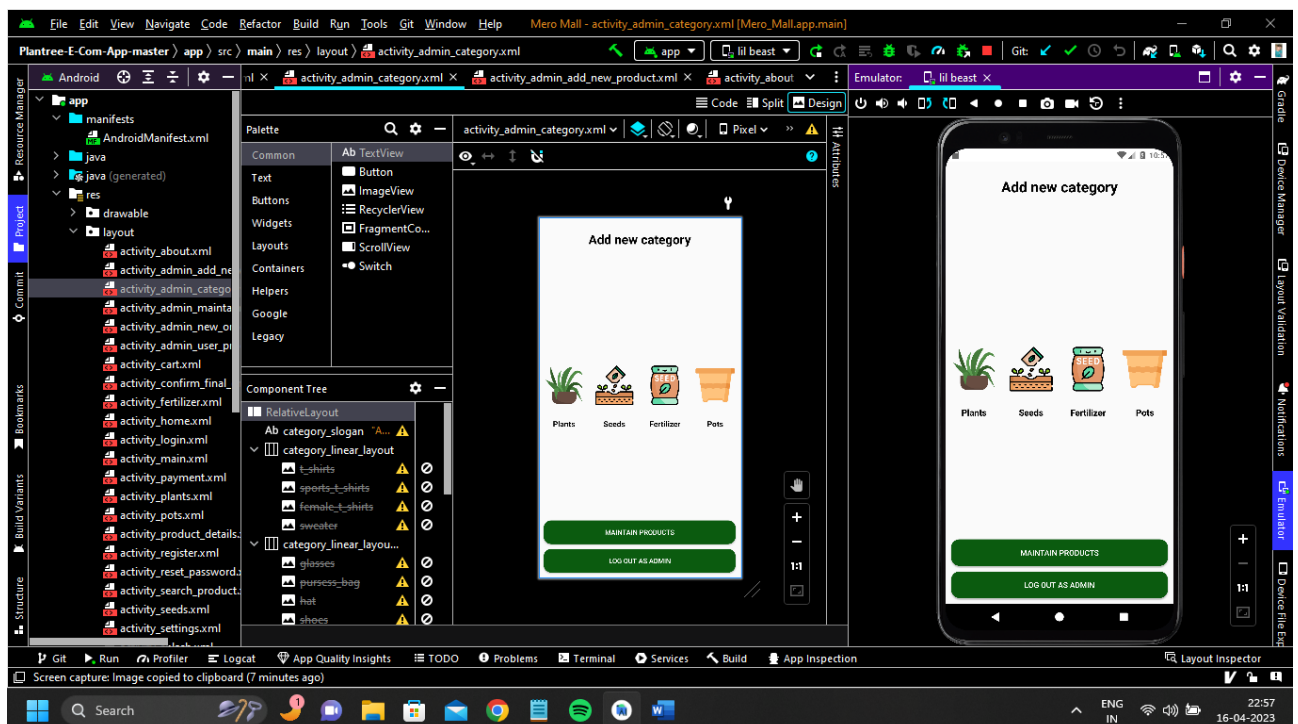


PLANTREE E-COMMERCE APPLICATION

Admin Add New Product : -

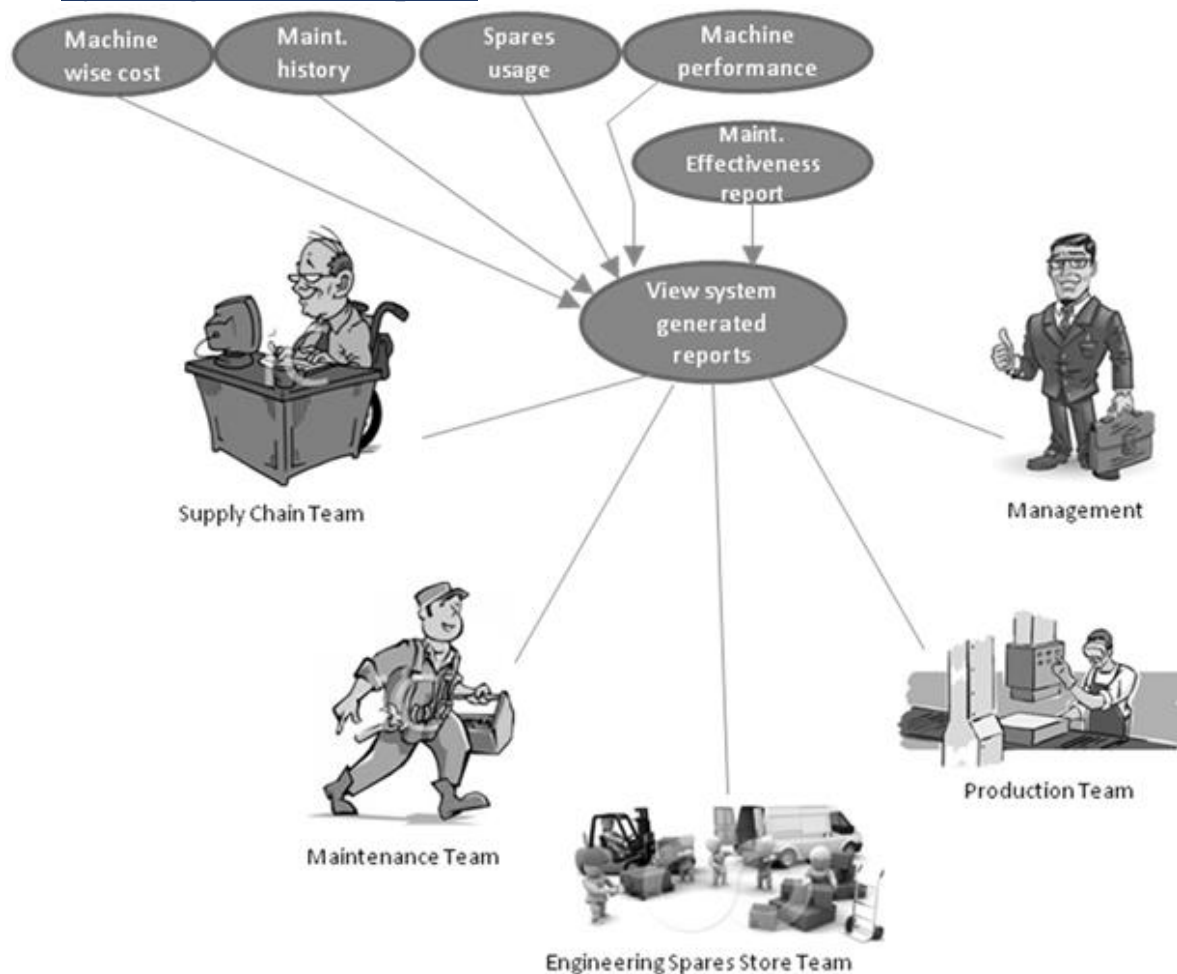


Admin Add new Category Page : -





6.4 System generated Reports



Sales Report: This report provides information on the sales volume, revenue generated, and the average order value for a specific time period.

Inventory Report: This report provides a snapshot of the current inventory levels of the ecommerce business, including the number of products in stock and the quantity of each product.

Customer Report: This report provides information on customer demographics, purchase history, and other metrics such as customer retention rates.

Marketing Report: This report provides information on the performance of marketing campaigns such as email marketing, social media advertising, and paid search advertising.

Shipping Report: This report provides information on the number of orders shipped, shipping costs, and the average shipping time.

Financial Report: This report provides an overview of the financial performance of the ecommerce business, including revenue, expenses, and profitability.

Please note that the specific reports generated by an ecommerce application can vary depending on the business model and goals of the company.



7. SOFTWARE TESTING

7.1 Test Cases & Test Data Design

Test Cases and Test Data Design for a Plantree E-commerce Application can include the following:

User Interface Testing: This involves testing the user interface of the application, such as buttons, links, dropdowns, etc. The test cases can include checking the placement, size, color, and alignment of these elements. Test data can include valid and invalid inputs, different browsers, and devices.

Functionality Testing: This involves testing the functionality of the application, such as adding products to the cart, placing an order, and payment processing. Test cases can include testing different scenarios, such as adding a product with a discount code, canceling an order, and payment with different payment methods. Test data can include valid and invalid inputs, different currencies, and payment gateways.

Performance Testing: This involves testing the performance of the application, such as its speed, scalability, and reliability. Test cases can include testing the application under different loads and checking its response time, memory usage, and CPU usage. Test data can include various sizes of data, different network speeds, and different platforms.

Security Testing: This involves testing the security of the application, such as its authentication and authorization mechanisms, data encryption, and user privacy. Test cases can include testing different scenarios, such as trying to access restricted pages, injecting SQL queries, and testing password strength. Test data can include different types of data, such as sensitive user information and credit card details.

Compatibility Testing: This involves testing the application's compatibility with different browsers, operating systems, and devices. Test cases can include testing the application on different versions of browsers, operating systems, and devices. Test data can include different configurations of hardware and software.

Usability Testing: This involves testing the usability of the application, such as its ease of use, user-friendliness, and accessibility. Test cases can include testing the application's navigation, labeling, and feedback mechanisms. Test data can include different user profiles, such as novice and expert users.

Overall, it's essential to create a comprehensive set of test cases and test data to ensure that the Plantree E-commerce Application works as expected, is secure, and provides an excellent user experience.



7.2 Output Comparison

Output comparison is a technique used in software testing to verify the accuracy and correctness of the output produced by a software application. It involves comparing the actual output generated by the application to the expected output, which is determined based on the input data and the functional requirements of the system.

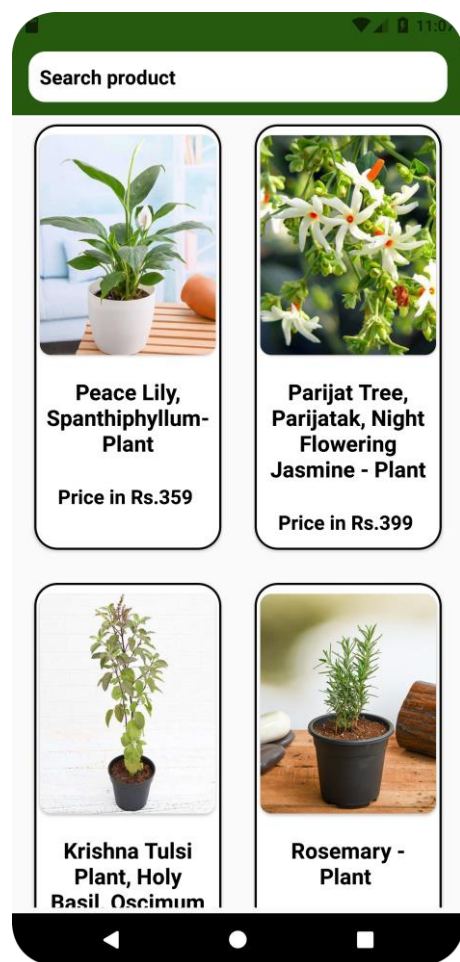
Output comparison is typically performed as part of a larger testing process, such as regression testing or functional testing. Test cases are designed to generate specific input data, and the expected output for each test case is predetermined. Once the test is run, the actual output is compared to the expected output to determine whether the system is functioning correctly.

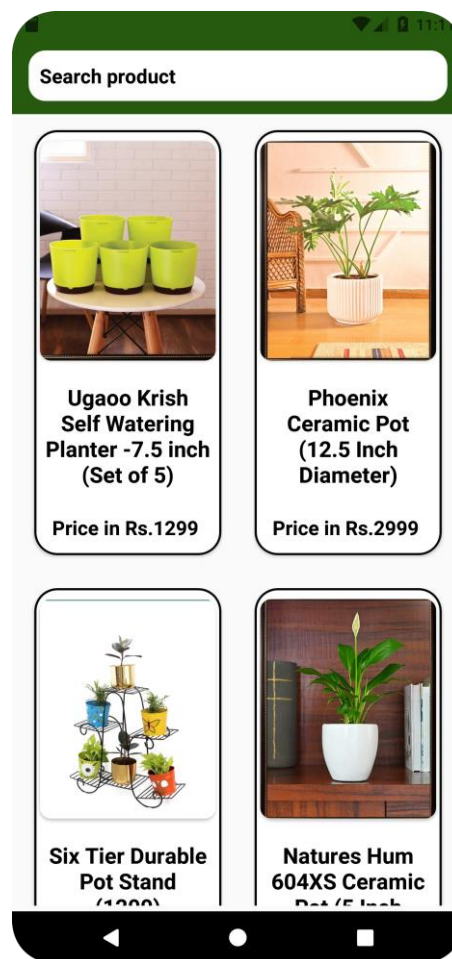
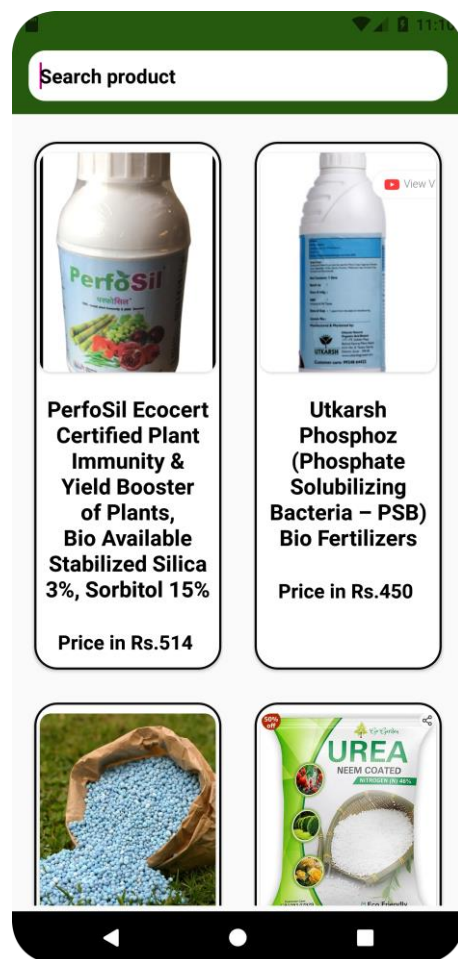
Output comparison is an important technique because it helps to identify defects in the software application. If the actual output does not match the expected output, then it may be an indication that there is a problem with the application's functionality or that there is a defect in the code. By comparing the output, software testers can identify and resolve issues before the application is released to production, thereby ensuring that the software works as intended and meets the requirements of the end-users.



PLANTREE E-COMMERCE APPLICATION

User Side Phase :-

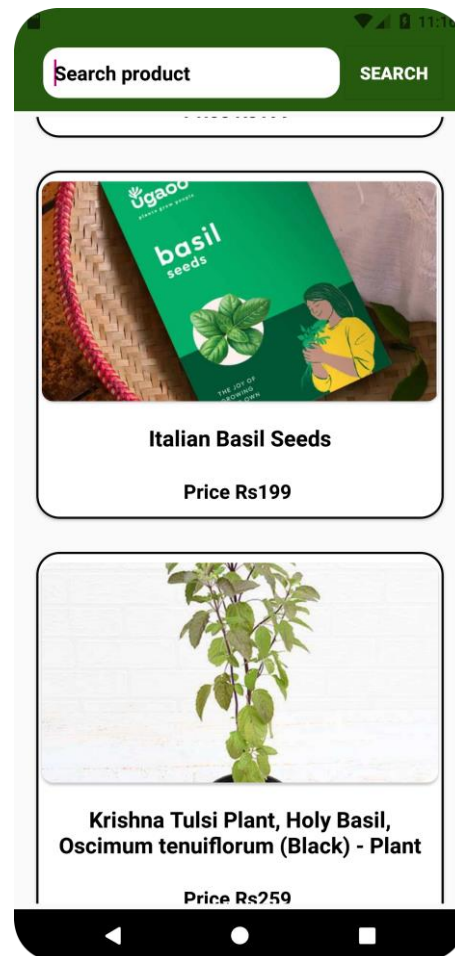
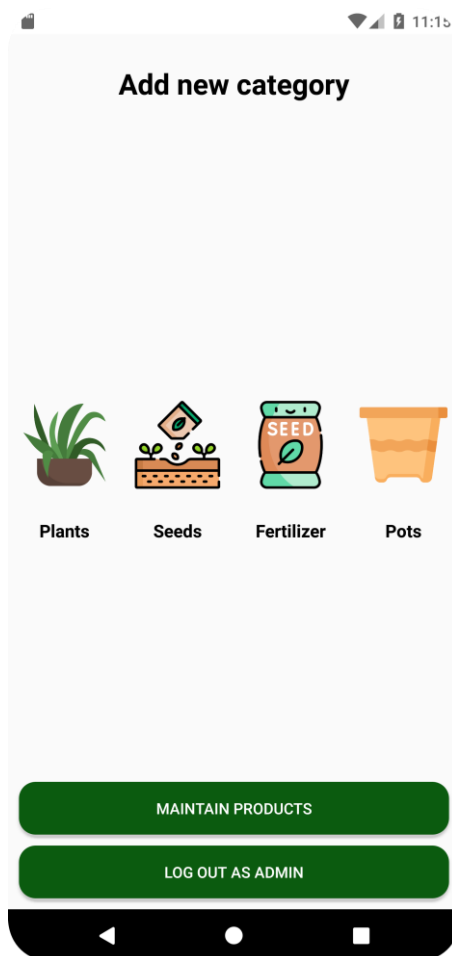






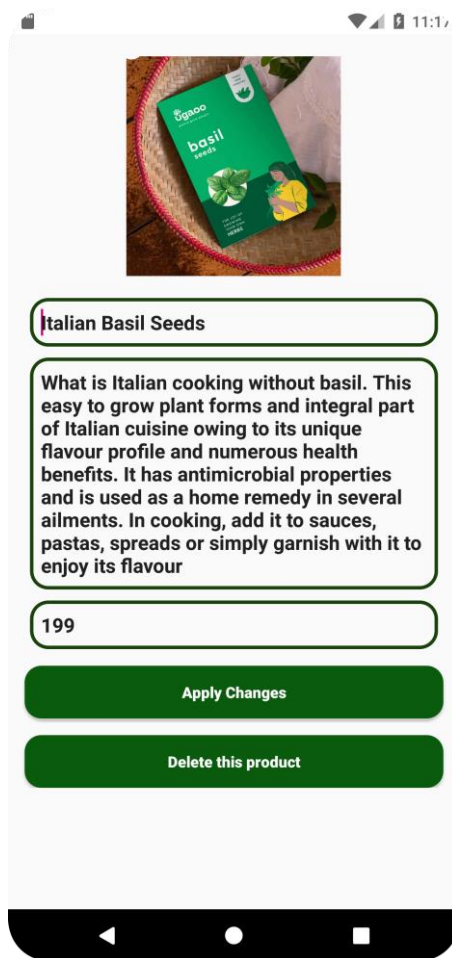
PLANTREE E-COMMERCE APPLICATION

Admin Side Phase : -





PLANTREE E-COMMERCE APPLICATION





7.3 Testing Strategies

A Testing strategies refer to the overall approach and techniques used to test a software application. It involves identifying the scope of testing, creating a plan of action, and defining the methodologies and techniques that will be used to ensure that the software meets the specified requirements and works as intended.

There are several testing strategies that can be used, depending on the nature of the software application and the specific requirements of the project. Some common testing strategies include:

- **Manual testing:** This involves testing the software manually, without the use of automated tools or scripts. Manual testing can be time-consuming and labor-intensive, but it can be useful for testing specific scenarios or functionality.
- **Automated testing:** This involves using tools and scripts to automate the testing process. Automated testing can be more efficient and accurate than manual testing, and it can be useful for testing large or complex software applications.
- **Exploratory testing:** This involves exploring the software application to identify defects or issues that may not be identified through other testing methods. Exploratory testing is often used in conjunction with other testing strategies to ensure that all possible scenarios and edge cases are covered.
- **Regression testing:** This involves testing the software application to ensure that changes or updates to the software do not break existing functionality or introduce new defects. Regression testing is often performed after new features or updates are added to the software.
- **Acceptance testing:** This involves testing the software application to ensure that it meets the requirements and expectations of the end-users. Acceptance testing is typically performed by the end-users themselves, and it is used to ensure that the software meets their needs and is user-friendly.

By using a combination of testing strategies, software testers can ensure that the software application is thoroughly tested and meets the specified requirements and standards.



7.4 Unit Testing

A Unit testing is a type of software testing that involves testing individual units or components of a software application. The purpose of unit testing is to verify that each unit of the software application is working as intended and performing its intended function correctly.

In unit testing, each unit of the application is tested in isolation from the rest of the application. This means that the unit is tested without any dependencies on other parts of the application or external systems. Unit testing typically involves the use of automated testing tools and frameworks to test the individual units of code.

Unit testing is important because it helps to identify defects or issues in the software application early in the development process. By testing each unit of code in isolation, developers can identify and resolve issues before they have a chance to propagate throughout the rest of the application. This can help to reduce the overall cost and effort required for testing and debugging, and can improve the overall quality and reliability of the software application.

Unit testing is often performed by developers themselves, as part of the development process. However, it can also be performed by dedicated software testers or quality assurance professionals to ensure that the software meets the specified requirements and standards.



7.5 Integration Testing

Integration testing is a type of software testing that focuses on testing the interfaces and interactions between different modules or components of a software application. The purpose of integration testing is to verify that the different modules or components of the application are working together correctly and producing the expected results.

In integration testing, different modules or components of the application are combined and tested as a group. This can involve testing the interactions between different software modules, as well as testing the interactions between the software application and external systems or APIs. Integration testing can be performed using both manual and automated testing methods, and can involve testing at various levels of the software stack, including the UI, database, and application logic layers.

Integration testing is important because it helps to identify defects or issues that may arise due to the interactions between different parts of the application. By testing the interactions between different modules or components of the application, developers can identify and resolve issues before they have a chance to propagate throughout the rest of the application. This can help to improve the overall quality and reliability of the software application, and reduce the risk of issues arising in production.

Integration testing is often performed by dedicated software testers or quality assurance professionals, as it requires a deep understanding of the different components of the application and the interactions between them. It is typically performed after unit testing and before system testing, as part of the overall software testing process.



7.6 System Testing

A System testing is a type of software testing that focuses on testing the entire software application as a whole, rather than testing individual components or modules. The purpose of system testing is to verify that the software application meets the specified requirements and works as intended in a real-world environment.

In system testing, the software application is tested as a complete system, including all of its components and interactions with external systems or APIs. This can involve testing the application's functionality, performance, security, and reliability, among other things. System testing is typically performed using both manual and automated testing methods, and can involve testing at various levels of the software stack, including the UI, database, and application logic layers.

System testing is important because it helps to ensure that the software application is ready for deployment in a real-world environment. By testing the entire application as a whole, software testers can identify and resolve issues that may arise due to the interactions between different components or modules of the application. This can help to improve the overall quality and reliability of the software application, and reduce the risk of issues arising in production.

System testing is often performed by dedicated software testers or quality assurance professionals, as it requires a deep understanding of the software application and the various components and interactions between them. It is typically performed after integration testing, as part of the overall software testing process.



7.7 Alpha Testing

Alpha testing is a type of software testing that is performed by a select group of users, typically within the organization developing the software application. The purpose of alpha testing is to identify and resolve any issues or defects in the software application before it is released to a wider audience.

Alpha testing is typically performed in a controlled environment, with the testers using the software application in a simulated real-world environment. This can involve testing the application's functionality, usability, and overall user experience. The testers provide feedback to the development team, who then use this feedback to make improvements to the application before it is released to a wider audience.

Alpha testing is important because it helps to identify any issues or defects in the software application early in the development process, before it is released to the public. This can help to reduce the overall cost and effort required for testing and debugging, and can improve the overall quality and reliability of the software application.

Alpha testing is typically performed by a small group of users who have a deep understanding of the software application and its intended use cases. It is often performed as part of the overall software testing process, before the software application is released to beta testers or the general public.



8. SOFTWARE IMPLIMENTATION

8.1 User Training

Training Objective: The objective of this user training is to equip users with the knowledge and skills required to navigate and utilize the Plantree E-commerce application effectively and efficiently.

Target Audience: This training is designed for all users of the Plantree E-commerce application, including customers, sellers, and administrators.

Training Content: Overview of the Plantree E-commerce Application:

- Introduction to the application interface
 - Explanation of key features and functionalities
 - Understanding of user roles and responsibilities
- **Customer Training:**
- Creating an account and setting up a profile
 - Browsing and searching for products
 - Adding products to the cart and checking out
 - Tracking orders and viewing order history
 - Providing feedback and ratings
- **Seller Training:**
- Creating a seller account and setting up a profile
 - Uploading products and managing inventory
 - Managing orders and shipments
 - Handling customer queries and complaints
 - Monitoring sales performance and analytics



8.2 User Manual / Help / SOP

Welcome to the user manual for Plantree E-commerce application! This guide will walk you through the various features and functionalities of the application, to help you get the most out of your online shopping experience.

Getting Started:

Creating an Account:

To start using Plantree E-commerce application, you need to create an account. You can do this by clicking on the 'Sign Up' button on the homepage of the website or the application. Once you have provided your details, click on the 'Create Account' button. You will receive an email with a verification link to activate your account.

Logging In:

If you already have an account, you can log in to the application by clicking on the 'Log In' button on the homepage. Enter your email address and password, and then click on the 'Log In' button.

Browsing Products:

Once you are logged in, you can browse the various products available on the application. You can browse by category or use the search bar to find a specific product.

Adding Products to Cart:

To add a product to your cart, click on the 'Add to Cart' button on the product page. You can add multiple products to your cart, and then proceed to checkout.

Checkout:

To proceed to checkout, click on the 'Checkout' button in your cart. You will be asked to enter your shipping and payment details. Once you have provided this information, click on the 'Place Order' button to complete your purchase.



9. LIMITATIONS / CONSTRAINTS

Technical issues: E-commerce applications can experience technical issues, such as slow loading times, website crashes, and payment processing errors, which can negatively impact the user experience and lead to lost sales.

Limited functionality: Some e-commerce applications may lack certain features or integrations that are essential for certain businesses or customers. For example, an e-commerce application may not have a built-in inventory management system or support for certain payment methods.

Security concerns: E-commerce applications can be vulnerable to cyber-attacks and data breaches, which can compromise the sensitive information of customers and the business itself.

Customer service challenges: E-commerce businesses may face challenges in providing adequate customer service and support, particularly if they operate primarily online and do not have a physical storefront.

Shipping and logistics issues: E-commerce businesses must ensure that they have a reliable shipping and logistics system in place to ensure that products are delivered to customers on time and in good condition. Shipping delays and damaged products can lead to dissatisfied customers and lost sales.



10.FUTURE ENHANCEMENT PATH -A-HEAD

Some future enhancement paths for e-commerce applications may include:

Improved user experience: E-commerce applications can benefit from improved user interfaces and user experiences to make it easier and more enjoyable for customers to browse and purchase products.

Personalization: E-commerce applications can use machine learning and AI to offer personalized recommendations to customers based on their browsing and purchase history, as well as other data points such as demographic information and preferences.

Integration with emerging technologies: As new technologies such as augmented reality and virtual reality become more mainstream, e-commerce applications can explore ways to integrate these technologies into their platforms to provide customers with more immersive and engaging experiences.

Enhanced mobile capabilities: With more and more people using smartphones to shop online, e-commerce applications should focus on improving their mobile capabilities, such as mobile-optimized interfaces, faster load times, and streamlined checkout processes.

Improved security measures: To address growing concerns around cyber threats and data breaches, e-commerce applications should continue to invest in security measures such as encryption, multi-factor authentication, and regular security audits.

Sustainable practices: As consumers become more conscious of their environmental impact, e-commerce applications can promote sustainable practices, such as offering eco-friendly packaging, partnering with sustainable suppliers, and promoting carbon-neutral shipping options.

Enhanced customer service: E-commerce businesses should focus on improving their customer service and support, such as offering live chat support, implementing a robust returns and refunds policy, and providing personalized follow-up after purchases to ensure customer satisfaction.



11.BIBIOGRAPHY / APPENDIX/REFERENCES

References : -

Websites

<https://www.w3schools.com/>

<https://www.geeksforgeeks.org/>

<https://github.com/>

<https://developer.android.com/>

<https://nurserylive.com/>

Books

1. Krishnan, P.R., Kaila, R.K., Mewari, J.C. and Roy, M.M. (2014) Plant Nursery Management and Plant Nursery Management: Principles and Practices, Central Arid
2. Kumar. N., (1997) Introduction to Horticulture. Raja Lakshmi Publications, 28/5 – 693,Vepamoodu Junction, Nagercoil. Pp.: 15.47-15.50.
3. Landis, T.D., Tinos, R.W., McDonald, S.E., and Barnett, J.P. (1994) Nursery Planning, Development and Management. Vol. 1, the container tree nursery manual. Agriculture Handbook 674. Washington, DC, USA: US Department
4. Wwww. The free dictionary -com/business. copyright (c)2011 Retrieved 2011-09-15
5. Nestor, O.G., John, H. and Steve, H. (undated) The Operational Effectiveness of The Forest Nursery Sector in Leyte, The Philippines. Improving the Triple Bottom Line Return from Small Scale Forestry. Pp. 155-165



12.OTHER SOFTWARE ENGINEERING PRINCIPLES / TOOLS / TECHNIQUES / MODELS / GUIDELINES

Here are some other software engineering principles, tools, techniques, models, and guidelines that can be applied to Plantree E-commerce application:

Agile Development: Agile development is an iterative approach to software development that emphasizes flexibility, collaboration, and customer satisfaction. It can help ensure that the Plantree E-commerce application meets customer needs and adapts to changing requirements.

Continuous Integration/Continuous Delivery (CI/CD): CI/CD is a software development practice that involves continuously integrating code changes into a shared repository and delivering them to production. This can help ensure that changes are tested and deployed quickly and reliably.

Test-Driven Development (TDD): TDD is a software development technique that involves writing tests before writing the code. This can help ensure that the code meets the requirements and is maintainable.

Model-View-Controller (MVC): MVC is a software design pattern that separates the application into three components: the model (data), the view (user interface), and the controller (logic). This can help ensure that the application is modular, maintainable, and extensible.

Object-Oriented Design (OOD): OOD is a software design principle that emphasizes the use of objects to model real-world entities. This can help ensure that the Plantree E-commerce application is easy to understand, maintain, and extend.

Dependency Injection (DI): DI is a software design pattern that involves injecting dependencies into an object rather than having the object create them itself. This can help ensure that the Plantree E-commerce application is modular and extensible.

Code Review: Code review is a process where other developers review the code written by a developer. This can help ensure that the code is maintainable, meets the requirements, and follows best practices.



PLANTREE E-COMMERCE APPLICATION

Version Control: Version control is a system that tracks changes to files over time. This can help ensure that changes can be tracked and reverted if necessary.

Continuous Monitoring: Continuous monitoring is a process that involves monitoring the application and infrastructure for issues and vulnerabilities. This can help ensure that the Plantree E-commerce application is secure and reliable.

User-Centered Design: User-centered design is a process that involves designing the application with the user's needs and goals in mind.

This can help ensure that the Plantree E-commerce application is easy to use and meets the user's needs.